

SOLIDWORKS API (.NET VBA C#) — Examples (7/50):

Compare DimXpert Annotations In Differen-

Create_Advanced_Hole_Example_CSharp

SOLIDWORKS API Documentation (converted from CHM)

Compare DimXpert Annotations in Different Versions of Same Part Example (C#)

This example shows how to compare the DimXpert annotations in different versions of the same part.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\block.sldprt.
//    a. Click DimXpertManager > Auto Dimension Scheme > OK.
//    b. Save the part as block1.sldprt.
//    c. Right-click TXD2@Scheme2, change 0.25mm to 0.5mm, and click OK.
//    d. Save the part as block2.sldprt.
//    e. Open block1.sldprt.
// 2. Verify that c:\temp exists.
// 3. Open the Immediate window.
//
// Postconditions:
// 1. Compares the DimXpert annotations in the parts and creates
//    the report and bitmap image files.
// 2. Examine the Immediate window.
// 3. Open and examine the files in c:\temp\CompareBlocks.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            bool status = false;
            string referenceDocument = null;
            string modifiedDocument = null;
            string reportName = null;
            string reportFolderPath = null;
            int reportSaveOptions = 0;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            referenceDocument = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\block1.sldprt";
            modifiedDocument = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\block2.sldprt";
            reportName = "CompareBlocks";
            reportFolderPath = "c:\\temp";
            reportSaveOptions = (int)sw3DPMISaveOptions_e.swDimXpertData + (int)sw3DPMISaveOptions_e.swReferenceData;
            status = swModelDocExt.Compare3DPMI(referenceDocument, modifiedDocument, reportName, reportFolderPath, reportSaveOptions);
            Debug.Print("Method executed: " + status);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Compare DimXpert Annotations in Different Versions of Same Part Example (VBA)

This example shows how to compare the DimXpert annotations in different versions of the same part.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\block.sldprt.
'    a. Click DimXpertManager > Auto Dimension Scheme > OK.
'    b. Save the part as block1.sldprt.
'    c. Right-click TXD2@Scheme2, change 0.25mm to 0.5mm, and click OK.
'    d. Save the part as block2.sldprt.
'    e. Open block1.sldprt.
' 2. Verify that c:\temp exists.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Compares the DimXpert annotations in the parts and creates
'    the report and bitmap image files.
' 2. Examine the Immediate window.
' 3. Open and examine the files in c:\temp\CompareBlocks.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim status As Boolean
Dim referenceDocument As String
Dim modifiedDocument As String
Dim reportName As String
Dim reportFolderPath As String
Dim reportSaveOptions As Long

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension
    referenceDocument = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\block1.sldprt"
    modifiedDocument = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\block2.sldprt"
    reportName = "CompareBlocks"
    reportFolderPath = "c:\temp"
    reportSaveOptions = sw3DPMISaveOptions_e.swDimXpertData & sw3DPMISaveOptions_e.swReferenceData
    status = swModelDocExt.Compare3DPMI(referenceDocument, modifiedDocument, reportName, reportFolderPath, reportSaveOptions)
    Debug.Print "Method executed: " & status

End Sub

```

Compare DimXpert Annotations in Different Versions of Same Part Example (VB.NET)

This example shows how to compare the DimXpert annotations in different versions of the same part.

```

-----
Preconditions:
1. Open public_documents\samples\tutorial\api\block.sldprt.
   a. Click DimXpertManager > Auto Dimension Scheme > OK.
   b. Save the part as block1.sldprt.
   c. Right-click TXD2@Scheme2, change 0.25mm to 0.5mm, and click OK.
   d. Save the part as block2.sldprt.
   e. Open block1.sldprt.
2. Verify that c:\temp exists.
3. Open the Immediate window.
-----
Postconditions:
1. Compares the DimXpert annotations in the parts and creates
   the report and bitmap image files.
2. Examine the Immediate window.
3. Open and examine the files in c:\temp\CompareBlocks.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim status As Boolean
        Dim referenceDocument As String
        Dim modifiedDocument As String
        Dim reportName As String
        Dim reportFolderPath As String
        Dim reportSaveOptions As Integer

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        referenceDocument = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\block1.sldprt"
        modifiedDocument = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\block2.sldprt"
        reportName = "CompareBlocks"
        reportFolderPath = "c:\temp"
        reportSaveOptions = sw3DPMISaveOptions_e.swDimXpertData & sw3DPMISaveOptions_e.swReferenceData
        status = swModelDocExt.Compare3DPMI(referenceDocument, modifiedDocument, reportName, reportFolderPath, reportSaveOptions)
        Debug.Print("Method executed: " & status)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Compare Selected Objects and Their Persistent Reference IDs Example (C#)

This example shows how to determine if two selected entities are the same entities and if those entities have the same persistent IDs.

NOTE: Using [IModelDocExtension::IsSamePersistentID](#) and [ISldWorks::IsSame](#) might be helpful when your application is passed two entities that it didn't select, and your application needs to know if they're the same entity.

```
//-----
// Preconditions:
// 1. Open a part document is open and select two
// different entities (e.g., face, edge, vertex, etc.).
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Compares the selected entities.
// 2. Examine the code and Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            object selObj1 = null;
            object selObj2 = null;
            object selObjPID1 = null;
            object selObjPID2 = null;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swModelDocExt = swModel.Extension;

            // Get the selected objects
            selObj1 = swSelMgr.GetSelectedObject6(1, -1);
            selObj2 = swSelMgr.GetSelectedObject6(2, -1);
            Debug.Print("Compare the selected objects:");
            switch (swApp.IsSame(selObj1, selObj2))
            {
                case 0:
                    Debug.Print(" Selected objects are not the same.");
                    break;
                case 1:
                    Debug.Print(" Selected objects are the same.");
                    break;
                case -1:
                    Debug.Print(" Unable to determine if selected objects are the same.");
                    break;
            }

            Debug.Print("");

            Debug.Print("Compare the same objects:");
            switch (swApp.IsSame(selObj1, selObj1))
            {
                case 0:
                    Debug.Print(" Selected objects are not the same.");
                    break;
                case 1:
                    Debug.Print(" Selected objects are the same.");
                    break;
                case -1:
                    Debug.Print(" Unable to determine if selected objects are the same.");
                    break;
            }

            Debug.Print("");

            // Get the persistent reference IDs of the
            // selected objects
            selObjPID1 = swModelDocExt.GetPersistReference3(selObj1);
            selObjPID2 = swModelDocExt.GetPersistReference3(selObj2);
            Debug.Print("Compare the persistent reference IDs of the selected objects:");
            switch (swModelDocExt.IsSamePersistentID(selObjPID1, selObjPID2))
            {
                case 0:
                    Debug.Print(" Selected objects do not have the same persistent reference ID.");
                    break;
                case 1:
                    Debug.Print(" Selected objects do have the same persistent reference ID.");
                    break;
                case -1:
                    Debug.Print(" Unable to determine if the selected objects have the same persistent reference ID.");
                    break;
            }

            Debug.Print("");

            Debug.Print("Compare the persistent reference IDs of the same objects:");
            switch (swModelDocExt.IsSamePersistentID(selObjPID1, selObjPID1))
            {
                case 0:
                    Debug.Print(" Selected objects do not have the same persistent reference ID.");
                    break;
                case 1:

```

```
        Debug.Print(" Selected objects do have the same persistent reference ID.");
        break;
    case -1:
        Debug.Print(" Unable to determine if the selected objects have the same persistent reference ID.");
        break;
    }
}
public SldWorks swApp;
}
```

Compare Selected Objects and Their Persistent Reference IDs Example (VBA)

This example shows how to determine if two selected entities are the same entities and if those entities have the same persistent IDs.

NOTE: Using [IModelDocExtension::IsSamePersistentID](#) and [ISldWorks::IsSame](#) might be helpful when your application is passed two entities that it didn't select, and your application needs to know if they're the same entity.

```

'-----
' Preconditions:
' 1. Open a part document and select two different entities
'   (e.g., face, edge, vertex, etc.)
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Compares the selected entities.
' 2. Examine the code and Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim selObj1 As Object
Dim selObj2 As Object
Dim selObjPID1 As Variant
Dim selObjPID2 As Variant

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swModelDocExt = swModel.Extension

    ' Get the selected objects
    Set selObj1 = swSelMgr.GetSelectedObject6(1, -1)
    Set selObj2 = swSelMgr.GetSelectedObject6(2, -1)

    Debug.Print ("Compare the selected objects:")
    Select Case swApp.IsSame(selObj1, selObj2)
        Case 0
            Debug.Print (" Selected objects are not the same.")
        Case 1
            Debug.Print (" Selected objects are the same.")
        Case -1
            Debug.Print (" Unable to determine if selected objects are the same.")
    End Select

    Debug.Print ("")

    Debug.Print ("Compare the same objects:")
    Select Case swApp.IsSame(selObj1, selObj1)
        Case 0
            Debug.Print (" Selected objects are not the same.")
        Case 1
            Debug.Print (" Selected objects are the same.")
        Case -1
            Debug.Print (" Unable to determine if selected objects are the same.")
    End Select

    Debug.Print ("")

    ' Get the persistent reference IDs of the
    ' selected objects
    selObjPID1 = swModelDocExt.GetPersistReference3(selObj1)
    selObjPID2 = swModelDocExt.GetPersistReference3(selObj2)

    Debug.Print ("Compare the persistent reference IDs of the selected objects:")
    Select Case swModelDocExt.IsSamePersistentID(selObjPID1, selObjPID2)
        Case 0
            Debug.Print (" Selected objects do not have the same persistent reference ID.")
        Case 1
            Debug.Print (" Selected objects do have the same persistent reference ID.")
        Case -1
            Debug.Print (" Unable to determine if the selected objects have the same persistent reference ID.")
    End Select

    Debug.Print ("")

    Debug.Print ("Compare the persistent reference IDs of the same objects:")
    Select Case swModelDocExt.IsSamePersistentID(selObjPID1, selObjPID1)
        Case 0
            Debug.Print (" Selected objects do not have the same persistent reference ID.")
        Case 1
    
```

```
        Debug.Print ("  Selected objects do have the same persistent reference ID.")
    Case -1
        Debug.Print ("  Unable to determine if the selected objects have the same persistent reference ID.")
End Select
```

```
End Sub
```

Compare Selected Objects and Their Persistent Reference IDs Example (VB.NET)

This example shows how to determine if two selected entities are the same entities and if those entities have the same persistent IDs.

NOTE: Using [IModelDocExtension::IsSamePersistentID](#) and [ISldWorks::IsSame](#) might be helpful when your application is passed two entities that it didn't select, and your application needs to know if they're the same entity.

```

'-----
' Preconditions:
' 1. Open a part document and select two different entities
'   (e.g., face, edge, vertex, etc.)
' 2. Open the Immediate window.
'-----
' Postconditions:
' 1. Compares the selected entities.
' 2. Examine the code and Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swModelDocExt As ModelDocExtension
        Dim selObj1 As Object
        Dim selObj2 As Object
        Dim selObjPID1 As Object
        Dim selObjPID2 As Object

        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager
        swModelDocExt = swModel.Extension

        ' Get the selected objects
        selObj1 = swSelMgr.GetSelectedObject6(1, -1)
        selObj2 = swSelMgr.GetSelectedObject6(2, -1)

        Debug.Print("Compare the selected objects:")
        Select Case swApp.IsSame(selObj1, selObj2)
            Case 0
                Debug.Print(" Selected objects are not the same.")
            Case 1
                Debug.Print(" Selected objects are the same.")
            Case -1
                Debug.Print(" Unable to determine if selected objects are the same.")
        End Select

        Debug.Print("")

        Debug.Print("Compare the same objects:")
        Select Case swApp.IsSame(selObj1, selObj1)
            Case 0
                Debug.Print(" Selected objects are not the same.")
            Case 1
                Debug.Print(" Selected objects are the same.")
            Case -1
                Debug.Print(" Unable to determine if selected objects are the same.")
        End Select

        Debug.Print("")

        ' Get the persistent reference IDs of the
        ' selected objects
        selObjPID1 = swModelDocExt.GetPersistReference3(selObj1)
        selObjPID2 = swModelDocExt.GetPersistReference3(selObj2)

        Debug.Print("Compare the persistent reference IDs of the selected objects:")
        Select Case swModelDocExt.IsSamePersistentID(selObjPID1, selObjPID2)
            Case 0
                Debug.Print(" Selected objects do not have the same persistent reference ID.")
            Case 1
                Debug.Print(" Selected objects do have the same persistent reference ID.")
            Case -1
                Debug.Print(" Unable to determine if the selected objects have the same persistent reference ID.")
        End Select

        Debug.Print("")

        Debug.Print("Compare the persistent reference IDs of the same objects:")
        Select Case swModelDocExt.IsSamePersistentID(selObjPID1, selObjPID1)
            Case 0
                Debug.Print(" Selected objects do not have the same persistent reference ID.")
            Case 1
                Debug.Print(" Selected objects do have the same persistent reference ID.")
            Case -1
                Debug.Print(" Unable to determine if the selected objects have the same persistent reference ID.")
        End Select

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Get Names of Configurations Using BSTR* C-array Example (C++)

```
// ConfigTraversAddin.cpp : Implementation of CConfigTraversAddin
// Add-in for getting the names of configurations in an active model document
// Demonstrates ModelDoc2::IGetConfigurationNames taking a BSTR* C-array

#include "stdafx.h"
#include "ConfigTraversAddin.h"
#include "BitmapHandler.h"

// CConfigTraversAddin
// This method adds the CommandManager to the SOLIDWORKS Design user-interface
void CConfigTraversAddin::AddCommandManager()
{
    CComPtr<ICommandGroup> icmdGroup;
    CComObject<CBitmapHandler> *iBmp;
    CComObject<CBitmapHandler>::CreateInstance(&iBmp);
    long cmdIndex0, cmdIndex1;
    const int array_size = 2;
    HRESULT hres;

    int* docTypes = new int[array_size];
    docTypes[0] = swDocASSEMBLY;
    docTypes[1] = swDocDRAWING;
    docTypes[2] = swDocPART;

    CComBSTR title;
    title.LoadString(IDS_TOOLBAR_TITLE);

    CComBSTR hint;
    hint.LoadString(IDS_MENU_HINT);

    icmdMgr->CreateCommandGroup(1, title, title, hint, -1, &icmdGroup);

    CComBSTR smallImageFile;
    iBmp->CreateBitmapFileFromResource(IDB_TOOLBAR_SMALL, &smallImageFile);
    icmdGroup->put_SmallIconList(smallImageFile);

    CComBSTR largeImageFile;
    iBmp->CreateBitmapFileFromResource(IDB_TOOLBAR_LARGE, &largeImageFile);
    icmdGroup->put_LargeIconList(largeImageFile);

    CComBSTR largeIconFile;
    iBmp->CreateBitmapFileFromResource(IDB_ICON_LARGE, &largeIconFile);
    icmdGroup->put_LargeMainIcon(largeIconFile);

    CComBSTR smallIconFile;
    iBmp->CreateBitmapFileFromResource(IDB_ICON_SMALL, &smallIconFile);
    icmdGroup->put_SmallMainIcon(smallIconFile);

    CComBSTR tip;
    CComBSTR callback;
    CComBSTR enable;
    long cmdIndex;
    VARIANT_BOOL cmdActivated;

    callback.LoadString(IDS_TOOLBAR_CALLBACK0);
    enable.LoadString(IDS_TOOLBAR_ENABLE0);
    tip.LoadString(IDS_TOOLBAR_TIP0);
    hint.LoadString(IDS_TOOLBAR_HINT0);
    icmdGroup->AddCommandItem(tip, -1, hint, tip, 0, callback, enable, 0, &cmdIndex0);

    icmdGroup->put_HasToolbar(true);
    icmdGroup->put_HasMenu(true);
    icmdGroup->Activate(&cmdActivated);

    for(int i=0; i < array_size + 1; i++)
```

```

{
    CComPtr<ICommandTab> pTab = NULL;
    long TabCount, docType = docTypes[i];
    CComPtr<ICommandTab> AddinTab;
    iCmdMgr->GetCommandTabCount(docType, &TabCount);
    // Check for tab
    iCmdMgr->GetCommandTab(docType, title, &pTab);

    if(pTab == NULL)
    {
        // If no tab, then add one
        iCmdMgr->AddCommandTab(docType, title, &pTab);
        CComPtr<ICommandTabBox> pBox;
        pTab->AddCommandTabBox(&pBox);

        // Create two commands on this tab
        long CommandIDCount = 2;
        long* CommandIDs = new long[2];
        long* TextDisplayStyles = new long[2];
        long cmdID = 0;

        // The two command buttons have different text styles
        iCmdGroup->get_CommandID(cmdIndex0, &cmdID);
        CommandIDs[0] = cmdID;
        TextDisplayStyles[0] = swCommandTabButton_TextHorizontal;

        iCmdGroup->get_ToolbarId(&cmdID);
        CommandIDs[1] = cmdID;

        TextDisplayStyles[1] = swCommandTabButton_TextHorizontal;

        VARIANT_BOOL vbResult = VARIANT_FALSE;
        pBox->IAddCommands(CommandIDCount, CommandIDs, TextDisplayStyles, &vbResult);

        CommandIDCount = 1;
        CommandIDs = new long[1];
        TextDisplayStyles = new long[1];

        iCmdGroup->get_ToolbarId(&cmdID);
        CommandIDs[0] = cmdID;

        TextDisplayStyles[0] = swCommandTabButton_TextBelow | swCommandTabButton_ActionFlyout;

        CComPtr<ICommandTabBox> pBox1;

        pTab->AddCommandTabBox(&pBox1);

        pBox1->IAddCommands(CommandIDCount, CommandIDs, TextDisplayStyles, &vbResult);

        CComPtr<ICommandTabBox> pBoxNew1, pBoxNew2;

        pTab->AddSeparator(pBox1, cmdID, &pBoxNew1);
    }

}

// Clean up
iBmp->Dispose();
iBmp->Release();
}

void CConfigTraversAddin::RemoveCommandManager()
{
    VARIANT_BOOL cmdRemoved;
    iCmdMgr->RemoveCommandGroup(1, &cmdRemoved);
}

// Event handlers

// Called when the active document in SOLIDWORKS Design changes
STDMETHODIMP CConfigTraversAddin::OnDocChange(void)
{

```

```

        // TODO: Add your implementation code here
        return S_OK;
    }

    // Called when a new document is created or a document is loaded
    STDMETHODIMP CConfigTraversAddin::OnDocLoad(BSTR docTitle, BSTR docPath)
    {
        // TODO: Add your implementation code here
        return S_OK;
    }

    // Called when the active model document changes in SOLIDWORKS Design
    STDMETHODIMP CConfigTraversAddin::OnModelDocChange(void)
    {
        // TODO: Add your implementation code here
        return S_OK;
    }

    // Called when a new file is created
    STDMETHODIMP CConfigTraversAddin::OnFileNew(LPDISPATCH newDoc, long docType, BSTR templateName)
    {
        // TODO: Add your implementation code here
        return S_OK;
    }

    // Utility methods

    // Set up the add-in to catch events
    VARIANT_BOOL CConfigTraversAddin::AttachEventHandlers()
    {
        VARIANT_BOOL attached = VARIANT_TRUE;
        this->m_libid = LIBID_SldWorks;
        this->m_wMajorVerNum = GetSldWorksTlbMajor();
        this->m_wMinorVerNum = 0;

        CSldWorksEvents::_tih.m_wMajor = this->m_wMajorVerNum;

        // Connect to the SldWorks event sink
        HRESULT success = this->DispEventAdvise(iSwApp, &__uuidof(DSldWorksEvents));

        if (success != S_OK)
            return VARIANT_FALSE;
        return attached;
    }

    // Stop listening for events
    VARIANT_BOOL CConfigTraversAddin::DetachEventHandlers()
    {
        VARIANT_BOOL detached = VARIANT_TRUE;

        // Disconnect from the SldWorks event sink
        HRESULT success = this->DispEventUnadvise(iSwApp, &__uuidof(DSldWorksEvents));

        CSldWorksEvents::_tih.m_plibid = &GUID_NULL;

        if (success != S_OK)
            return VARIANT_FALSE;
        return detached;
    }

    // ISwAddin Methods

    // This is the starting point for the add-in
    STDMETHODIMP CConfigTraversAddin::ConnectToSw(LPDISPATCH ThisSW, long Cookie, VARIANT_BOOL * IsConnected)
    {
        ThisSW->QueryInterface(__uuidof(ISldWorks), (void**)&iSwApp);
        addinID = Cookie;
        iSwApp->GetCommandManager(Cookie, &iCmdMgr);

        VARIANT_BOOL status = VARIANT_FALSE;

        iSwApp->SetAddinCallbackInfo((long)_AtlBaseModule.GetModuleInstance(), static_cast<IConfigTraversAddin*>(this), addinID, &status);
    }

```

```

// Get the current type library version
{
    USES_CONVERSION;
    CComBSTR bstrNum;
    std::string strNum;
    char *buffer;

    iSwApp->RevisionNumber(&bstrNum);

    strNum = W2A(bstrNum);
    m_swMajNum = strtol(strNum.c_str(), &buffer, 10 );

    m_swMinNum=0;
}

// Create the addin's user-interface
AddCommandManager();
// Listen for events
*IsConnected = AttachEventHandlers();
*IsConnected = VARIANT_TRUE;
return S_OK;
}

STDMETHODIMP CConfigTraversAddin::DisconnectFromSw(VARIANT_BOOL * IsDisconnected)
{
    // Remove the addin's user-interface
    RemoveCommandManager();
    // Stop listening for events
    *IsDisconnected = DetachEventHandlers();

    iCmdMgr.Release();
    // Make sure you release the SOLIDWORKS pointer last
    iSwApp.Release();

    return E_NOTIMPL;
}

// IConfigTraversAddin methods

// Menu and toolbar callbacks

STDMETHODIMP CConfigTraversAddin::ToolbarCallback0(void)
{
    // Use ATL smart pointers
    CComPtr<ISldWorks> swApp;
    CComPtr<IModelDoc2> swModel;

    swApp = iSwApp;

    swApp->get_IActiveDoc2(&swModel);

    if (! swModel) {
        return(S_OK);
    }

    CComBSTR strModelTitle;
    long nDocumentType; // swDocumentTypes_e

    swModel->GetTitle(&strModelTitle);
    swModel->GetType(&nDocumentType);

    long lNumConfigurations;

    swModel->GetConfigurationCount(&lNumConfigurations);

    BSTR* aConfigurationNames = new BSTR[lNumConfigurations];

    swModel->IGetConfigurationNames(&lNumConfigurations, aConfigurationNames);

    for (int i = 0; i < lNumConfigurations; i++) {
        CComBSTR bstrConfigurationName(aConfigurationNames[i]);
    }
}

```

```
}

delete [] aConfigurationNames;

return(S_OK);
}

STDMETHODIMP CConfigTraversAddin::ToolBarEnable0(long* status)
{
    // TODO: Add your implementation code here
    *status = 1;
    return S_OK;
}
```

Get Names of Configurations Using Variant Example (C++)

This example shows how to:

- get the names of the configurations in the active model.
- pass a Variant to IModelDoc2::GetConfigurationNames.
- use the ISafeArrayUtility interface to get the names of the configurations.

```
//-----
// Preconditions:
// 1. Start up SOLIDWORKS Design and open a part or assembly document.
// 2. Start Microsoft Visual Studio 2019 or later.
//   a. Click File > New > Project > (C++ Windows) Console App.
//   b. Click Next.
//   c. Type the name of your project in Project Name.
//   d. Click Create.
//   e. Right-click project, select Add, New Item, Select ATL > ATL Support, Click Add.
//   f. Click Project > projectname Properties > Configuration Properties >
//      C/C++ and type the path to sldworks.tlb and swconst.tlb,
//      typically C:\Program Files\SOLIDWORKS Corp\SOLIDWORKS,
//      in Additional Include Directories.
//   g. Click Apply or OK.
// 3. Replace the code in the code window with This code.
// 4. Click Debug > Start Debugging.
//
// Postconditions:
// 1. Pops up a message box for each configuration in the active
//    model that contains the name of that configuration.
// 2. Click OK to close each message box.
//-----

//This code
#include "framework.h"
#import "sldworks.tlb" raw_interfaces_only, raw_native_types, no_namespace, named_guids // SOLIDWORKS type library
#import "swconst.tlb" raw_interfaces_only, raw_native_types, no_namespace, named_guids // SOLIDWORKS constants type library

int _tmain(int argc, _TCHAR* argv[])
{
    // Initialize COM
    // Do this before using ATL smart pointers so that
    // COM is available
    CoInitialize(NULL);

    // Use a block so that the smart pointers are destructed when
    // the scope of this block is exited
    {
        // Use ATL smart pointers
        CComPtr<ISldWorks> pSwApp;

        if(pSwApp.CoCreateInstance(__uuidof(SldWorks), NULL, CLSCTX_LOCAL_SERVER) != S_OK) {
            return(0);
        }

        pSwApp->put_UserControl(VARIANT_TRUE);
        pSwApp->put_Visible(VARIANT_TRUE);

        CComPtr<IModelDoc2> pSwModel;
        pSwApp->get_IActiveDoc2(&pSwModel);

        if (! pSwModel) {
            return(0);
        }

        CComBSTR strModelTitle;
        long nDocumentType; // swDocumentTypes_e

        pSwModel->GetTitle(&strModelTitle);
        pSwModel->GetType(&nDocumentType);

        VARIANT vConfigurationNames;
        ::VariantInit(&vConfigurationNames);

        // Get the names of the configurations in the active model
        // NOTE: This is an out-of-process client, so you cannot
        // use ModelDoc2::IGetConfigurationNames
        pSwModel->GetConfigurationNames(&vConfigurationNames);

        CComPtr<IDispatch> pDispatchSafeArray = NULL;
        CComPtr<ISafeArrayUtility> pSwSafeArray = NULL;
        HRESULT hres;
        hres = pSwApp->GetSafeArrayUtility(&pDispatchSafeArray);
        hres = pDispatchSafeArray.QueryInterface<ISafeArrayUtility>(&pSwSafeArray);
```

```

    long saCount = 0;
    long saType = 0;
    long* visible = 0;

    long nbrConfigs = 0;
    pSwModel->GetConfigurationCount(&nbrConfigs);

    //Get the name of each configuration
    for (long i = 0; i < nbrConfigs; i++) {
        BSTR configurationName;
        pSwSafeArray->GetBstr(vConfigurationNames, i, &configurationName);
        CComBSTR msg;
        msg = (OLESTR("Name of configuration: "));
        msg.Append(configurationName);
        long res;
        pSwApp->SendMsgToUser2(msg, 0,0, &res);
    }

}

// ATL smart pointers are destructed so that all COM objects
// held on to are released
// Shut down COM because you no longer need it

// Stop COM
CoUninitialize();

return(0);
}

```

Get Names of Configurations Example (C++/CLI)

```
// ConfigurationTraversal.cpp : main project file
// Get names of configurations in the active model document

#include "stdafx.h"

using namespace System;

// Add the SOLIDWORKS primary interop assemblies to the references
using namespace SOLIDWORKS::Interop::sldworks;
using namespace SOLIDWORKS::Interop::swconst;

int main(array<System::String ^> ^args)
{
    SldWorks^    swApp;

    swApp = gcnew SldWorksClass;

    if (! swApp) {
        return(0);
    }

    IModelDoc2^  swModel;
    swModel = swApp->IActiveDoc2;

    if (! swModel) {
        return(0);
    }

    String^      strModelTitle;
    swDocumentTypes_e  nDocumentType;

    strModelTitle = swModel->GetTitle();
    nDocumentType = (swDocumentTypes_e)swModel->GetType();

    array<String^>^ aConfigurationNames;

    aConfigurationNames = safe_cast<array<String^>^>(swModel->GetConfigurationNames());

    for each (String^ strConfigurationName in aConfigurationNames) {
        System::Console::WriteLine(strConfigurationName);
    }

    return(0);
}
```

Connect to SOLIDWORKS Design from Third-party Application Example (C++)

This example shows how to connect to SOLIDWORKS Design from a third-party application using C++.

```
// HelloSW.cpp : Defines the entry point for the console application.
//
#include "stdafx.h"
#include "objbase.h"

#import "sldworks.tlb" no_namespace raw_interfaces_only

int _tmain(int argc, _TCHAR* argv[])
{
    ::CoInitialize(NULL);
    {
        CLSID clsid;
        ::CLSIDFromProgID(L"SldWorks.Application", &clsid);

        ISldWorks *swApp = NULL;
        ::CoCreateInstance(clsid, NULL, CLSCTX_LOCAL_SERVER, __uuidof(ISldWorks), (void**)&swApp);
        if (swApp != NULL)
        {
            BSTR bstrHello = ::SysAllocString(L"Hello from SOLIDWORKS.");
            swApp->SendMessageToUser(bstrHello);

            ::SysFreeString(bstrHello);
            swApp->Release();
        }
    }
    ::CoUninitialize();
    return 0;
}
```

Connect to SOLIDWORKS Session Example (C#)

This example shows how to connect to a SOLIDWORKS Design session.

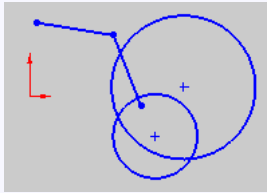
using System;

```
namespace ConsoleApplication1
{
    /// <summary>
    /// Summary description for Class1
    /// </summary>
    class Class1
    {
        /// <summary>
        /// Main entry point for the application
        /// </summary>
        [STAThread]
        static void Main(string[] args)
        {
            //
            // To do: Add code to start application here
            //
            SldWorks.SldWorks swApp = new SldWorks.SldWorksClass();
            SldWorks.ModelDoc2 swDoc = swApp.IActiveDoc2;
            swDoc.Extension.SelectByID2("Top Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swDoc.InsertSketch2(false);
            swDoc.SketchRectangle(0, 0.04, 0, 0.01, 0, 0, true);
            swDoc.FeatureManager.FeatureExtrusion(true, false, false, 0, 0, 0.09, 0, false, false, false, false, 0.0, 0.0, false, false, false, false, true, false, false);
        }
    }
}
```

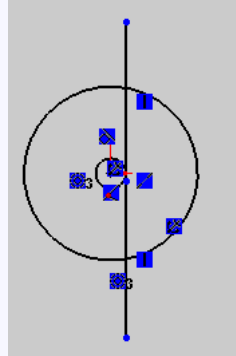
Constrain Sketch Example (C#)

This example shows how to fully constrain a sketch.

Before constraining the sketch



After constraining the sketch



```
//-----
// Preconditions: Before constraining the sketch sketch exists.
//
// Postconditions: Fully constrains the sketch, which looks like
// After constraining the sketch.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System.Windows.Forms;
namespace ConstrainSketch_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        SketchManager swSketchMgr;
        Sketch swSketch;
        SelectionMgr swSelMgr;
        Feature swFeat;
        long nSketchStatus;
        bool boolstatus;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;

            // Is a model document active?
            if (swModel == null)
            {
                swApp.SendMsgToUser2("A part document must be open and the active document.", (int)swMessageBoxIcon_e.swMbWarning,
                (int)swMessageBoxBtn_e.swMbOk);
                return;
            }

            // Is it a part document?
            long modelType = 0;
            modelType = swModel.GetType();

            if (modelType != (int)swDocumentTypes_e.swDocPART)
            {
                swApp.SendMsgToUser2("A part document must be open and the active document.", (int)swMessageBoxIcon_e.swMbWarning,
                (int)swMessageBoxBtn_e.swMbOk);
                return;
            }

            swSketchMgr = swModel.SketchManager;
            swSketch = swSketchMgr.ActiveSketch;

            if (swSketch == null)
            {
                swApp.SendMsgToUser2("No active sketch; thus, a sketch could not be selected.",
                (int)swMessageBoxIcon_e.swMbWarning, (int)swMessageBoxBtn_e.swMbOk);
            }
        }
    }
}
```

```

        return;
    }

    // Select the lines and make them colinear and vertical
    boolstatus = swModel.Extension.SelectByID("Line2", "SKETCHSEGMENT", 0.02116924482339, 0.04904427527406, 0, false, 0,
null, 0);
    boolstatus = swModel.Extension.SelectByID("Line3", "SKETCHSEGMENT", 0.06508556638246, 0.02563976857491, 0, true, 0,
null, 0);

    swModel.SketchAddConstraints("sgCOLINEAR");
    swModel.SketchAddConstraints("sgVERTICAL2D");

    MessageBox.Show("The lines have been selected, made colinear, and vertically constrained.");
    swModel.ClearSelection2(true);

    //Select the center of the circles and constrain them to the origin
    boolstatus = swModel.Extension.SelectByID("Point7", "SKETCHPOINT", 0.1074240560292, 0.006179841656516, 0, false, 0,
null, 0);
    boolstatus = swModel.Extension.SelectByID("Point1@Origin", "EXTSKETCHPOINT", 0, 0, 0, true, 0, null, 0);

    swModel.SketchAddConstraints("sgCOINCIDENT");
    MessageBox.Show("The center of the circles and the origin were selected and made coincident");
    swModel.ClearSelection2(true);

    // Select a line and the circle and make them tangent
    boolstatus = swModel.Extension.SelectByID("Line2", "SKETCHSEGMENT", 0.005390925700365, 0.009861449451888, 0, false, 0,
null, 0);
    boolstatus = swModel.Extension.SelectByID("Arc1", "SKETCHSEGMENT", -0.01222819732034, 0.04720347137637, 0, true, 0,
null, 0);

    swModel.SketchAddConstraints("sgTANGENT");
    MessageBox.Show("One line and a circle were selected; both lines are now tangent with the circle.");
    swModel.ClearSelection2(true);

    //Select the circles and make them concentric
    boolstatus = swModel.Extension.SelectByID("Arc2", "SKETCHSEGMENT", -0.0290584043849, 0.03116218026797, 0, false, 0,
null, 0);
    boolstatus = swModel.Extension.SelectByID("Arc1", "SKETCHSEGMENT", -0.01222819732034, 0.04720347137637, 0, true, 0,
null, 0);

    swModel.SketchAddConstraints("sgCONCENTRIC");
    MessageBox.Show("The circles have been selected and made concentric.");
    swModel.ClearSelection2(true);

    //Select all the sketch entities and fix their positions
    MessageBox.Show("All sketch entities will be selected and made fixed to fully constrain the sketch.");
    boolstatus = swModel.Extension.SelectByID("Line2", "SKETCHSEGMENT", 0.02116924482339, 0.04904427527406, 0, false, 0,
null, 0);
    boolstatus = swModel.Extension.SelectByID("Line3", "SKETCHSEGMENT", 0.06508556638246, 0.02563976857491, 0, true, 0,
null, 0);
    boolstatus = swModel.Extension.SelectByID("Arc2", "SKETCHSEGMENT", -0.0290584043849, 0.03116218026797, 0, false, 0,
null, 0);
    boolstatus = swModel.Extension.SelectByID("Arc1", "SKETCHSEGMENT", -0.01222819732034, 0.04720347137637, 0, true, 0,
null, 0);

    swModel.SketchAddConstraints("sgFIXED");
    swModel.ClearSelection2(true);
}

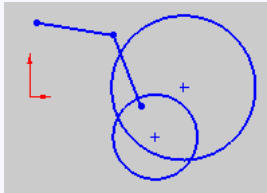
public SldWorks swApp;
}
}

```

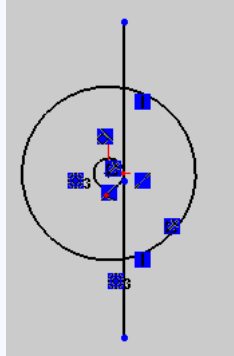
Constrain Sketch Example (VBA)

This example shows how to fully constrain a sketch.

Before constraining the sketch



After constraining the sketch



```
'-----
' Preconditions: Before constraining the sketch sketch exists.
'
' Postconditions: Fully constrains the sketch, which looks like
' After constraining the sketch.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swSketchMgr As SldWorks.SketchManager
Dim swSketch As SldWorks.Sketch
Dim swSelMgr As SldWorks.SelectionMgr
Dim swFeat As SldWorks.Feature
Dim nSketchStatus As Long
Dim boolstatus As Boolean
```

Sub main()

```
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    ' Is a model document active?
    If swModel Is Nothing Then
        swApp.SendMsgToUser2 "A part document must be open and the active document.", swMbWarning, swMbOk
        Exit Sub
    End If

    ' Is it a part document?
    Dim modelType As Long
    modelType = swModel.GetType
    If modelType <> SwConst.swDocPART Then
        swApp.SendMsgToUser2 "A part document must be open and the active document.", swMbWarning, swMbOk
        Exit Sub
    End If

    Set swSketchMgr = swModel.SketchManager
    Set swSketch = swSketchMgr.ActiveSketch

    If swSketch Is Nothing Then
        swApp.SendMsgToUser2 "No active sketch; thus, a sketch could not be selected.", swMbWarning, swMbOk
        Exit Sub
    End If

    ' Select the lines and make them colinear and vertical
    boolstatus = swModel.Extension.SelectByID2("Line2", "SKETCHSEGMENT", 0.02116924482339, 0.04904427527406, 0, False, 0, Nothing, 0)
    boolstatus = swModel.Extension.SelectByID2("Line3", "SKETCHSEGMENT", 0.06508556638246, 0.02563976857491, 0, True, 0, Nothing, 0)
    swModel.SketchAddConstraints "sgCOLINEAR"
    swModel.SketchAddConstraints "sgVERTICAL2D"
```

MsgBox "The lines have been selected, made colinear, and vertically constrained."

swModel.ClearSelection2 True

'Select the center of the circles and constrain them to the origin

boolstatus = swModel.Extension.SelectByID2("Point7", "SKETCHPOINT", 0.1074240560292, 0.006179841656516, 0, False, 0, Nothing, 0)

boolstatus = swModel.Extension.SelectByID2("Point1@Origin", "EXTSKETCHPOINT", 0, 0, 0, True, 0, Nothing, 0)

swModel.SketchAddConstraints "sgCOINCIDENT"

MsgBox "The center of the circles and the origin were selected and made coincident"

swModel.ClearSelection2 True

' Select a line and the circle and make them tangent

boolstatus = swModel.Extension.SelectByID2("Line2", "SKETCHSEGMENT", 0.005390925700365, 0.009861449451888, 0, False, 0, Nothing, 0)

boolstatus = swModel.Extension.SelectByID2("Arc1", "SKETCHSEGMENT", -0.01222819732034, 0.04720347137637, 0, True, 0, Nothing, 0)

swModel.SketchAddConstraints "sgTANGENT"

MsgBox "One line and a circle were selected; both lines are now tangent with the circle."

swModel.ClearSelection2 True

'Select the circles and make them concentric

boolstatus = swModel.Extension.SelectByID2("Arc2", "SKETCHSEGMENT", -0.0290584043849, 0.03116218026797, 0, False, 0, Nothing, 0)

boolstatus = swModel.Extension.SelectByID2("Arc1", "SKETCHSEGMENT", -0.01222819732034, 0.04720347137637, 0, True, 0, Nothing, 0)

swModel.SketchAddConstraints "sgCONCENTRIC"

MsgBox "The circles have been selected and made concentric."

swModel.ClearSelection2 True

'Select all the sketch entities and fix their positions

MsgBox "All sketch entities will be selected and made fixed to fully constrain the sketch."

boolstatus = swModel.Extension.SelectByID2("Line2", "SKETCHSEGMENT", 0.02116924482339, 0.04904427527406, 0, False, 0, Nothing, 0)

boolstatus = swModel.Extension.SelectByID2("Line3", "SKETCHSEGMENT", 0.06508556638246, 0.02563976857491, 0, True, 0, Nothing, 0)

boolstatus = swModel.Extension.SelectByID2("Arc2", "SKETCHSEGMENT", -0.0290584043849, 0.03116218026797, 0, False, 0, Nothing, 0)

boolstatus = swModel.Extension.SelectByID2("Arc1", "SKETCHSEGMENT", -0.01222819732034, 0.04720347137637, 0, True, 0, Nothing, 0)

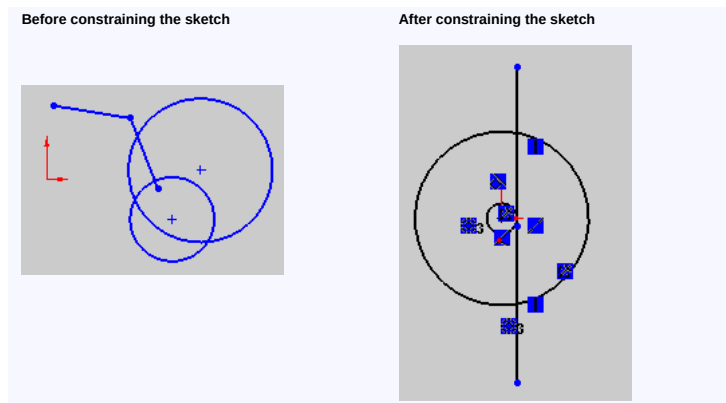
swModel.SketchAddConstraints "sgFIXED"

swModel.ClearSelection2 True

End Sub

Constrain Sketch Example (VB.NET)

This example shows how to fully constrain a sketch.



Preconditions: **Before constraining the sketch** sketch exists.

Postconditions: Fully constrains the sketch, which looks like
After constraining the sketch.

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim swModel As ModelDoc2
    Dim swSketchMgr As SketchManager
    Dim swSketch As Sketch
    Dim swSelMgr As SelectionMgr
    Dim swFeat As Feature
    Dim nSketchStatus As Long
    Dim boolstatus As Boolean
```

```
    Sub main()
```

```
        swModel = swApp.ActiveDoc
```

```
        ' Is a model document active?
        If swModel Is Nothing Then
```

```
            swApp.SendMsgToUser2("A part document must be open and the active document.", swMessageBoxIcon_e.swMbWarning,
            swMessageBoxBtn_e.swMbOK)
            Exit Sub
```

```
        End If
```

```
        ' Is it a part document?
        Dim modelType As Long
        modelType = swModel.GetType
```

```
        If modelType <> swDocumentTypes_e.swDocPART Then
```

```
            swApp.SendMsgToUser2("A part document must be open and the active document.", swMessageBoxIcon_e.swMbWarning,
            swMessageBoxBtn_e.swMbOK)
            Exit Sub
```

```
        End If
```

```
        swSketchMgr = swModel.SketchManager
        swSketch = swSketchMgr.ActiveSketch
```

```
        If swSketch Is Nothing Then
```

```
            swApp.SendMsgToUser2("No active sketch; thus, a sketch could not be selected.", swMessageBoxIcon_e.swMbWarning,
            swMessageBoxBtn_e.swMbOK)
            Exit Sub
```

```
        End If
```

```
        ' Select the lines and make them colinear and vertical
```

```

        boolstatus = swModel.Extension.SelectByID2("Line2", "SKETCHSEGMENT", 0.02116924482339, 0.04904427527406, 0, False, 0,
Nothing, 0)
        boolstatus = swModel.Extension.SelectByID2("Line3", "SKETCHSEGMENT", 0.06508556638246, 0.02563976857491, 0, True, 0,
Nothing, 0)

        swModel.SketchAddConstraints("sgCOLINEAR")
        swModel.SketchAddConstraints("sgVERTICAL2D")

        MsgBox("The lines have been selected, made colinear, and vertically constrained.")
        swModel.ClearSelection2(True)

        'Select the center of the circles and constrain them to the origin
        boolstatus = swModel.Extension.SelectByID2("Point7", "SKETCHPOINT", 0.1074240560292, 0.006179841656516, 0, False, 0,
Nothing, 0)
        boolstatus = swModel.Extension.SelectByID2("Point1@Origin", "EXTSKETCHPOINT", 0, 0, 0, True, 0, Nothing, 0)

        swModel.SketchAddConstraints("sgCOINCIDENT")
        MsgBox("The center of the circles and the origin were selected and made coincident")
        swModel.ClearSelection2(True)

        ' Select a line and the circle and make them tangent
        boolstatus = swModel.Extension.SelectByID2("Line2", "SKETCHSEGMENT", 0.005390925700365, 0.009861449451888, 0, False, 0,
Nothing, 0)
        boolstatus = swModel.Extension.SelectByID2("Arc1", "SKETCHSEGMENT", -0.01222819732034, 0.04720347137637, 0, True, 0,
Nothing, 0)

        swModel.SketchAddConstraints("sgTANGENT")
        MsgBox("One line and a circle were selected; both lines are now tangent with the circle.")
        swModel.ClearSelection2(True)

        'Select the circles and make them concentric
        boolstatus = swModel.Extension.SelectByID2("Arc2", "SKETCHSEGMENT", -0.0290584043849, 0.03116218026797, 0, False, 0,
Nothing, 0)
        boolstatus = swModel.Extension.SelectByID2("Arc1", "SKETCHSEGMENT", -0.01222819732034, 0.04720347137637, 0, True, 0,
Nothing, 0)

        swModel.SketchAddConstraints("sgCONCENTRIC")
        MsgBox("The circles have been selected and made concentric.")
        swModel.ClearSelection2(True)

        'Select all the sketch entities and fix their positions
        MsgBox("All sketch entities will be selected and made fixed to fully constrain the sketch.")
        boolstatus = swModel.Extension.SelectByID2("Line2", "SKETCHSEGMENT", 0.02116924482339, 0.04904427527406, 0, False, 0,
Nothing, 0)
        boolstatus = swModel.Extension.SelectByID2("Line3", "SKETCHSEGMENT", 0.06508556638246, 0.02563976857491, 0, True, 0,
Nothing, 0)
        boolstatus = swModel.Extension.SelectByID2("Arc2", "SKETCHSEGMENT", -0.0290584043849, 0.03116218026797, 0, False, 0,
Nothing, 0)
        boolstatus = swModel.Extension.SelectByID2("Arc1", "SKETCHSEGMENT", -0.01222819732034, 0.04720347137637, 0, True, 0,
Nothing, 0)

        swModel.SketchAddConstraints("sgFIXED")
        swModel.ClearSelection2(True)

End Sub

Public swApp As SldWorks

End Class

```

Convert Curves into 3D Sketches Example (C#)

This example:

- Shows how to convert curves (edges) into 3D sketches.
- Uses IModelDoc2::SketchConvertIsoCurves to extract ISO-parametric (UV) curves from a face or surface. Specifically, this code shows how to extract the curves containing a vertex.

```
//-----
// Preconditions:
// 1. Open a part or fully resolved assembly.
// 2. Select a face.
// 3. Press the Ctrl key and select a vertex.
//
// Postconditions:
// 1. Generates two 3D sketches:
//    * First 3D sketch is edge of face in V direction
//    * Second 3D sketch is edge of face in U direction
//    * from the selected vertex.
// 2. Examine the graphics area and FeatureManager design
//    tree.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            SelectData swSelData = default(SelectData);
            Face2 swFace = default(Face2);
            Vertex swVertex = default(Vertex);
            Entity swFaceEnt = default(Entity);
            Entity swVertexEnt = default(Entity);
            bool bRet = false;

            swModel = swApp.ActiveDoc;

            swSelMgr = swModel.SelectionManager;
            swSelData = swSelMgr.CreateSelectData();
            swFace = swSelMgr.GetSelectedObject6(1, -1);
            swVertex = swSelMgr.GetSelectedObject6(2, -1);

            swFaceEnt = swFace;
            swVertexEnt = swVertex;

            swModel.ClearSelection2(true);

            bRet = swFaceEnt.Select4(true, swSelData);
            bRet = swVertexEnt.Select4(true, swSelData);

            swModel.SketchConvertIsoCurves(100.0, false, true, true);
            swModel.SketchConvertIsoCurves(100.0, true, true, true);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Convert Curves into 3D Sketches Example (VBA)

This example:

- Shows how to convert curves (edges) into 3D sketches.
- Uses IModelDoc2::SketchConvertIsoCurves to extract ISO-parametric (UV) curves from a face or surface. Specifically, this code shows how to extract the curves containing a vertex.

```
'-----  
' Preconditions:  
' 1. Open a part or fully resolved assembly.  
' 2. Select a face.  
' 3. Press the Ctrl key and select a vertex.  
'  
' Postconditions:  
' 1. Generates two 3D sketches:  
'   * First 3D sketch is edge of face in V direction  
'     from the selected vertex.  
'   * Second 3D sketch is edge of face in U direction  
'     from the selected vertex.  
' 2. Examine the graphics area and FeatureManager design  
'    tree.  
'-----
```

Option Explicit

Sub main()

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swSelData As SldWorks.SelectData  
Dim swFace As SldWorks.Face2  
Dim swVertex As SldWorks.Vertex  
Dim swFaceEnt As SldWorks.Entity  
Dim swVertexEnt As SldWorks.Entity  
Dim bRet As Boolean  
  
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc  
  
Set swSelMgr = swModel.SelectionManager  
Set swSelData = swSelMgr.CreateSelectData  
Set swFace = swSelMgr.GetSelectedObject6(1, -1)  
Set swVertex = swSelMgr.GetSelectedObject6(2, -1)  
  
Set swFaceEnt = swFace  
Set swVertexEnt = swVertex  
  
swModel.ClearSelection2 True  
  
bRet = swFaceEnt.Select4(True, swSelData)  
bRet = swVertexEnt.Select4(True, swSelData)  
  
swModel.SketchConvertIsoCurves 100#, False, True, True  
swModel.SketchConvertIsoCurves 100#, True, True, True
```

End Sub

Convert Curves into 3D Sketches Example (VB.NET)

This example:

- Shows how to convert curves (edges) into 3D sketches.
- Uses IModelDoc2::SketchConvertIsoCurves to extract ISO-parametric (UV) curves from a face or surface. Specifically, this code shows how to extract the curves containing a vertex.

```
-----  
Preconditions:  
1. Open a part or fully resolved assembly.  
2. Select a face.  
3. Press the Ctrl key and select a vertex.  
-----  
Postconditions:  
1. Generates two 3D sketches:  
   * First 3D sketch is edge of face in V direction  
     from the selected vertex.  
   * Second 3D sketch is edge of face in U direction  
     from the selected vertex.  
2. Examine the graphics area and FeatureManager design  
   tree.  
-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System
```

Partial Class SolidWorksMacro

```
Public Sub main()  
    Dim swModel As ModelDoc2  
    Dim swSelMgr As SelectionMgr  
    Dim swSelData As SelectData  
    Dim swFace As Face2  
    Dim swVertex As Vertex  
    Dim swFaceEnt As Entity  
    Dim swVertexEnt As Entity  
    Dim bRet As Boolean  
  
    swModel = swApp.ActiveDoc  
  
    swSelMgr = swModel.SelectionManager  
    swSelData = swSelMgr.CreateSelectData  
    swFace = swSelMgr.GetSelectedObject6(1, -1)  
    swVertex = swSelMgr.GetSelectedObject6(2, -1)  
  
    swFaceEnt = swFace  
    swVertexEnt = swVertex  
  
    swModel.ClearSelection2(True)  
  
    bRet = swFaceEnt.Select4(True, swSelData)  
    bRet = swVertexEnt.Select4(True, swSelData)  
  
    swModel.SketchConvertIsoCurves(100.0#, False, True, True)  
    swModel.SketchConvertIsoCurves(100.0#, True, True, True)
```

End Sub

```
''' <summary>  
''' The SldWorks swApp variable is pre-assigned for you.  
''' </summary>  
Public swApp As SldWorks
```

End Class

Convert Drawing Views to Sketch Blocks Example (C#)

This example shows how to convert drawing views to sketches and sketch blocks.

```
//-----
// Preconditions: Open public_documents\samples\tutorial\api\ReplaceView.slddrw
//
// Postconditions:
// 1. Converts Drawing View1 to a sketch.
// 2. Converts Drawing View2 to a sketch block
// 3. Converts Drawing View3 to a sketch block at a new position in the drawing.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace ConvertViewToSketch_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        bool boolstatus;
        DrawingDoc drawDoc;
        SelectionMgr selMan;
        View drview;
        double[] nPt = new double[3];
        object vPt;
        MathUtility swMathUtil;
        MathPoint insertionPt;

        MathPoint position;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            drawDoc = (DrawingDoc)Part;
            swMathUtil = (MathUtility)swApp.GetMathUtility();
            selMan = (SelectionMgr)Part.SelectionManager;

            boolstatus = Part.Extension.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, false, 0, null, 0);
            drview = (View)selMan.GetSelectedObject6(1, 0);
            boolstatus = drview.ReplaceViewWithSketch();

            Part.ClearSelection2(true);
            boolstatus = Part.Extension.SelectByID2("Drawing View2", "DRAWINGVIEW", 0, 0, 0, false, 0, null, 0);
            drview = (View)selMan.GetSelectedObject6(1, 0);

            nPt[0] = 1.41;
            nPt[1] = 3.88;
            nPt[2] = 0;
            vPt = nPt;
            insertionPt = (MathPoint)swMathUtil.CreatePoint(vPt);
            boolstatus = drview.ReplaceViewWithBlock(insertionPt);

            Part.ClearSelection2(true);
            boolstatus = Part.Extension.SelectByID2("Drawing View3", "DRAWINGVIEW", 0, 0, 0, false, 0, null, 0);
            drview = (View)selMan.GetSelectedObject6(1, 0);

            nPt[0] = 5.48;
            nPt[1] = 5.22;
            nPt[2] = 0;
            vPt = nPt;
            position = (MathPoint)swMathUtil.CreatePoint(vPt);
            boolstatus = drview.InsertViewAsBlock(insertionPt, position);

        }

        public SldWorks swApp;
    }
}
```

Convert Drawing Views to Sketch Blocks Example (VBA)

This example shows how to convert drawing views to sketches and sketch blocks.

```
'-----
' Preconditions: Open public_documents\samples\tutorial\api\ReplaceView.slddrw
'
' Postconditions:
' 1. Converts Drawing View1 to a sketch.
' 2. Converts Drawing View2 to a sketch block
' 3. Converts Drawing View3 to a sketch block at a new position in the drawing.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim boolstatus As Boolean
Dim drawDoc As SldWorks.DrawingDoc
Dim selMan As SldWorks.SelectionMgr
Dim drview As SldWorks.View
Dim nPt(2) As Double
Dim vPt As Variant
Dim swMathUtil As SldWorks.MathUtility
Dim insertionPt As SldWorks.MathPoint
Dim position As SldWorks.MathPoint

Sub main()

    Set swApp = Application.SldWorks

    Set Part = swApp.ActiveDoc
    Set drawDoc = Part
    Set swMathUtil = swApp.GetMathUtility
    Set selMan = Part.SelectionManager

    boolstatus = Part.Extension.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
    Set drview = selMan.GetSelectedObject6(1, 0)
    boolstatus = drview.ReplaceViewWithSketch

    Part.ClearSelection2 True
    boolstatus = Part.Extension.SelectByID2("Drawing View2", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
    Set drview = selMan.GetSelectedObject6(1, 0)

    nPt(0) = 1.41
    nPt(1) = 3.88
    nPt(2) = 0
    vPt = nPt
    Set insertionPt = swMathUtil.CreatePoint(vPt)
    boolstatus = drview.ReplaceViewWithBlock(insertionPt)

    Part.ClearSelection2 True
    boolstatus = Part.Extension.SelectByID2("Drawing View3", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
    Set drview = selMan.GetSelectedObject6(1, 0)

    nPt(0) = 5.48
    nPt(1) = 5.22
    nPt(2) = 0
    vPt = nPt
    Set position = swMathUtil.CreatePoint(vPt)
    boolstatus = drview.InsertViewAsBlock(insertionPt, position)

End Sub
```

Convert Drawing Views to Sketch Blocks Example (VB.NET)

This example shows how to convert drawing views to sketches and sketch blocks.

```

'-----
' Preconditions: Open public_documents\samples\tutorial\api\ReplaceView.slddrw
'
' Postconditions:
' 1. Converts Drawing View1 to a sketch.
' 2. Converts Drawing View2 to a sketch block
' 3. Converts Drawing View3 to a sketch block at a new position in the drawing.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

```

```

Partial Class SolidWorksMacro

```

```

    Dim Part As ModelDoc2
    Dim boolstatus As Boolean
    Dim drawDoc As DrawingDoc
    Dim selMan As SelectionMgr
    Dim drview As View
    Dim nPt(2) As Double
    Dim vPt As Object
    Dim swMathUtil As MathUtility
    Dim insertionPt As MathPoint
    Dim position As MathPoint

```

```

    Sub main()

```

```

        Part = swApp.ActiveDoc
        drawDoc = Part
        swMathUtil = swApp.GetMathUtility
        selMan = Part.SelectionManager

```

```

        boolstatus = Part.Extension.SelectByID2("Drawing View1", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
        drview = selMan.GetSelectedObject6(1, 0)
        boolstatus = drview.ReplaceViewWithSketch

```

```

        Part.ClearSelection2(True)
        boolstatus = Part.Extension.SelectByID2("Drawing View2", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
        drview = selMan.GetSelectedObject6(1, 0)

```

```

        nPt(0) = 1.41
        nPt(1) = 3.88
        nPt(2) = 0
        vPt = nPt
        insertionPt = swMathUtil.CreatePoint(vPt)
        boolstatus = drview.ReplaceViewWithBlock(insertionPt)

```

```

        Part.ClearSelection2(True)
        boolstatus = Part.Extension.SelectByID2("Drawing View3", "DRAWINGVIEW", 0, 0, 0, False, 0, Nothing, 0)
        drview = selMan.GetSelectedObject6(1, 0)

```

```

        nPt(0) = 5.48
        nPt(1) = 5.22
        nPt(2) = 0
        vPt = nPt
        position = swMathUtil.CreatePoint(vPt)
        boolstatus = drview.InsertViewAsBlock(insertionPt, position)

```

```

    End Sub

```

```

    Public swApp As SldWorks

```

```

End Class

```

Convert Edges and Inner Loops of Face to Sketch Entities Example (C#)

This example shows how to convert the edges and inner loops of a selected face to sketch entities on a sketch plane.

```
//-----
// Preconditions: Open public_documents\samples\tutorial\api\cover_plate.slprt.
//
// Postconditions:
// 1. Creates an offset plane named Plane1.
// 2. Opens a sketch on Plane1.
// 3. Selects a face on the part.
// 4. Converts the edges and inner loops of the selected face to sketch
//    entities and creates Sketch2.
// 5. Examine the graphics area and FeatureManager design tree.
//
// NOTE: Because this part is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            FeatureManager swFeatureManager = default(FeatureManager);
            RefPlane swRefPlane = default(RefPlane);
            SketchManager swSketchManager = default(SketchManager);
            bool boolstatus = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swSketchManager = (SketchManager)swModel.SketchManager;

            //Create offset plane
            boolstatus = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, true, 0, null, 0);
            swRefPlane = (RefPlane)swFeatureManager.InsertRefPlane(8, 0.05, 0, 0, 0, 0);
            swModel.ClearSelection2(true);

            //Open sketch on Plane1
            boolstatus = swModelDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, false, 0, null, 0);
            swSketchManager.InsertSketch(true);

            //Select face whose edges and inner loops to convert
            boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.0456486773091456, 0.085157409722342, 0, false, 0, null, 0);

            //Convert edges and inner loops of selected face to sketch entities
            boolstatus = swSketchManager.SketchUseEdge3(false, true);

            //Clear the selections and close the sketch
            swModel.ClearSelection2(true);
            swSketchManager.InsertSketch(true);

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Convert Edges and Inner Loops of Face to Sketch Entities Example (VBA)

This example shows how to convert the edges and inner loops of a selected face to sketch entities on a sketch plane.

```
'-----
' Preconditions: Open public_documents\samples\tutorial\api\cover_plate.slp.
'
' Postconditions:
' 1. Creates an offset plane named Plane1.
' 2. Opens a sketch on Plane1.
' 3. Selects a face on the part.
' 4. Converts the edges and inner loops of the selected face to sketch
'    entities and creates Sketch2.
' 5. Examine the graphics area and FeatureManager design tree.
'
' NOTE: Because this part is used elsewhere, do not save changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatureManager As SldWorks.FeatureManager
Dim swRefPlane As SldWorks.RefPlane
Dim swSketchManager As SldWorks.SketchManager
Dim boolstatus As Boolean
```

Sub main()

```
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension
    Set swFeatureManager = swModel.FeatureManager
    Set swSketchManager = swModel.SketchManager

    'Create offset plane
    boolstatus = swModelDocExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
    Set swRefPlane = swFeatureManager.InsertRefPlane(8, 0.05, 0, 0, 0, 0)
    swModel.ClearSelection2 True

    'Open sketch on Plane1
    boolstatus = swModelDocExt.SelectByID("Plane1", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    swSketchManager.InsertSketch True
    'Select face whose edges and inner loops to convert
    boolstatus = swModelDocExt.SelectByID("", "FACE", 4.56486773091456E-02, 0.085157409722342, 0, False, 0, Nothing, 0)

    'Convert edges and inner loops of selected face to sketch entities
    boolstatus = swSketchManager.SketchUseEdge3(False, True)

    'Clear the selections and close the sketch
    swModel.ClearSelection2 True
    swSketchManager.InsertSketch True
```

End Sub

Convert Edges and Inner Loops of Face to Sketch Entities Example (VBA)

This example shows how to convert the edges and inner loops of a selected face to sketch entities on a sketch plane.

```

-----
' Preconditions: Open public_documents\samples\tutorial\api\cover_plate.slprrt.
-----
' Postconditions:
' 1. Creates an offset plane named Plane1.
' 2. Opens a sketch on Plane1.
' 3. Selects a face on the part.
' 4. Converts the edges and inner loops of the selected face to sketch
'    entities and creates Sketch2.
' 5. Examine the graphics area and FeatureManager design tree.
-----
' NOTE: Because this part is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swFeatureManager As FeatureManager
        Dim swRefPlane As RefPlane
        Dim swSketchManager As SketchManager
        Dim boolstatus As Boolean

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        swFeatureManager = swModel.FeatureManager
        swSketchManager = swModel.SketchManager

        'Create offset plane
        boolstatus = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
        swRefPlane = swFeatureManager.InsertRefPlane(0, 0.05, 0, 0, 0, 0)
        swModel.ClearSelection2(True)

        'Open sketch on Plane1
        boolstatus = swModelDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swSketchManager.InsertSketch(True)

        'Select face whose edges and inner loops to convert
        boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.0456486773091456, 0.085157409722342, 0, False, 0, Nothing, 0)

        'Convert edges and inner loops of selected face to sketch entities
        boolstatus = swSketchManager.SketchUseEdge3(False, True)

        'Clear the selections and close the sketch
        swModel.ClearSelection2(True)
        swSketchManager.InsertSketch(True)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

End Class

Convert Faces' Edges to Sketch Entities Example (VBA)

This example shows how to convert the edges of selected faces to sketch entities.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\cosmosxpress\aw_hook.sldprt.  
' 2. If prompted to convert to appearances, click OK.  
'  
' Postconditions:  
' 1. Converts the edges of the selected faces to sketch entities  
'    onto Plane1, and creates Sketch3.  
' 2. Examine the FeatureManager design tree.  
'  
' NOTE: Because this part is used elsewhere, do not save changes.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim swSketchManager As SldWorks.SketchManager  
Dim boolstatus As Boolean  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModelDocExt = swModel.Extension  
    Set swSketchManager = swModel.SketchManager  
  
    ' Open sketch on Plane1  
    boolstatus = swModelDocExt.SelectByID("Plane1", "PLANE", 0, 0, 0, False, 0, Nothing, 0)  
    swSketchManager.InsertSketch True  
  
    ' Select disjoint faces  
    boolstatus = swModelDocExt.SelectByID("", "FACE", 0.1098855514325, -0.05688720168837, 0.03453543805836, False, 0,  
    Nothing, 0)  
    boolstatus = swModelDocExt.SelectByID("", "FACE", 0.1161834220602, 0.002276177312467, 0.03345674799152, True, 0,  
    Nothing, 0)  
  
    ' Convert edges of faces to sketch entities  
    boolstatus = swSketchManager.SketchUseEdge2(False)  
  
    ' Clear the selections and close the sketch  
    swModel.ClearSelection2 True  
    swSketchManager.InsertSketch True  
  
End Sub
```

Convert Faces' Edges to Sketch Entities Example (VB.NET)

This example shows how to convert the edges of a face to sketch entities.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\cosmosxpress\aw_hook.sldprt.
' 2. If prompted to convert to appearances, click OK.
'
' Postconditions:
' 1. Converts the edges of the selected faces to sketch entities
'    onto Plane1, and creates Sketch3.
' 2. Examine the FeatureManager design tree.
'
' NOTE: Because this part is used elsewhere, do not save changes.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchManager As SketchManager
        Dim boolstatus As Boolean

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        swSketchManager = swModel.SketchManager

        ' Open sketch on Plane1
        boolstatus = swModelDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swSketchManager.InsertSketch(True)

        ' Select disjoint faces
        boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.1098855514325, -0.05688720168837, 0.03453543805836, False, 0, Nothing,
0)
        boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.1161834220602, 0.002276177312467, 0.03345674799152, True, 0, Nothing,
0)

        ' Convert edges of faces to sketch entities
        boolstatus = swSketchManager.SketchUseEdge2(False)

        ' Clear the selections and close the sketch
        swModel.ClearSelection2(True)
        swSketchManager.InsertSketch(True)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Copy and Paste Appearances Example (C#)

This example shows how to copy an appearance from one entity and apply it to other entities.

```
//-----
// Preconditions:
// 1. Open public_documents\introsw\vanity_assembly.sldasm.
// 2. Press F5 repeatedly while observing the changes in the appearances of
//    entities in the model.
//
// Postconditions: None
//
// Note: Because the model is used elsewhere, do not save changes when closing.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace CopyAndPasteAppearances_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;

        bool boolstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;

            object pEnt = null;
            SelectionMgr selMgr = default(SelectionMgr);
            selMgr = (SelectionMgr)Part.SelectionManager;

            Debug.Print("Selected a face, feature, body, component, or part and copied its appearance to the clipboard.");
            boolstatus = Part.Extension.SelectByID("", "FACE", -0.196625056591301, -0.204766940654167, 0.863250195790926, false,
0, null, 0);
            pEnt = selMgr.GetSelectedObject6(1, -1);
            boolstatus = swApp.CopyAppearance(pEnt);

            Debug.Print("Selected a face to which was applied the copied appearance.");
            boolstatus = Part.Extension.SelectByID("", "FACE", 0.273402696806954, 0.0150637315567224, 0.26416741603316, false, 0,
null, 0);
            pEnt = selMgr.GetSelectedObject6(1, -1);
            boolstatus = swApp.PasteAppearance(pEnt, (int)swAppearanceTargetType_e.swAppearanceTargetFace);
            System.Diagnostics.Debugger.Break();

            Debug.Print("Selected a face to whose component was applied the copied appearance.");
            boolstatus = Part.Extension.SelectByID("", "FACE", -2.59512148343788E-02, 0.178648261563922, 1.13882797742889, false,
0, null, 0);
            pEnt = selMgr.GetSelectedObject6(1, -1);
            boolstatus = swApp.PasteAppearance(pEnt, (int)swAppearanceTargetType_e.swAppearanceTargetComponent);
            System.Diagnostics.Debugger.Break();

            Debug.Print("Selected a face to whose feature was applied the copied appearance.");
            boolstatus = Part.Extension.SelectByID("", "FACE", -0.00438233207216854, 0.239755098177056, 0.383214316118654, false,
0, null, 0);
            pEnt = selMgr.GetSelectedObject6(1, -1);
            boolstatus = swApp.PasteAppearance(pEnt, (int)swAppearanceTargetType_e.swAppearanceTargetFeature);
            System.Diagnostics.Debugger.Break();

            Debug.Print("Selected a face to whose part was applied the copied appearance.");
            boolstatus = Part.Extension.SelectByID("", "FACE", -0.0112719408850239, 0.0695930730344685, 0.505696689751943, false,
0, null, 0);
            pEnt = selMgr.GetSelectedObject6(1, -1);
            boolstatus = swApp.PasteAppearance(pEnt, (int)swAppearanceTargetType_e.swAppearanceTargetPart);
            System.Diagnostics.Debugger.Break();

            Debug.Print("Selected a component to which was applied the copied appearance.");
            boolstatus = Part.Extension.SelectByID("door-1@vanity_assembly", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            pEnt = selMgr.GetSelectedObject6(1, -1);
            // The second parameter of SldWorks::PasteAppearance is ignored, if the selected object is not a face.
            boolstatus = swApp.PasteAppearance(pEnt, (int)swAppearanceTargetType_e.swAppearanceTargetAppearanceFilter);
            System.Diagnostics.Debugger.Break();
        }

        public SldWorks swApp;
    }
}
```


Copy and Paste Appearances Example (VBA)

This example shows how to copy an appearance from one entity and apply it to other entities.

```

'-----
' Preconditions:
' 1. Open a part or assembly that has appearances.
' 2. Press F5.
' 3. At the breakpoints, follow the instructions in the Immediate window.
' 4. Repeat steps 2 and 3 until the macro is finished.
'
' Postconditions: None
'-----

```

```

Dim swApp As SldWorks.SldWorks
Dim Part As SldWorks.ModelDoc2
Dim boolstatus As Boolean

```

```
Option Explicit
```

```
Sub main()
```

```
    Set swApp = Application.SldWorks
    Set Part = swApp.ActiveDoc

```

```

    Dim pEnt As Object
    Dim selMgr As SelectionMgr
    Set selMgr = Part.SelectionManager

```

```

    Debug.Print "Select a face, feature, body, component, or part that has an appearance."
    Stop

```

```
    Set pEnt = selMgr.GetSelectedObject6(1, -1)
```

```

    ' Copy appearance to the clipboard
    boolstatus = swApp.CopyAppearance(pEnt)

```

```
    If Part.GetType = swDocPART Then
```

```

        Debug.Print "Select a face to which to apply the copied appearance."
        Stop

```

```

        Set pEnt = selMgr.GetSelectedObject6(1, -1)
        boolstatus = swApp.PasteAppearance(pEnt, swAppearanceTargetFace)

```

```

        Debug.Print "Select a face to whose feature is applied the copied appearance."
        Stop

```

```

        Set pEnt = selMgr.GetSelectedObject6(1, -1)
        boolstatus = swApp.PasteAppearance(pEnt, swAppearanceTargetFeature)

```

```

        Debug.Print "Select a face to whose body is applied the copied appearance."
        Stop

```

```

        Set pEnt = selMgr.GetSelectedObject6(1, -1)
        boolstatus = swApp.PasteAppearance(pEnt, swAppearanceTargetBody)

```

```

        Debug.Print "Select a face to whose part is applied the copied appearance."
        Stop

```

```

        Set pEnt = selMgr.GetSelectedObject6(1, -1)
        boolstatus = swApp.PasteAppearance(pEnt, swAppearanceTargetPart)

```

```

    Debug.Print "Select a face. All appearances in the model that match that of the selected face are changed to the copied appearance."
    Stop

```

```

        Set pEnt = selMgr.GetSelectedObject6(1, -1)
        boolstatus = swApp.PasteAppearance(pEnt, swAppearanceTargetAppearanceFilter)

```

```

        Debug.Print "Select a feature to which to apply the copied appearance."
        Stop

```

```

        Set pEnt = selMgr.GetSelectedObject6(1, -1)
        boolstatus = swApp.PasteAppearance(pEnt, swAppearanceTargetAppearanceFilter) ' The second parameter is ignored.

```

```

        Debug.Print "Select a body to which to apply the copied appearance."
        Stop

```

```

        Set pEnt = selMgr.GetSelectedObject6(1, -1)
        boolstatus = swApp.PasteAppearance(pEnt, swAppearanceTargetAppearanceFilter) ' The second parameter is ignored.

```

```

Debug.Print "No input entity. Preselect an entity in the graphics area to whose body is applied the copied appearance."
Stop

boolstatus = swApp.PasteAppearance(Nothing, swAppearanceTargetBody)

ElseIf Part.GetType = swDocASSEMBLY Then

    Debug.Print "Select a part to which to apply the copied appearance."
    Stop

    Set pEnt = selMgr.GetSelectedObject6(1, -1)
    boolstatus = swApp.PasteAppearance(pEnt, swAppearanceTargetAppearanceFilter) ' The second parameter is ignored.

    Debug.Print "Select a face to whose component is applied the copied appearance."
    Stop

    Set pEnt = selMgr.GetSelectedObject6(1, -1)
    boolstatus = swApp.PasteAppearance(pEnt, swAppearanceTargetComponent)

    Debug.Print "Select a face to whose body is applied the copied appearance."
    Stop

    Set pEnt = selMgr.GetSelectedObject6(1, -1)
    boolstatus = swApp.PasteAppearance(pEnt, swAppearanceTargetBody)

    Debug.Print "Select a face to whose feature is applied the copied appearance."
    Stop

    Set pEnt = selMgr.GetSelectedObject6(1, -1)
    boolstatus = swApp.PasteAppearance(pEnt, swAppearanceTargetFeature)

    Debug.Print "Select a face to which is applied the copied appearance."
    Stop

    Set pEnt = selMgr.GetSelectedObject6(1, -1)
    boolstatus = swApp.PasteAppearance(pEnt, swAppearanceTargetFace)

    Debug.Print "Select a face to whose part is applied the copied appearance."
    Stop

    Set pEnt = selMgr.GetSelectedObject6(1, -1)
    boolstatus = swApp.PasteAppearance(pEnt, swAppearanceTargetPart)
End If
End Sub

```

Copy and Paste Appearances Example (VB.NET)

This example shows how to copy an appearance from one entity and apply it to other entities.

```

'-----
' Preconditions:
' 1. Open public_documents\introsw\vanity_assembly.sldasm.
' 2. Press F5 repeatedly while observing the changes in the appearances of
'    entities in the model.
'
' Postconditions: None
'
' Note: Because the model is used elsewhere, do not save changes when closing.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim Part As ModelDoc2
    Dim boolstatus As Boolean

    Sub main()

        Part = swApp.ActiveDoc

        Dim pEnt As Object
        Dim selMgr As SelectionMgr
        selMgr = Part.SelectionManager

        Debug.Print("Selected a face, feature, body, component, or part and copied its appearance to the clipboard.")
        boolstatus = Part.Extension.SelectByID2("", "FACE", -0.196625056591301, -0.204766940654167, 0.863250195790926, False, 0,
Nothing, 0)
        pEnt = selMgr.GetSelectedObject6(1, -1)
        boolstatus = swApp.CopyAppearance(pEnt)

        Debug.Print("Selected a face to which was applied the copied appearance.")
        boolstatus = Part.Extension.SelectByID2("", "FACE", 0.273402696806954, 0.0150637315567224, 0.26416741603316, False, 0,
Nothing, 0)
        pEnt = selMgr.GetSelectedObject6(1, -1)
        boolstatus = swApp.PasteAppearance(pEnt, swAppearanceTargetType_e.swAppearanceTargetFace)
        Stop

        Debug.Print("Selected a face to whose component was applied the copied appearance.")
        boolstatus = Part.Extension.SelectByID2("", "FACE", -0.0259512148343788, 0.178648261563922, 1.13882797742889, False, 0,
Nothing, 0)
        pEnt = selMgr.GetSelectedObject6(1, -1)
        boolstatus = swApp.PasteAppearance(pEnt, swAppearanceTargetType_e.swAppearanceTargetComponent)
        Stop

        Debug.Print("Selected a face to whose feature was applied the copied appearance.")
        boolstatus = Part.Extension.SelectByID2("", "FACE", -0.00438233207216854, 0.239755098177056, 0.383214316118654, False, 0,
Nothing, 0)
        pEnt = selMgr.GetSelectedObject6(1, -1)
        boolstatus = swApp.PasteAppearance(pEnt, swAppearanceTargetType_e.swAppearanceTargetFeature)
        Stop

        Debug.Print("Selected a face to whose part was applied the copied appearance.")
        boolstatus = Part.Extension.SelectByID2("", "FACE", -0.0112719408850239, 0.0695930730344685, 0.505696689751943, False, 0,
Nothing, 0)
        pEnt = selMgr.GetSelectedObject6(1, -1)
        boolstatus = swApp.PasteAppearance(pEnt, swAppearanceTargetType_e.swAppearanceTargetPart)
        Stop

        Debug.Print("Selected a component to which was applied the copied appearance.")
        boolstatus = Part.Extension.SelectByID2("door-1@vanity_assembly", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        pEnt = selMgr.GetSelectedObject6(1, -1)
        'The second parameter of SldWorks::PasteAppearance is ignored, if the selected object is not a face.
        boolstatus = swApp.PasteAppearance(pEnt, swAppearanceTargetType_e.swAppearanceTargetAppearanceFilter)
        Stop

    End Sub

    Public swApp As SldWorks

End Class

```

Copy and Paste Drawing Sheet Example (C#)

This example shows how to copy and paste drawing sheets.

```
//-----
// Preconditions:
// 1. Open a drawing document containing one sheet
//    named Sheet1.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Activates Sheet1.
// 2. Copy and pastes Sheet1 as Sheet1(2) and activates Sheet1(2).
// 3. Copy and pastes Sheet1 as Sheet1(3) and activates Sheet1(3).
// 4. Examine the FeatureManager design tree and Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            DrawingDoc Part = default(DrawingDoc);
            ModelDoc2 swModel = default(ModelDoc2);
            bool boolstatus = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            Part = (DrawingDoc)swModel;
            if ((Part == null))
            {
                Debug.Print(" Please open a drawing document. ");
                return;
            }

            Sheet currentsheet = default(Sheet);
            currentsheet = (Sheet)Part.GetCurrentSheet();
            Part.ActivateSheet(currentsheet.GetName());
            Debug.Print("Active sheet: " + currentsheet.GetName());

            boolstatus = swModel.Extension.SelectByID("Sheet1", "SHEET", 0.09205356547875, 0.10872368523, 0, false, 0, null, 0);
            swModel.EditCopy();
            boolstatus = Part.PasteSheet((int)swInsertOptions_e.swInsertOption_BeforeSelectedSheet, (int)swRenameOptions_e.swRenameOption_No);
            currentsheet = (Sheet)Part.GetCurrentSheet();
            Part.ActivateSheet(currentsheet.GetName());
            Debug.Print("Active sheet: " + currentsheet.GetName());

            boolstatus = swModel.Extension.SelectByID("Sheet1", "SHEET", 0.09205356547875, 0.10872368523, 0, false, 0, null, 0);
            swModel.EditCopy();
            boolstatus = Part.PasteSheet((int)swInsertOptions_e.swInsertOption_AfterSelectedSheet, (int)swRenameOptions_e.swRenameOption_No);
            currentsheet = (Sheet)Part.GetCurrentSheet();
            Part.ActivateSheet(currentsheet.GetName());
            Debug.Print("Active sheet: " + currentsheet.GetName());

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Copy and Paste Drawing Sheet Example (VBA)

This example shows how to copy and paste drawing sheets.

```
'-----
' Preconditions:
' 1. Open a drawing document containing one sheet
'    named Sheet1.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Activates Sheet1.
' 2. Copy and pastes Sheet1 as Sheet1(2) and activates Sheet1(2).
' 3. Copy and pastes Sheet1 as Sheet1(3) and activates Sheet1(3).
' 4. Examine the FeatureManager design tree and Immediate window.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim Part As DrawingDoc
Dim swModel As ModelDoc2
Dim boolstatus As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set Part = swModel

    If (Part Is Nothing) Then
        MsgBox " Please open a drawing document. "
    End
End If

Dim currentsheet As Sheet
Set currentsheet = Part.GetCurrentSheet
Part.ActivateSheet (currentsheet.GetName)
Debug.Print "Active sheet: " & currentsheet.GetName

boolstatus = Part.Extension.SelectByID2("Sheet1", "SHEET", 0.09205356547875, 0.10872368523, 0, False, 0, Nothing, 0)
swModel.EditCopy
boolstatus = Part.PasteSheet(swInsertOption_BeforeSelectedSheet, swRenameOption_No)
Set currentsheet = Part.GetCurrentSheet
Part.ActivateSheet (currentsheet.GetName)
Debug.Print "Active sheet: " & currentsheet.GetName

boolstatus = Part.Extension.SelectByID2("Sheet1", "SHEET", 0.09205356547875, 0.10872368523, 0, False, 0, Nothing, 0)
swModel.EditCopy
boolstatus = Part.PasteSheet(swInsertOption_AfterSelectedSheet, swRenameOption_No)
Set currentsheet = Part.GetCurrentSheet
Part.ActivateSheet (currentsheet.GetName)
Debug.Print "Active sheet: " & currentsheet.GetName

End Sub
```

Copy and Paste Drawing Sheet Example (VB.NET)

This example shows how to copy and paste drawing sheets.

```

-----
Preconditions:
1. Open a drawing document containing one sheet
   named Sheet1.
2. Open the Immediate window.
Postconditions:
1. Activates Sheet1.
2. Copy and pastes Sheet1 as Sheet1(2) and activates Sheet1(2).
3. Copy and pastes Sheet1 as Sheet1(3) and activates Sheet1(3).
4. Examine the FeatureManager design tree and Immediate window.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()
        Dim Part As DrawingDoc
        Dim swModel As ModelDoc2
        Dim boolstatus As Boolean

        swModel = swApp.ActiveDoc
        Part = swModel
        If (Part Is Nothing) Then
            MsgBox(" Please open a drawing document. ")
            Exit Sub
        End If

        Dim currentsheet As Sheet
        currentsheet = Part.GetCurrentSheet
        Part.ActivateSheet(currentsheet.GetName)
        Debug.Print("Active sheet: " & currentsheet.GetName)

        boolstatus = swModel.Extension.SelectByID("Sheet1", "SHEET", 0.09205356547875, 0.10872368523, 0, False, 0, Nothing, 0)
        swModel.EditCopy()
        boolstatus = Part.PasteSheet(swInsertOptions_e.swInsertOption_BeforeSelectedSheet, swRenameOptions_e.swRenameOption_No)
        currentsheet = Part.GetCurrentSheet
        Part.ActivateSheet(currentsheet.GetName)
        Debug.Print("Active sheet: " & currentsheet.GetName)

        boolstatus = swModel.Extension.SelectByID("Sheet1", "SHEET", 0.09205356547875, 0.10872368523, 0, False, 0, Nothing, 0)
        swModel.EditCopy()
        boolstatus = Part.PasteSheet(swInsertOptions_e.swInsertOption_AfterSelectedSheet, swRenameOptions_e.swRenameOption_No)
        currentsheet = Part.GetCurrentSheet
        Part.ActivateSheet(currentsheet.GetName)
        Debug.Print("Active sheet: " & currentsheet.GetName)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Copy and Paste Feature Example (C#)

This example shows how to copy and paste a feature in a part document.

```
//-----
// Preconditions: Verify that the specified part document to
// open exists.
//
// Postconditions:
// 1. Displays the Copy Confirmation dialog after calling
//   IModelDoc2::Paste.
// 2. Click Delete.
// 3. Pastes the copied feature on the selected face.
//
// NOTE: Because this part is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace EditCopyPasteModelDoc2CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        ModelDocExtension swModelDocExt;
        string fileName;
        bool status;
        int errors;
        int warnings;

        public void Main()
        {
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\testpart1.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);

            // Select the feature to copy
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("Boss-Extrude1", "BODYFEATURE", 0, 0, 0, false, 0, null, 0);
            swModel.EditCopy();

            swModel.ClearSelection2(true);

            // Select the face where to paste the copied feature
            status = swModelDocExt.SelectByID2("", "FACE", 0.0297472797980731, 0.0564587562103043, 0.00676585125080464, false, 0, null, 0);

            // Paste the copied feature on the selected face
            swModel.Paste();

            // Display the Copy Confirmation dialog
            // Click Delete
            // Pastes the copied feature on the selected face

            //Zoom to selection, then zoom to fit
            swModel.ViewZoomToSelection();
            swModel.ViewZoomtofit2();

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>

        public SldWorks swApp;
    }
}
```

Copy and Paste Feature Example (VBA)

This example shows how to copy and paste a feature in a part document.

```
'-----  
' Preconditions: Verify that the specified part document to  
' open exists.  
'  
' Postconditions:  
' 1. Displays the Copy Confirmation dialog after calling  
'    IModelDoc2::Paste.  
' 2. Click Delete.  
' 3. Pastes the copied feature on the selected face.  
'  
' NOTE: Because this part is used elsewhere, do not save changes.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim fileName As String  
Dim status As Boolean  
Dim errors As Long, warnings As Long  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\testpart1.sldprt"  
    Set swModel = swApp.OpenDoc6(fileName, swDocPART, swOpenDocOptions_Silent, "", errors, warnings)  
  
    ' Select the feature to copy  
    Set swModelDocExt = swModel.Extension  
    status = swModelDocExt.SelectByID2("Boss-Extrude1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)  
    swModel.EditCopy  
  
    swModel.ClearSelection2 True  
  
    ' Select the face where to paste the copied feature  
    status = swModelDocExt.SelectByID2("", "FACE", 2.97472797980731E-02, 5.64587562103043E-02, 6.76585125080464E-03, False, 0, Nothing, 0)  
  
    ' Paste the copied feature on the selected face  
    swModel.Paste  
  
    ' Displays the Copy Confirmation dialog  
    ' Click Delete  
    ' Pastes the copied feature on the selected face  
  
    ' Zoom to selection, then zoom to fit  
    swModel.ViewZoomToSelection  
    swModel.ViewZoomtofit2  
  
End Sub
```

Copy and Paste Feature Example (VB.NET)

This example shows how to copy and paste a feature in a part document.

```

'-----
' Preconditions: Verify that the specified part document to
' open exists.
' Postconditions:
' 1. Displays the Copy Confirmation dialog after calling
'   IModelDoc2::Paste.
' 2. Click Delete.
' 3. Pastes the copied feature on the selected face.
' NOTE: Because this part is used elsewhere, do not save changes.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim fileName As String
    Dim status As Boolean
    Dim errors As Integer, warnings As Integer

    Sub main()

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\testpart1.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

        ' Select the feature to copy
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("Boss-Extrude1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, 0)
        swModel.EditCopy()

        swModel.ClearSelection2(True)

        ' Select the face where to paste the copied feature
        status = swModelDocExt.SelectByID2("", "FACE", 0.0297472797980731, 0.0564587562103043, 0.00676585125080464, False, 0, Nothing, 0)

        ' Paste the copied feature on the selected face
        swModel.Paste()

        ' Displays the Copy Confirmation dialog
        ' Click Delete
        ' Pastes on the copied feature on the selected face

        ' Zoom to selection, then zoom to fit
        swModel.ViewZoomToSelection()
        swModel.ViewZoomtofit2()

    End Sub

    ''' <summary>
    ''' The Sldworks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Copy and Rotate Body Using Edge Example (VBA)

This example shows how to copy and rotate a body using an edge.

```
'-----
' Preconditions:
' 1. Verify that the specified part document exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Selects a solid body and two vertices.
' 3. Inserts a move/copy body feature.
' 4. Selects an edge.
' 5. Rotates the move/copy body about the selected edge.
' 6. Examine the graphics area.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatureManager As SldWorks.FeatureManager
Dim swFeature As SldWorks.Feature
Dim swMoveCopyBodyFeatureData As SldWorks.MoveCopyBodyFeatureData
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swEdge As Edge
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim fileName As String

Sub main()

    Set swApp = Application.SldWorks

    'Open part document
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\multibody\multi_inter.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension
    Set swFeatureManager = swModel.FeatureManager

    'Select solid body and vertices for move/copy body feature
    status = swModelDocExt.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, 0, 0.065, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, -0.09, 0.065, True, 0, Nothing, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, False, 1, Nothing, 0)
    status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, 0, 0.065, True, 4, Nothing, 0)
    status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, -0.09, 0.065, True, 8, Nothing, 0)

    'Insert move/copy body feature
    Set swFeature = swFeatureManager.InsertMoveCopyBody2(0, 0, 0, -0.085, 0, 0.065, 0, 0, 0, True, 1)

    'Roll forward the feature and get and set move/copy body feature data
    Set swMoveCopyBodyFeatureData = swFeature.GetDefinition
    status = swMoveCopyBodyFeatureData.AccessSelections(swModel, Nothing)

    'Rotate body about the selected edge
    swMoveCopyBodyFeatureData.TransformType = swMoveCopyBodyFeatureTransformType_e.swTransformType_Rotation
    status = swModelDocExt.SelectByID2("", "EDGE", -8.53353899645981E-02, -1.26994939741394E-02, 6.53641069000059E-02, False, 0, Nothing, 0)
    Set swSelectionMgr = swModel.SelectionManager
    Set swEdge = swSelectionMgr.GetSelectedObject6(1, -1)
    swMoveCopyBodyFeatureData.TransformReferenceEntity = swEdge
    swMoveCopyBodyFeatureData.TransformX = 0.09

    'Roll back
    status = swFeature.ModifyDefinition(swMoveCopyBodyFeatureData, swModel, Nothing)
    swMoveCopyBodyFeatureData.ReleaseSelectionAccess

End Sub
```

Copy Component With Profile Center Mate Example (C#)

This example shows how to:

- create a new assembly.
- add two components to the assembly.
- create a profile center mate between the components.
- copy a component with that mate.

```
//-----
// Preconditions:
// 1. Open a new SOLIDWORKS Design session.
// 2. Copy public_documents\samples\tutorial\api\block20.sldprt and
//    cylinder20.sldprt to c:\temp.
// 3. Verify that the specified assembly document template exists.
//
// Postconditions:
// 1. Opens both parts.
// 2. Creates a new assembly.
// 3. Inserts both parts into the new assembly.
// 4. Creates a profile center mate between the two components.
// 5. Copies a component and the profile center mate.
// 6. To verify steps 4 and 5:
//    * Examine the graphics area and FeatureManager design tree.
//    * Expand Mates in the FeatureManager design tree.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            PartDoc swPart1 = default(PartDoc);
            PartDoc swPart2 = default(PartDoc);
            ModelDoc2 swModel = default(ModelDoc2);
            AssemblyDoc swAssemblyDoc = default(AssemblyDoc);
            Component2 swComponent1 = default(Component2);
            Component2 swComponent2 = default(Component2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            Mate2 swMate = default(Mate2);
            bool status = false;
            int errors = 0;
            int warnings = 0;
            DispatcherWrapper[] dispWrapperComponentArray = new DispatcherWrapper[1];
            Component2[] swComponentArray = new Component2[1];
            bool[] repeatArray = new bool[1];
            DispatcherWrapper[] dispWrapperMateReferencesArray = new DispatcherWrapper[1];
            double[] valueArray = new double[1];
            bool[] flipAlignmentArray = new bool[1];
            bool[] flipDimensionArray = new bool[1];
            bool[] lockRotationArray = new bool[1];
            int[] orientationArray = new int[1];

            // Open parts for new assembly
            swPart1 = (PartDoc)swApp.OpenDoc6("C:\\temp\\block20.sldprt", (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swPart2 = (PartDoc)swApp.OpenDoc6("C:\\temp\\cylinder20.sldprt", (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);

            // Open new assembly document
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2016\\templates\\Assembly.asmdot", 0, 0, 0);
            swApp.ActivateDoc2("Assem1", false, ref errors);
            swAssemblyDoc = (AssemblyDoc)swModel;

            // Add components to assembly document
            swComponent1 = (Component2)swAssemblyDoc.AddComponent5("C:\\temp\\block20.sldprt", (int)swAddComponentConfigOptions_e.swAddComponentConfigOptions_CurrentSelectedConfig, "", false, "", 0.0522792702800426, 0.0658677995643489, 0.102016044707081);
            swComponent2 = (Component2)swAssemblyDoc.AddComponent5("C:\\temp\\cylinder20.sldprt", (int)swAddComponentConfigOptions_e.swAddComponentConfigOptions_CurrentSelectedConfig, "", false, "", 0.177061497059185, -0.00151244836160913, 0.0673233033157885);
            swModel.ViewZoomtofit2();

            // Add profile center mate
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("", "FACE", 0.0631695178495306, 0.0856797995642182, 0.137370061843797, true, 1, null, 0);
            status = swModelDocExt.SelectByID2("", "FACE", 0.141204290267694, 0.031253551638315, 0.0843440006535161, true, 1, null, 0);
            swMate = (Mate2)swAssemblyDoc.AddMate5((int)swMateType_e.swMatePROFILECENTER, (int)swMateAlign_e.swMateAlignALIGNED, true, 0.0762, 0.0254, 0.0254, 0.0254, 0.0254, 0.5235987755983, false, true, (int)swMateWidthOptions_e.swMateWidth_Centered, out errors);
            swModel.ClearSelection2(true);

            // Copy component with profile center mate
            swComponentArray[0] = swComponent2;
            dispWrapperComponentArray[0] = new DispatcherWrapper(swComponentArray[0]);
            repeatArray[0] = true;
            dispWrapperMateReferencesArray[0] = new DispatcherWrapper(null);
            valueArray[0] = 0.05;
            flipAlignmentArray[0] = true;
            flipDimensionArray[0] = true;
            lockRotationArray[0] = false;
            orientationArray[0] = 0;
            status = swAssemblyDoc.CopyWithMates2(dispWrapperComponentArray, repeatArray, dispWrapperMateReferencesArray, valueArray, flipAlignmentArray, flipDimensionArray, lockRotationArray, orientationArray);

        }
    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
```

Copy Component With Profile Center Mate Example (VBA)

This example shows how to:

- create a new assembly.
- add two components to the assembly.
- create a profile center mate between the components.
- copy a component with that mate.

```

'-----
' Preconditions:
' 1. Open a new SOLIDWORKS Design session.
' 2. Copy public_documents\samples\tutorial\api\block20.sldprt and
'   cylinder20.sldprt to c:\temp.
' 3. Verify that the specified assembly document template exists.
'
' Postconditions:
' 1. Opens both parts.
' 2. Creates a new assembly.
' 3. Inserts both parts into the new assembly.
' 4. Creates a profile center mate between the two components.
' 5. Copies a component and the profile center mate.
' 6. To verify steps 4 and 5:
'    * Examine the graphics area and FeatureManager design tree.
'    * Expand Mates in the FeatureManager design tree.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swPart1 As SldWorks.PartDoc
Dim swPart2 As SldWorks.PartDoc
Dim swModel As SldWorks.ModelDoc2
Dim swAssemblyDoc As SldWorks.AssemblyDoc
Dim swComponent1 As SldWorks.Component2
Dim swComponent2 As SldWorks.Component2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swMate As SldWorks.Mate2
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim swComponentArray(0) As SldWorks.Component2
Dim repeatArray(0) As Boolean
Dim mateReferencesArray(0) As Object
Dim valueArray(0) As Double
Dim flipAlignmentArray(0) As Boolean
Dim flipDimensionArray(0) As Boolean
Dim lockRotationArray(0) As Boolean
Dim orientationArray(0) As Long

Sub main()

    Set swApp = Application.SldWorks

    ' Open parts for new assembly
    Set swPart1 = swApp.OpenDoc6("C:\temp\block20.sldprt", swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swPart2 = swApp.OpenDoc6("C:\temp\cylinder20.sldprt", swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors,
warnings)

    'Open new assembly document
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\Assembly.asmdot", 0, 0, 0)
    Set swAssemblyDoc = swModel

    ' Add components to assembly document
    Set swComponent1 = swAssemblyDoc.AddComponent5("C:\temp\block20.sldprt",
swAddComponentConfigOptions_e.swAddComponentConfigOptions_CurrentSelectedConfig, "", False, "", 5.22792702800426E-02, 6.58677995643489E-02,
0.102016044707081)
    Set swComponent2 = swAssemblyDoc.AddComponent5("C:\temp\cylinder20.sldprt",
swAddComponentConfigOptions_e.swAddComponentConfigOptions_CurrentSelectedConfig, "", False, "", 0.177061497059185, -1.51244836160913E-03,
6.73233033157885E-02)
    swModel.ViewZoomtofit2

    ' Add profile center mate
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID2("", "FACE", 6.31695178495306E-02, 8.56797995642182E-02, 0.137370061843797, True, 1, Nothing, 0)
    status = swModelDocExt.SelectByID2("", "FACE", 0.141204290267694, 0.031253551638315, 8.43440006535161E-02, True, 1, Nothing, 0)
    Set swMate = swAssemblyDoc.AddMate5(swMateType_e.swMatePROFILECENTER, swMateAlign_e.swMateAlignALIGNED, True, 0.0762, 0.0254, 0.0254, 0.0254,
0, 0.5235987755983, 0.5235987755983, False, True, swMateWidthOptions_e.swMateWidth_Centered, errors)
    swModel.ClearSelection2 True

    ' Copy component with profile center mate
    Set swComponentArray(0) = swComponent2
    repeatArray(0) = True
    Set mateReferencesArray(0) = Nothing
    valueArray(0) = 0.05
    flipAlignmentArray(0) = True
    flipDimensionArray(0) = True
    lockRotationArray(0) = False
    orientationArray(0) = 0
    status = swAssemblyDoc.CopyWithMates2(swComponentArray, repeatArray, mateReferencesArray, valueArray, flipAlignmentArray, flipDimensionArray,
lockRotationArray, orientationArray)

End Sub

```

Copy Component With Profile Center Mate Example (VB.NET)

This example shows how to:

- create a new assembly.
- add two components to the assembly.
- create a profile center mate between the components.
- copy a component with that mate.

```

'-----
' Preconditions:
' 1. Open a new SOLIDWORKS Design session.
' 2. Copy install_dir\samples\tutorial\api\block20.sldprt and
'    cylinder20.sldprt to c:\temp.
' 3. Verify that the specified assembly document template exists.
'-----
' Postconditions:
' 1. Opens both parts.
' 2. Creates a new assembly.
' 3. Inserts both parts into the new assembly.
' 4. Creates a profile center mate between the two components.
' 5. Copies a component and the profile center mate.
' 6. To verify steps 4 and 5:
'    * Examine the graphics area and FeatureManager design tree.
'    * Expand Mates in the FeatureManager design tree.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Sub main()
        Dim swPart1 As PartDoc
        Dim swPart2 As PartDoc
        Dim swModel As ModelDoc2
        Dim swAssemblyDoc As AssemblyDoc
        Dim swComponent1 As Component2
        Dim swComponent2 As Component2
        Dim swModelDocExt As ModelDocExtension
        Dim swMate As Mate2
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim dispWrapperComponentArray(0) As DispatcherWrapper
        Dim swComponentArray(0) As Component2
        Dim repeatArray(0) As Boolean
        Dim dispWrapperMateReferencesArray(0) As DispatcherWrapper
        Dim valueArray(0) As Double
        Dim flipAlignmentArray(0) As Boolean
        Dim flipDimensionArray(0) As Boolean
        Dim lockRotationArray(0) As Boolean
        Dim orientationArray(0) As Integer

        ' Open parts for new assembly
        swPart1 = swApp.OpenDoc6("C:\temp\block20.sldprt", swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swPart2 = swApp.OpenDoc6("C:\temp\cylinder20.sldprt", swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warning
s)

        ' Open new assembly document
        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2016\templates\Assembly.asmdot", 0, 0, 0)
        swApp.ActivateDoc2("Assemi", False, errors)
        swAssemblyDoc = swModel

        ' Add components to assembly document
        swComponent1 = swAssemblyDoc.AddComponent5("C:\temp\block20.sldprt", swAddComponentConfigOptions_e.swAddComponentConfigOptions_CurrentSelectedConfi
g, "", False, "", 0.0522792702800426, 0.0658677995643489, 0.102016044707081)
        swComponent2 = swAssemblyDoc.AddComponent5("C:\temp\cylinder20.sldprt", swAddComponentConfigOptions_e.swAddComponentConfigOptions_CurrentSelectedC
onfig, "", False, "", 0.177061497059185, -0.00151244836160913, 0.0673233033157885)
        swModel.ViewZoomtofit2()

        ' Add profile center mate
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID2("", "FACE", 0.0631695178495306, 0.0856797995642182, 0.137370061843797, True, 1, Nothing, 0)
        status = swModelDocExt.SelectByID2("", "FACE", 0.141204290267694, 0.031253551638315, 0.0843440006535161, True, 1, Nothing, 0)
        swMate = swAssemblyDoc.AddMate5(swMateType_e.swMatePROFILERCENTER, swMateAlign_e.swMateAlignALIGNED, True, 0.0762, 0.0254, 0.0254, 0.0254, 0.0254,
0, 0.5235987755983, 0.5235987755983, False, True, swMateWidthOptions_e.swMateWidth_Centered, errors)
        swModel.ClearSelection2(True)

        ' Copy component with profile center mate
        swComponentArray(0) = swComponent2
        dispWrapperComponentArray(0) = New DispatcherWrapper(swComponentArray(0))
        repeatArray(0) = True
        dispWrapperMateReferencesArray(0) = New DispatcherWrapper(Nothing)
        valueArray(0) = 0.05
        flipAlignmentArray(0) = True
        flipDimensionArray(0) = True
        lockRotationArray(0) = False
        orientationArray(0) = 0
        status = swAssemblyDoc.CopyWithMates2(dispWrapperComponentArray, repeatArray, dispWrapperMateReferencesArray, valueArray, flipAlignmentArray, flip
DimensionArray, lockRotationArray, orientationArray)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Copy Components With Mates to Assembly Example (C#)

This example shows how to copy components with mates in an assembly.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\driveworksxpess\mobile gantry.SLDASM.
// 2. Inspect Leg<1> and Leg<2> in the assembly.
//
// Postconditions:
// 1. Replaces Leg<1> with a copy of Leg<2>.
// 2. Inspect Leg<2> and leg<3> in the assembly.
//
// NOTE: Because this assembly is used elsewhere, do not save any changes.
//-----

using SolidWorks.Interop.sldworks;
using System.Runtime.InteropServices;

namespace CopyComponentsWithMatesToAssembly_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelMgr = default(SelectionMgr);
            bool[] blRepeat = new bool[3];
            bool[] blFlip = new bool[3];
            object SelectedComps = null;
            object RepeatOptions = null;
            object[] MateRefs = new object[3];
            object InpValues = null;
            object FlipValues = null;
            AssemblyDoc swAssy = default(AssemblyDoc);
            Component2 swItem = default(Component2);
            Feature swFeature = default(Feature);
            RefPlane refPlane = default(RefPlane);
            object[] arrMateEntities = new object[3];
            object[] arrCompsToCopy = new object[1];
            bool boolstatus = false;
            double[] dValues = new double[3];

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swAssy = (AssemblyDoc)swModel;

            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swModelDocExt = swModel.Extension;
            boolstatus = swModelDocExt.SelectByID2("Leg-1@mobile gantry", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            swModel.EditDelete();

            swModel.ClearSelection2(true);

            boolstatus = swModelDocExt.SelectByID2("Leg-2@mobile gantry", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            swItem = (Component2)swSelMgr.GetSelectedObject6(1, -1);
            arrCompsToCopy[0] = swItem;
            arrMateEntities[0] = null;
            arrMateEntities[1] = null;
            swModel.ClearSelection2(true);
            boolstatus = swModelDocExt.SelectByID2("Left End@Universal Beam-1@mobile gantry", "PLANE", 0, 0, 0, false, 0, null, 0);
            swFeature = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            refPlane = (RefPlane)swFeature.GetSpecificFeature2();
            arrMateEntities[2] = refPlane;

            swModel.ClearSelection2(true);
```

```

        blRepeat[0] = true;
        blRepeat[1] = true;
        blRepeat[2] = false;
        SelectedComps = arrCompsToCopy;
        RepeatOptions = blRepeat;
        MateRefs = arrMateEntities;
        dValues[0] = 0.0;
        dValues[1] = 0.0;
        dValues[2] = 0.0;
        InpValues = dValues;
        blFlip[0] = false;
        blFlip[1] = false;
        blFlip[2] = false;
        FlipValues = blFlip;
        DispatchWrapper[] disparrCompsToCopy = new DispatchWrapper[1];
        disparrCompsToCopy[0] = new DispatchWrapper(arrCompsToCopy[0]);
        DispatchWrapper[] dispMateRefs = new DispatchWrapper[3];
        dispMateRefs[0] = new DispatchWrapper(MateRefs[0]);
        dispMateRefs[1] = new DispatchWrapper(MateRefs[1]);
        dispMateRefs[2] = new DispatchWrapper(MateRefs[2]);

        //Repeat all mates except the coincident mate with "Right End@Universal Beam-1"
        swAssy.CopyWithMates((disparrCompsToCopy), RepeatOptions, (dispMateRefs), InpValues, FlipValues);
        swAssy.EditRebuild();
    }
    public SldWorks swApp;
}
}

```

Copy Components With Mates to Assembly Example (VBA)

This example shows how to copy components with mates in an assembly.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\driveworks\express\mobile gantry.SLDASM.
' 2. Inspect Leg<1> and Leg<2> in the assembly.
'
' Postconditions:
' 1. Replaces Leg<1> with a copy of Leg<2>.
' 2. Inspect Leg<2> and Leg<3> in the assembly.
'
' NOTE: Because this assembly is used elsewhere, do not save changes.
'-----
Option Explicit

Sub main()

    Dim SwApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim blRepeat(2) As Boolean
    Dim SelectedComps As Variant
    Dim RepeatOptions As Variant
    Dim MateRefs As Variant
    Dim InpValues As Variant
    Dim FlipValues As Variant
    Dim swAssy As AssemblyDoc
    Dim swItem As Object
    Dim arrMateEntities(2) As Object
    Dim arrCompsToCopy(0) As Object
    Dim boolstatus As Boolean
    Dim dValues(2) As Double

    Set SwApp = Application.SldWorks
    Set swModel = SwApp.ActiveDoc

    Set swAssy = swModel
    Set swSelMgr = swModel.SelectionManager
    Set swModelDocExt = swModel.Extension
    boolstatus = swModelDocExt.SelectByID2("Leg-1@mobile gantry", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    swModel.EditDelete

    swModel.ClearSelection2 True

    boolstatus = swModelDocExt.SelectByID2("Leg-2@mobile gantry", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    Set swItem = swSelMgr.GetSelectedObject6(1, -1)
    Set arrCompsToCopy(0) = swItem

    swModel.ClearSelection2 True

    'Repeat all mates except the coincident mate with the "Right End@Universal Beam-1"
    Set arrMateEntities(0) = Nothing
    Set arrMateEntities(1) = Nothing
    boolstatus = swModelDocExt.SelectByID2("Left End@Universal Beam-1@mobile gantry", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    Set swItem = swSelMgr.GetSelectedObject6(1, -1)
    Set arrMateEntities(2) = swItem

    swModel.ClearSelection2 True

    blRepeat(0) = True
    blRepeat(1) = True
    blRepeat(2) = False

    SelectedComps = arrCompsToCopy
    RepeatOptions = blRepeat
    MateRefs = arrMateEntities

    dValues(0) = 0#
    dValues(1) = 0#
    dValues(2) = 0#
    InpValues = dValues

    blRepeat(0) = False
    blRepeat(1) = False
    blRepeat(2) = False
    FlipValues = blRepeat

    swAssy.CopyWithMates SelectedComps, RepeatOptions, MateRefs, InpValues, FlipValues
End Sub
```

Copy Components With Mates to Assembly Example (VB.NET)

This example shows how to copy components with mates in an assembly.

```

'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\driveworks\express\mobile gantry.SLDASM.
' 2. Inspect Leg<1> and Leg<2> in the assembly.
'
' Postconditions:
' 1. Replaces Leg<1> with a copy of Leg<2>.
' 2. Inspect Leg<2> and Leg<3> in the assembly.
'
' NOTE: Because this assembly is used elsewhere, do not save changes.
'-----

```

Imports SolidWorks.Interop.sldworks

Imports System.Runtime.InteropServices

Partial Class SolidWorksMacro

Sub main()

```

    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim swSelMgr As SelectionMgr
    Dim blRepeat(2) As Boolean
    Dim blFlip(2) As Boolean
    Dim SelectedComps As Object
    Dim RepeatOptions As Object
    Dim MateRefs As Object
    Dim InpValues As Object
    Dim FlipValues As Object
    Dim swAssy As AssemblyDoc
    Dim swItem As Component
    Dim swFeature As Feature
    Dim refPlane As RefPlane
    Dim arrMateEntities(2) As Object
    Dim arrCompsToCopy(0) As Object
    Dim boolstatus As Boolean
    Dim dValues(2) As Double

```

```

    swModel = swApp.ActiveDoc
    swAssy = swModel
    swSelMgr = swModel.SelectionManager
    swModelDocExt = swModel.Extension
    boolstatus = swModelDocExt.SelectByID2("Leg-1@mobile gantry", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    swModel.EditDelete()

```

```

    swModel.ClearSelection2(True)

```

```

    boolstatus = swModelDocExt.SelectByID2("Leg-2@mobile gantry", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
    swItem = CType(swSelMgr.GetSelectedObject6(1, -1), Component2)
    arrCompsToCopy(0) = swItem
    arrMateEntities(0) = Nothing
    arrMateEntities(1) = Nothing
    swModel.ClearSelection2(True)
    boolstatus = swModelDocExt.SelectByID2("Left End@Universal Beam-1@mobile gantry", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    swFeature = CType(swSelMgr.GetSelectedObject6(1, -1), Feature)
    refPlane = CType(swFeature.GetSpecificFeature2, RefPlane)
    arrMateEntities(2) = refPlane

```

```

    swModel.ClearSelection2(True)

```

```

    blRepeat(0) = True
    blRepeat(1) = True
    blRepeat(2) = False
    SelectedComps = arrCompsToCopy
    RepeatOptions = blRepeat
    MateRefs = arrMateEntities
    dValues(0) = 0.0

```

```
dValues(1) = 0.0
dValues(2) = 0.0
InpValues = dValues
blFlip(0) = False
blFlip(1) = False
blFlip(2) = False
FlipValues = blFlip
```

```
Dim disparrCompsToCopy(0) As Dispatcher
disparrCompsToCopy(0) = New Dispatcher(arrCompsToCopy(0))
```

```
Dim dispMateRefs(2) As Dispatcher
dispMateRefs(0) = New Dispatcher(MateRefs(0))
dispMateRefs(1) = New Dispatcher(MateRefs(1))
dispMateRefs(2) = New Dispatcher(MateRefs(2))
```

```
'Repeat all mates except the coincident mate with "Right End@Universal Beam-1"
swAssy.CopyWithMates((disparrCompsToCopy), RepeatOptions, (dispMateRefs), InpValues, FlipValues)
swAssy.EditRebuild()
```

```
End Sub
```

```
Public swApp As SldWorks
```

```
End Class
```

Copy Document and Its Dependencies Example (C#)

This example shows how to copy a document and its dependencies to this macro's folder.

```
//-----
// Preconditions: Open an assembly document.
//
// Postconditions:
// 1. Closes the assembly document.
// 2. Copies the assembly document and its dependencies to the macro folder.
// 3. Examine the macro folder.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace GetCurrentMacroPathName_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            string source = null;
            string target = null;
            string sourcefile = null;
            string tempfile = null;
            bool traverse = false;
            bool search = false;
            bool addreadonlyinfo = false;
            object[] depends = null;
            string[] sourcefiles = null;
            string[] targetfiles = null;
            int idx = 0;
            int sourcecount = 0;
            int copyopt = 0;
            int errors = 0;
            int lind = 0;
            ModelDoc2 doc = default(ModelDoc2);

            doc = (ModelDoc2)swApp.ActiveDoc;

            source = swApp.GetCurrentWorkingDirectory();
            sourcefile = doc.GetPathName();

            tempfile = swApp.GetCurrentMacroPathName();
            lind = tempfile.LastIndexOf(@"\");
            target = tempfile.Substring(0, lind);

            traverse = true;
            search = true;
            addreadonlyinfo = false;

            depends = (object[])swApp.GetDocumentDependencies2(sourcefile, traverse, search, addreadonlyinfo);

            if ((depends == null))
                return;

            idx = 1;

            while (idx <= depends.GetUpperBound(0))
            {
                Array.Resize(ref sourcefiles, sourcecount + 1);
                Array.Resize(ref targetfiles, sourcecount + 1);

                sourcefiles[sourcecount] = (string)depends[idx];
                lind = sourcefiles[sourcecount].LastIndexOf(@"\");
                targetfiles[sourcecount] = target + sourcefiles[sourcecount].Substring(lind, sourcefiles[sourcecount].Length - lind);

                sourcecount = sourcecount + 1;
                idx = idx + 2;
            }

            swApp.CloseAllDocuments(true);

            copyopt = (int)swMoveCopyOptions_e.swMoveCopyOptionsOverwriteExistingDocs;
            errors = swApp.CopyDocument(source + sourcefile, target + sourcefile, (sourcefiles), (targetfiles), (int)copyopt);
        }

        public SldWorks swApp;
    }
}
```


Copy Document and Its Dependencies Example (VBA)

This example shows how to copy a document and its dependencies to this macro's folder.

```

'-----
' Preconditions: Open an assembly document.
'
' Postconditions:
' 1. Closes the assembly document.
' 2. Copies the assembly document and its dependencies to the macro folder.
' 3. Examine the macro folder.
'-----
Option Explicit

Sub main()
    Dim swApp As SldWorks.SldWorks
    Dim source As String
    Dim target As String
    Dim sourcefile As String
    Dim traverse As Boolean
    Dim search As Boolean
    Dim addreadonlyinfo As Boolean
    Dim depends As Variant
    Dim sourcefiles() As String
    Dim targetfiles() As String
    Dim idx As Integer
    Dim sourcecount As Integer
    Dim copyopt As Long
    Dim errors As Long
    Dim doc As SldWorks.ModelDoc2

    Set swApp = Application.SldWorks
    Set doc = swApp.ActiveDoc

    source = swApp.GetCurrentWorkingDirectory
    sourcefile = doc.GetPathName

    target = swApp.GetCurrentMacroPathName
    target = Left(target, InStrRev(target, "\"))

    traverse = True
    search = True
    addreadonlyinfo = False

    depends = swApp.GetDocumentDependencies2(sourcefile, traverse, search, addreadonlyinfo)
    If IsEmpty(depends) Then Exit Sub

    idx = 1
    While idx <= UBound(depends)

        ReDim Preserve sourcefiles(sourcecount)
        ReDim Preserve targetfiles(sourcecount)

        sourcefiles(sourcecount) = depends(idx)
        targetfiles(sourcecount) = target + Right(sourcefiles(sourcecount), Len(sourcefiles(sourcecount)) - InStrRev(sourcefiles(sourcecount), "\"))

        sourcecount = sourcecount + 1
        idx = idx + 2
    Wend

    swApp.CloseAllDocuments True

    copyopt = SwConst.swMoveCopyOptions_e.swMoveCopyOptionsOverwriteExistingDocs
    errors = swApp.CopyDocument(source + sourcefile, target + sourcefile, (sourcefiles), (targetfiles), copyopt)
End Sub

```

Copy Document and Its Dependencies Example (VB.NET)

This example shows how to copy a document and its dependencies to this macro's folder.

```

'-----
' Preconditions: Open an assembly document.
'
' Postconditions:
' 1. Closes the assembly document.
' 2. Copies the assembly document and its dependencies to the macro folder.
' 3. Examine the macro folder.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Sub main()

        Dim source As String
        Dim target As String
        Dim sourcefile As String
        Dim traverse As Boolean
        Dim search As Boolean
        Dim addreadonlyinfo As Boolean
        Dim depends As Object
        Dim sourcefiles() As String = Nothing
        Dim targetfiles() As String = Nothing
        Dim idx As Integer
        Dim sourcecount As Integer
        Dim copyopt As Integer
        Dim errors As Integer
        Dim doc As ModelDoc2

        doc = swApp.ActiveDoc

        source = swApp.GetCurrentWorkingDirectory
        sourcefile = doc.GetPathName

        target = swApp.GetCurrentMacroPathName
        target = Left(target, InStrRev(target, "\"))

        traverse = True
        search = True
        addreadonlyinfo = False
        depends = swApp.GetDocumentDependencies2(sourcefile, traverse, search, addreadonlyinfo)

        If IsNothing(depends) Then Exit Sub

        idx = 1
        While idx <= UBound(depends)
            ReDim Preserve sourcefiles(sourcecount)
            ReDim Preserve targetfiles(sourcecount)

            sourcefiles(sourcecount) = depends(idx)
            targetfiles(sourcecount) = target + Right(sourcefiles(sourcecount), Len(sourcefiles(sourcecount)) -
InStrRev(sourcefiles(sourcecount), "\"))
            sourcecount = sourcecount + 1
            idx = idx + 2
        End While

        swApp.CloseAllDocuments(True)

        copyopt = swMoveCopyOptions_e.swMoveCopyOptionsOverwriteExistingDocs
        errors = swApp.CopyDocument(source + sourcefile, target + sourcefile, (sourcefiles), (targetfiles), copyopt)

    End Sub

    Public swApp As SldWorks

End Class

```

Copy Document Example (C++ COM)

This example shows how to copy documents.

```
// -----
void APITestFunction()
{
    HRESULT                hr = S_OK;
    VARIANT_BOOL           bRet = VARIANT_FALSE;
    long                   nRetVal = -1;
    CComPtr <ISldWorks>    pSldWorks;

    BSTR                   s1, s2, s3[1], s4[1];
    long                   error = -1;
    s1 = SysAllocString(_T("c:\\temp\\1\\assem1.sldasm"));
    s2 = SysAllocString(_T("c:\\temp\\2\\2\\assem2.sldasm"));
    s3[0] = SysAllocString(_T("c:\\temp\\1\\part1.sldprt"));
    s4[0] = SysAllocString(_T("c:\\temp\\2\\2\\part2.sldprt"));
    // connect to SW
    pSldWorks = TheApplication->GetSWApp();
    ASSERT(pSldWorks);
    hr = pSldWorks->ICopyDocument(
        s1, s2,
        1,
        s3, s4,
        swMoveCopyOptionsOverwriteExistingDocs + swMoveCopyOptionsCreateNewFolder,
        &error);
    return;
} //void APITestFunction()
// -----
```

Copy Document Example (VBA)

This example shows how to copy a document.

```
'-----  
' Preconditions:  
' 1. Verify that the specified file to copy exists.  
' 2. Verify that c:\temp exists.  
' 3. Open the Immediate window.  
'  
' Postconditions:  
' 1. Copies C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cstick.sldprt  
'    to c:\temp\cstick.sldprt. If c:\temp\cstick.sldprt already exists, the file is  
'    overwritten.  
' 2. Examine the Immediate window and c:\temp.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim fileName  
Dim retVal As Long  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\cstick.sldprt"  
    retVal = swApp.CopyDocument(fileName, "c:\temp\cstick.sldprt", "", "", swMoveCopyOptionsOverwriteExistingDocs)  
    Debug.Print "Status of copy (0 = success): " & retVal  
  
End Sub
```

Create, Unlink, and Purge Display States in Part Example (C#)

This example shows how to create, unlink, and purge display states in a part document.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\multibody\multi-inter.sldprt,
//    whose Default configuration has two display states:
//    * PhotoWorks Display
//    * Display State 1
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Creates and unlinks Display State 2.
// 2. Attempts to purge any display states identical to
//    Display State 2.
// 3. Closes the part document without saving any changes.
// 4. Examine the Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;
namespace AddAndPurgeDisplayStates_CSharp.csproj
{
    partial
    class SolidWorksMacro
    {
        public PartDoc swPart;

        public
        void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            Configuration swConfig = default(Configuration);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            bool boolstatus = false;
            string modelName = null;
            swModel = (ModelDoc2)swApp.ActiveDoc;

            // Get Default configuration and create a display state
            swConfig = (Configuration)swModel.GetConfigurationByName("Default");
            boolstatus = swConfig.CreateDisplayState("Display State 2");
            if (boolstatus) Debug.Print("Display State 2 created.");
            swModel.ForceRebuild3(true);

            // If display is linked, unlink it
            swModelDocExt = swModel.Extension;
            Debug.Print("Is Display State 2 linked? " + swModelDocExt.LinkedDisplayState);
            swModelDocExt.LinkedDisplayState = false;
            Debug.Print("Is Display State 2 still linked? " + swModelDocExt.LinkedDisplayState);

            // Purge any display states identical to Display State 2
            boolstatus = swModelDocExt.PurgeDisplayState();
            Debug.Print("Identical display states to Display State 2 purged? " + boolstatus);
            swModel.ForceRebuild3(true);

            // Close the part without saving changes
            modelName = swModel.GetTitle();
            swApp.QuitDoc(modelName);
        }
    }
    public SldWorks swApp;
}
}
```

Create, Unlink, and Purge Display States in Part Example (VBA)

This example shows how to create, unlink, and purge display states in a part document.

```
' -----
' Preconditions:
' 1. Open public_documents\samples\tutorial\multibody\multi-inter.sldprt,
'   whose Default configuration has two display states:
'   * PhotoWorks Display
'   * Display State 1
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates and unlinks Display State 2.
' 2. Attempts to purge any display states identical to
'   Display State 2.
' 3. Closes the part document without saving any changes.
' 4. Examine the Immediate window.
' -----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swConfig As SldWorks.Configuration
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim boolstatus As Boolean
Dim modelName As String

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    ' Get Default configuration and create a display state
    Set swConfig = swModel.GetConfigurationByName("Default")
    boolstatus = swConfig.CreateDisplayState("Display State 2")
    If boolstatus Then Debug.Print "Display State 2 created."
    swModel.ForceRebuild3 True

    ' If display is linked, unlink it
    Set swModelDocExt = swModel.Extension
    Debug.Print "Is Display State 2 linked? " & swModelDocExt.LinkedDisplayState
    swModelDocExt.LinkedDisplayState = False
    Debug.Print "Is Display State 2 still linked? " & swModelDocExt.LinkedDisplayState

    ' Purge any display states identical to Display State 2
    boolstatus = swModelDocExt.PurgeDisplayState
    Debug.Print "Identical display states to Display State 2 purged? " & boolstatus
    swModel.ForceRebuild3 True

    ' Close the part without saving changes
    modelName = swModel.GetTitle
    swApp.QuitDoc modelName

End Sub
```

Create, Unlink, and Purge Display States in Part Example (VB.NET)

This example shows how to create, unlink, and purge display states in a part document.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\multibody\multi-inter.sldprt,
'    whose Default configuration has two display states:
'    * PhotoWorks Display
'    * Display State 1
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates and unlinks Display State 2.
' 2. Attempts to purge any display states identical to
'    Display State 2.
' 3. Closes the part document without saving any changes.
' 4. Examine the Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swConfig As Configuration
        Dim swModelDocExt As ModelDocExtension
        Dim boolstatus As Boolean
        Dim modelName As String

        swModel = swApp.ActiveDoc

        ' Get Default configuration and create a display state
        swConfig = (swModel.GetConfigurationByName("Default"))
        boolstatus = swConfig.CreateDisplayState("Display State 2")
        If boolstatus Then Debug.Print("Display State 2 created.")
        swModel.ForceRebuild3(True)

        ' If display is linked, unlink it
        swModelDocExt = swModel.Extension
        Debug.Print("Is Display State 2 linked? " & swModelDocExt.LinkedDisplayState)
        swModelDocExt.LinkedDisplayState = False
        Debug.Print("Is Display State 2 still linked? " & swModelDocExt.LinkedDisplayState)

        ' Purge any display states identical to Display State 2
        boolstatus = swModelDocExt.PurgeDisplayState
        Debug.Print("Identical display states to Display State 2 purged? " & boolstatus)
        swModel.ForceRebuild3(True)

        ' Close the part without saving changes
        modelName = swModel.GetTitle
        swApp.QuitDoc(modelName)

    End Sub

End Class
```

Create 2D Spline Example (C#)

This topic shows how to create a 2D spline.

```
//-----
// Preconditions: Open a new part.
//
// Postconditions:
// 1. Creates a sketch of a 2D spline.
// 2. Examine the graphics area.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace Create2DSpline_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModelDoc;
        ModelDocExtension swModelDocExt;
        int i;
        int j;
        double[] x = new double[6];
        double[] y = new double[6];

        bool boolstatus;

        public void Main()
        {
            swModelDoc = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = swModelDoc.Extension;
            boolstatus = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);

            // Open sketch
            swModelDoc.InsertSketch2(true);

            // Calculate the values for x, y, and z
            for (i = 0; i <= 4; i++)
            {
                x[i] = i;
                y[i] = Math.Pow(i, 2) + 3 * i;
            }

            // Initialize the routine and sketch the first point of the spline at 0,0,0
            swModelDoc.SketchSpline(-1, 0, 0, 0);

            // Sketch four more points of the spline
            for (j = 4; j >= 0; j += -1)
            {
                swModelDoc.SketchSpline(j, x[j], y[j], 0);
            }

            // Exit sketch
            swModelDoc.InsertSketch2(true);
        }

        public SldWorks swApp;
    }
}
```

Create 2D Spline Example (VBA)

This topic shows how to create a 2D spline.

```
'-----  
' Preconditions: Open a new part.  
'  
' Postconditions:  
' 1. Creates a sketch of a 2D spline.  
' 2. Examine the graphics area.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim swModelDoc As SldWorks.ModelDoc2  
Dim swModelDocExt As SldWorks.ModelDocExtension  
Dim i As Long  
Dim j As Long  
Dim x(5) As Double  
Dim y(5) As Double  
Dim boolstatus As Boolean  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModelDoc = swApp.ActiveDoc  
    Set swModelDocExt = swModelDoc.Extension  
    boolstatus = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)  
  
    ' Open sketch  
    swModelDoc.InsertSketch2 True  
  
    ' Calculate the values for x, y, and z  
    For i = 0 To 4  
        x(i) = i  
        y(i) = i ^ 2 + 3 * i  
    Next i  
  
    ' Initialize the routine and sketch the first point of the spline at 0,0,0  
    swModelDoc.SketchSpline -1, 0, 0, 0  
  
    ' Sketch four more points of the spline  
    For j = 4 To 0 Step -1  
        swModelDoc.SketchSpline j, x(j), y(j), 0  
    Next j  
  
    ' Exit sketch  
    swModelDoc.InsertSketch2 True  
End Sub
```

Create 2D Spline Example (VB.NET)

This topic shows how to create a 2D spline.

' Preconditions: Open a new part.

' Postconditions:

- ' 1. Creates a sketch of a 2D spline.
- ' 2. Examine the graphics area.

Imports SolidWorks.Interop.sldworks

Imports SolidWorks.Interop.swconst

Imports System.Runtime.InteropServices

Imports System

Partial Class SolidWorksMacro

Dim swModelDoc As ModelDoc2

Dim swModelDocExt As ModelDocExtension

Dim i As Integer

Dim j As Integer

Dim x(5) As Double

Dim y(5) As Double

Dim boolstatus As Boolean

Sub main()

swModelDoc = swApp.ActiveDoc

swModelDocExt = swModelDoc.Extension

boolstatus = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)

' Open sketch

swModelDoc.InsertSketch2(True)

' Calculate the values for x, y, and z

For i = 0 To 4

x(i) = i

y(i) = i ^ 2 + 3 * i

Next i

' Initialize the routine and sketch the first point of the spline at 0,0,0

swModelDoc.SketchSpline(-1, 0, 0, 0)

' Sketch four more points of the spline

For j = 4 To 0 Step -1

swModelDoc.SketchSpline(j, x(j), y(j), 0)

Next j

' Exit sketch

swModelDoc.InsertSketch2(True)

End Sub

Public swApp As SldWorks

End Class

Create 360° Revolve Feature Example (C#)

This example shows how to create a 360° revolve feature.

```
//-----
// Preconditions:
// 1. Open a part document that contains Axis1
//    and Sketch1 features; Sketch1 must be a rectangle and Axis1
//    must have been created using an edge on the rectangle.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Creates a 360° revolve feature.
// 2. Examine the Immediate window, graphics area, and FeatureManager
//    design tree.
//-----

using Microsoft.VisualBasic;

using System;

using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;

namespace RevolveFeatDataType_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;

        ModelDocExtension swModelDocExt;

        FeatureManager swFeatMgr;

        Feature swFeat;

        RevolveFeatureData2 swRevolveFeat;

        bool boolstatus;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;

            swModelDocExt = swModel.Extension;

            boolstatus = swModelDocExt.SelectByID("Axis1", "AXIS", 0, 0, 0, false,
16, null,
0);

            boolstatus = swModelDocExt.SelectByID("Sketch1", "SKETCH",
0, 0, 0, true,
0, null,
0);

            swFeatMgr = swModel.FeatureManager;

            swFeat = swFeatMgr.FeatureRevolve2(true, true, false, false, false, false, 0,
0, 6.2831853071796, 0,
false, false,
0.01, 0.01, 0, 0, 0, true, true, true);

            swModel.ViewZoomtofit2();

            swRevolveFeat = (RevolveFeatureData2)swFeat.GetDefinition();

            // Set the type of revolve as
            defined in swRevolveType_e
        }
    }
}
```

```
        Debug.Print("Revolve feature type before setting to 5: " + swRevolveFeat.Type.ToString());

        swRevolveFeat.Type = 5;

        boolstatus = swFeat.ModifyDefinition(swRevolveFeat, swModel, null);

        Debug.Print("Revolve feature type after setting to 5: " + swRevolveFeat.Type.ToString());

    }

    public SldWorks swApp;

}

}
```

Create 360° Revolve Feature Example (VBA)

This example shows how to create a 360° revolve feature.

```
'-----
' Preconditions:
' 1. Open a part document that contains Axis1
'    and Sketch1 features; Sketch1 must be a rectangle and Axis1
'    must have been created using an edge on the rectangle.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates a 360° revolve feature.
' 2. Examine the Immediate window, graphics area, and FeatureManager
'    design tree.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatMgr As SldWorks.FeatureManager
Dim swFeat As SldWorks.Feature
Dim swRevolveFeat As SldWorks.RevolveFeatureData2
Dim boolstatus As Boolean

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension

    boolstatus = swModelDocExt.SelectByID2("Sketch1", "SKETCH", 0, 0, 0, False, 0, Nothing, 0)
    boolstatus = swModelDocExt.SelectByID2("Axis1", "AXIS", 0, 0, 0, True, 16, Nothing, 0)

    Set swFeatMgr = swModel.FeatureManager
    Set swFeat = swFeatMgr.FeatureRevolve2(True, True, False, False, False, False, 0, 0, 6.2831853071796, 0, False, False, 0.01, 0.01, 0, 0, 0, True, True, True)

    swModel.ViewZoomtofit2

    Set swRevolveFeat = swFeat.GetDefinition

    ' Set the type of revolve as defined in swRevolveType_e
    Debug.Print "Revolve feature type before setting to 5: " & swRevolveFeat.Type
    swRevolveFeat.Type = 5
    boolstatus = swFeat.ModifyDefinition(swRevolveFeat, swModel, Nothing)
    Debug.Print "Revolve feature type after setting to 5: " & swRevolveFeat.Type
End Sub
```

Create 360° Revolve Feature Example (VB.NET)

This example shows how to create a 360° revolve feature.

```
'-----
' Preconditions:
' 1. Open a part document that contains Axis1
'    and Sketch1 features; Sketch1 must be a rectangle and Axis1
'    must have been created using an edge on the rectangle.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Creates a 360° revolve feature.
' 2. Examine the Immediate window, graphics area, and FeatureManager
'    design tree.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim swFeatMgr As FeatureManager
    Dim swFeat As Feature
    Dim swRevolveFeat As RevolveFeatureData2
    Dim boolstatus As Boolean

    Sub main()

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension

        boolstatus = swModelDocExt.SelectByID2("Axis1", "AXIS", 0, 0, 0, False,
16, Nothing,
0)

        boolstatus = swModelDocExt.SelectByID2("Sketch1", "SKETCH",
0, 0, 0, True,
0, Nothing,
0)

        swFeatMgr = swModel.FeatureManager

        swFeat = swFeatMgr.FeatureRevolve2(True, True, False, False, False, False, 0,
0.2831853071796, 0, False, False,
0.01, 0.01, 0, 0, 0, True, True, True)

        swModel.ViewZoomtofit2()

        swRevolveFeat = swFeat.GetDefinition

        ' Set the type of revolve as
defined in swRevolveType_e
        Debug.Print("Revolve feature type
before setting to 5: " & swRevolveFeat.Type)

        swRevolveFeat.Type = 5

        boolstatus = swFeat.ModifyDefinition(swRevolveFeat, swModel, Nothing)

        Debug.Print("Revolve feature type after setting to 5: " & swRevolveFeat.Type)

    End Sub

Public swApp As SldWorks
```

End Class

Create 3-Point Arc Example (C#)

This example shows how to create a 3-point arc in a sketch.

```
//-----
// Preconditions: Open a new part.
//
// Postconditions:
// 1. Creates a 3-point arc in Sketch1.
// 2. Examine the graphics area and FeatureManager design tree.
// -----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace Create3PointArc_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        SketchSegment skSegment;
        bool boolstatus;
        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            Part.SketchManager.InsertSketch(true);
            boolstatus = Part.Extension.SelectByID2("Top Plane", "PLANE", -0.0603680341302297, 0.00853710569474597,
0.0260554052320825, false, 0, null, 0);
            Part.ClearSelection2(true);
            skSegment = Part.SketchManager.Create3PointArc(-0.043992, 0.026681, 0.0, 0.033828, 0.037798, 0.0, 0.01604, 0.014611,
0.0);
            Debug.Print("The sketch is 3D? " + skSegment.GetSketch().Is3D());
            Part.ClearSelection2(true);
            Part.SketchManager.InsertSketch(true);
        }
        public SldWorks swApp;
    }
}
```

Create 3-Point Arc Example (VBA)

This example shows how to create a 3-point arc in a sketch.

```
'-----  
' Preconditions: Open a new part.  
'  
' Postconditions:  
' 1. Creates a 3-point arc in Sketch1.  
' 2. Examine the graphics area and FeatureManager design tree.  
'-----  
  
Dim swApp As SldWorks.SldWorks  
Dim Part As SldWorks.ModelDoc2  
Dim skSegment As SldWorks.SketchSegment  
Dim boolstatus As Boolean  
Option Explicit  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set Part = swApp.ActiveDoc  
    Part.SketchManager.InsertSketch True  
    boolstatus = Part.Extension.SelectByID2("Top Plane", "PLANE", -6.03680341302297E-02, 8.53710569474597E-03, 2.60554052320825E-  
02, False, 0, Nothing, 0)  
    Part.ClearSelection2 True  
    Set skSegment = Part.SketchManager.Create3PointArc(-0.043992, 0.026681, 0#, 0.033828, 0.037798, 0#, 0.01604, 0.014611, 0#)  
    Debug.Print "The sketch is 3D? " & skSegment.GetSketch.Is3D  
    Part.ClearSelection2 True  
    Part.SketchManager.InsertSketch True  
End Sub
```

Create 3-Point Arc Example (VB.NET)

This example shows how to create a 3-point arc in a sketch.

```

'-----
' Preconditions: Open a new part.
'
' Postconditions:
' 1. Creates a 3-point arc in Sketch1.
' 2. Examine the graphics area and FeatureManager design tree.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim Part As ModelDoc2
    Dim skSegment As SketchSegment
    Dim boolstatus As Boolean

    Sub main()
        Part = swApp.ActiveDoc
        Part.SketchManager.InsertSketch(True)
        boolstatus = Part.Extension.SelectByID2("Top Plane", "PLANE", -0.0603680341302297, 0.00853710569474597, 0.0260554052320825,
False, 0, Nothing, 0)
        Part.ClearSelection2(True)
        skSegment = Part.SketchManager.Create3PointArc(-0.043992, 0.026681, 0.0#, 0.033828, 0.037798, 0.0#, 0.01604, 0.014611,
0.0#)
        Debug.Print("The sketch is 3D? " & skSegment.GetSketch.Is3D)
        Part.ClearSelection2(True)
        Part.SketchManager.InsertSketch(True)
    End Sub

    Public swApp As SldWorks
End Class

```

Create 3D Bounding Box for Cut List Item Example (C#)

This example shows how to create a 3D bounding box for a cut list item in a weldment part.

```
//-----
// Preconditions:
// 1. Open: public_documents\samples\tutorial\api\weldment_box3.sldprt.
// 2. Right-click the Cut list folder and select Update.
//
// Postconditions:
// 1. Expand the Cut-List-Item5 folder.
// 2. Select Bounding-Box_Cut-List-Item5.
// 3. Observe the sketch of the bounding box in the graphics area.
//
// NOTE: Because the model is used elsewhere, do not save changes when closing it.
// -----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace CreateBoundingBox_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        ModelDocExtension modDocExt;

        bool boolstatus;
        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            modDocExt = Part.Extension;

            boolstatus = modDocExt.SelectByID2("Cut-List-Item5", "SUBWELDFOLDER", 0, 0, 0, false, 0, null, 0);
            modDocExt.Create3DBoundingBox();
        }

        public SldWorks swApp;
    }
}
```

Create 3D Bounding Box for Cut List Item Example (VBA)

This example shows how to create a 3D bounding box for a cut list item in a weldment part.

```
'-----  
' Preconditions:  
' 1. Open: public_documents\samples\tutorial\api\weldment_box3.sldprt.  
' 2. Right-click the Cut list folder and select Update.  
'  
' Postconditions:  
' 1. Expand the Cut-List-Item5 folder.  
' 2. Select Bounding-Box_Cut-List-Item5.  
' 3. Observe the sketch of the bounding box in the graphics area.  
'  
' NOTE: Because the model is used elsewhere, do not save changes when closing it.  
'-----
```

```
Dim swApp As SldWorks.SldWorks  
Dim Part As SldWorks.ModelDoc2  
Dim modDocExt As SldWorks.ModelDocExtension  
Dim boolstatus As Boolean  
Option Explicit  
Sub main()  
    Set swApp = Application.SldWorks  
    Set Part = swApp.ActiveDoc  
    Set modDocExt = Part.Extension  
  
    boolstatus = modDocExt.SelectByID2("Cut-List-Item5", "SUBWELDFOLDER", 0, 0, 0, False, 0, Nothing, 0)  
    modDocExt.Create3DBoundingBox  
End Sub
```

Create 3D Bounding Box for Cut List Item Example (VB.NET)

This example shows how to create a 3D bounding box for a cut list item in a weldment part.

```
'-----  
' Preconditions:  
' 1. Open: public_documents\samples\tutorial\api\weldment_box3.sldprt.  
' 2. Right-click the Cut list folder and select Update.  
'  
' Postconditions:  
' 1. Expand the Cut-List-Item5 folder.  
' 2. Select Bounding-Box_Cut-List-Item5.  
' 3. Observe the sketch of the bounding box in the graphics area.  
'  
' NOTE: Because the model is used elsewhere, do not save changes when closing it.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Dim Part As ModelDoc2  
    Dim modDocExt As ModelDocExtension  
    Dim boolstatus As Boolean  
  
    Sub main()  
        Part = swApp.ActiveDoc  
        modDocExt = Part.Extension  
  
        boolstatus = modDocExt.SelectByID2("Cut-List-Item5", "SUBWELDFOLDER", 0, 0, 0, False, 0, Nothing, 0)  
        modDocExt.Create3DBoundingBox()  
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Create 3D Contact Feature Example (C#)

This example shows how to create a 3D Contact feature for use in motion studies.

```
//-----
// Preconditions:
// 1. Verify that the specified model document exists.
// 2. In SOLIDWORKS Design, verify that View > MotionManager is selected.
// 3. In the IDE, right-click the project name in the Project Explorer,
//    click Add Reference, browse to install_dir\api\redist, and select
//    SolidWorks.Interop.swmotionstudy.dll.
// 4. Click OK.
// 5. Open an Immediate window.
//
// Postconditions:
// 1. Selects the components to study.
// 2. Sets the static contact friction option.
// 3. Adds Solid Body Contact1 to the MotionManager design tree on the
//    Motion Study 1 tab.
// 4. Inspect the Immediate window.
//
// NOTE: Because the model is used elsewhere,
// do not save changes when closing it.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swmotionstudy;
using System.Runtime.InteropServices;

namespace CreateSim3DContFeat_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            MotionStudyManager swMotionMgr = default(MotionStudyManager);
            MotionStudy swMotionStudy1 = default(MotionStudy);
            Simulation3DContactFeatureData swContFeat = default(Simulation3DContactFeatureData);
            bool boolstatus = false;
            Feature swFeat = default(Feature);
            int longstatus = 0;
            int longwarnings = 0;

            swModel = swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\api\\beam_boltconnection.sldasm", 2, 0, "", ref longstatus, ref longwarnings);
            swApp.ActivateDoc2("beam_boltconnection", false, ref longstatus);
            swModel = (ModelDoc2)swApp.ActiveDoc;

            swModelDocExt = swModel.Extension;
            swMotionMgr = (MotionStudyManager)swModelDocExt.GetMotionStudyManager();
            swMotionStudy1 = swMotionMgr.GetMotionStudy("Motion Study 1");

            if ((swMotionStudy1 == null))
            {
                Debug.Print("Motion Study 1 is not available");
            }

            swMotionStudy1.StudyType = (int)swMotionStudyType_e.swMotionStudyTypePhysicalSimulation;

            // Activate Motion Study 1
            swMotionStudy1.Activate();

            // Create the feature data object
            swContFeat = (Simulation3DContactFeatureData)swMotionStudy1.CreateDefinition((int)swFeatureNameID_e.swFmAEM3DContact);

            if (swContFeat == null)
            {
                Debug.Print("Failed to create feature data object");
            }

            SelectionMgr swSelMgr = default(SelectionMgr);
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            // Select the components to study
            boolstatus = swModelDocExt.SelectByID2("Beam with holes-
1@beam_boltconnection", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            boolstatus = swModelDocExt.SelectByID2("Beam with holes-
2@beam_boltconnection", "COMPONENT", 0, 0, 0, true, 0, null, 0);
        }
    }
}
```

```

// Set the 3D Contact components
Component[] ContactObj = new Component[2];
ContactObj[0] = (Component)swSelMgr.GetSelectedObject6(1, -1);
ContactObj[1] = (Component)swSelMgr.GetSelectedObject6(2, -1);

swContFeat.ContactComponents = ContactObj;

// Set the static contact friction option
swContFeat.FrictionOption = (int)swMotionContactFrictionType_e.swMotionContactFrictionFull;

// Create a 3D Contact feature
swFeat = (Feature)swMotionStudy1.CreateFeature(swContFeat);

boolstatus = swModelDocExt.SelectByID2("Solid Body Contact1", "SIMULATION_ELEMENT", 0, 0, 0, false, 0, null, 0);

if (swFeat == null)
{
    Debug.Print("Failed to create feature");
}
else
{
    Debug.Print(swFeat.Name);
    if (swContFeat.UseRestitutionCoefficient)
    {
        Debug.Print("    Use coefficient of restitution:");
        Debug.Print("    Coefficient of restitution: " + swContFeat.RestitutionCoefficient);
    }
    else
    {
        Debug.Print("    Use impact force:");
        Debug.Print("    Exponent of exponential force: " + swContFeat.Exponent);
        Debug.Print("    Maximum damping coefficient: " + swContFeat.MaxDamping);
        Debug.Print("    Penetration at which maximum damping is applied: " + swContFeat.Penetration);
        Debug.Print("    Stiffness of material at boundary of interaction: " + swContFeat.Stiffness);
    }
    if (swContFeat.SpecifyMaterial)
    {
        Debug.Print("    Type of material as defined by swCosmosWorksMat:");
        Debug.Print("    First object: " + swContFeat.get_MaterialID(0));
        Debug.Print("    Second object: " + swContFeat.get_MaterialID(1));
    }
    if (swContFeat.FrictionOption == (int)swMotionContactFrictionType_e.swMotionContactFrictionFull)
    {
        Debug.Print("    Static contact friction:");
        Debug.Print("    Coefficient: " + swContFeat.StaticFrictionCoefficient);
        Debug.Print("    Velocity: " + swContFeat.StaticFrictionVelocity);
    }
    else
    {
        Debug.Print("    Dynamic contact friction:");
        Debug.Print("    Coefficient: " + swContFeat.DynamicFrictionCoefficient);
        Debug.Print("    Velocity: " + swContFeat.DynamicFrictionVelocity);
    }
}
}
}

```

```

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>

```

```
public SldWorks swApp;
```

```

}
}

```

Create 3D Contact Feature Example (VB.NET)

This example shows how to create a 3D Contact feature for use in motion studies.

```

'-----
' Preconditions:
' 1. Verify that the specified model document exists.
' 2. In SOLIDWORKS Design, verify that View > MotionManager is selected.
' 3. In the IDE, right-click the project name in the Project Explorer,
'    click Add Reference, browse to install_dir\api\redist, and select
'    SolidWorks.Interop.swmotionstudy.dll.
' 4. Click OK.
' 5. Open an Immediate window.
'
' Postconditions:
' 1. Selects the components to study.
' 2. Sets the static contact friction option.
' 3. Adds Solid Body Contact1 to the MotionManager design tree on the
'    Motion Study 1 tab.
' 4. Inspect the Immediate window.
'
' NOTE: Because the model is used elsewhere,
' do not save changes when closing it.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swmotionstudy
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

```

Partial Class SolidWorksMacro

```

    Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swMotionMgr As MotionStudyManager
        Dim swMotionStudy1 As MotionStudy
        Dim swContFeat As Simulation3DContactFeatureData
        Dim boolstatus As Boolean
        Dim swFeat As Feature
        Dim longstatus As Integer, longwarnings As Integer

        swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\api\beam_boltconnection.sldasm", 2, 0, "", longstatus, longwarnings)
        swApp.ActivateDoc2("beam_boltconnection", False, longstatus)
        swModel = swApp.ActiveDoc

        swModelDocExt = swModel.Extension
        swMotionMgr = swModelDocExt.GetMotionStudyManager()
        swMotionStudy1 = swMotionMgr.GetMotionStudy("Motion Study 1")

        If (swMotionStudy1 Is Nothing) Then
            Debug.Print("Motion Study 1 is not available")
        End If

        swMotionStudy1.StudyType = swMotionStudyType_e.swMotionStudyTypePhysicalSimulation

        ' Activate Motion Study 1
        swMotionStudy1.Activate()

        ' Create the feature data object
        swContFeat = swMotionStudy1.CreateDefinition(swFeatureNameID_e.swFmAEM3DContact)

        If swContFeat Is Nothing Then
            Debug.Print("Failed to create feature data object")
        End If

        Dim swSelMgr As SelectionMgr
        swSelMgr = swModel.SelectionManager

        ' Select the components to study
        boolstatus = swModelDocExt.SelectByID2("Beam with holes-1@beam_boltconnection", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        boolstatus = swModelDocExt.SelectByID2("Beam with holes-2@beam_boltconnection", "COMPONENT", 0, 0, 0, True, 0, Nothing, 0)

        ' Set the 3D Contact components
        Dim ContactObj(1) As Component
        ContactObj(0) = swSelMgr.GetSelectedObject6(1, -1)
        ContactObj(1) = swSelMgr.GetSelectedObject6(2, -1)

        swContFeat.ContactComponents = ContactObj

        ' Set the static contact friction option
        swContFeat.FrictionOption = swMotionContactFrictionType_e.swMotionContactFrictionFull

        ' Create a 3D Contact feature
    End Sub

```

```

swFeat = swMotionStudy1.CreateFeature(swContFeat)

boolstatus = swModelDocExt.SelectByID2("Solid Body Contact1", "SIMULATION_ELEMENT", 0, 0, 0, False, 0, Nothing, 0)

If swFeat Is Nothing Then
    Debug.Print("Failed to create feature")
Else
    Debug.Print(swFeat.Name)
    If swContFeat.UseRestitutionCoefficient Then
        Debug.Print("    Use coefficient of restitution:")
        Debug.Print("    Coefficient of restitution: " & swContFeat.RestitutionCoefficient)
    Else
        Debug.Print("    Use impact force:")
        Debug.Print("    Exponent of exponential force: " & swContFeat.Exponent)
        Debug.Print("    Maximum damping coefficient: " & swContFeat.MaxDamping)
        Debug.Print("    Penetration at which maximum damping is applied: " & swContFeat.Penetration)
        Debug.Print("    Stiffness of material at boundary of interaction: " & swContFeat.Stiffness)
    End If
    If swContFeat.SpecifyMaterial Then
        Debug.Print("    Type of material as defined by swCosmosWorksMat:")
        Debug.Print("    First object: " & swContFeat.MaterialID(0))
        Debug.Print("    Second object: " & swContFeat.MaterialID(1))
    End If
    If swContFeat.FrictionOption = swMotionContactFrictionType_e.swMotionContactFrictionFull Then
        Debug.Print("    Static contact friction:")
        Debug.Print("    Coefficient: " & swContFeat.StaticFrictionCoefficient)
        Debug.Print("    Velocity: " & swContFeat.StaticFrictionVelocity)
    Else
        Debug.Print("    Dynamic contact friction:")
        Debug.Print("    Coefficient: " & swContFeat.DynamicFrictionCoefficient)
        Debug.Print("    Velocity: " & swContFeat.DynamicFrictionVelocity)
    End If
End If

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

End Class

```

Create 3D Sketch Plane Example (C#)

This example shows how to create a 3D sketch plane.

```
//-----
// Preconditions: Verify that the specified part template exists.
//
// Postconditions:
// 1. Inserts a 3D sketch of two lines.
// 2. Inserts a 2D sketch of a circle.
// 3. Selects a line in the 3D sketch and the center of the circle
//    in the 2D sketch.
// 4. Inserts a 3D sketch plane.
// 5. Examine the graphics area and the FeatureManager design tree.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Create3DSketchPlaneCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchManager swSketchManager = default(SketchManager);
            SketchSegment swSketchSegment = default(SketchSegment);
            Sketch swSketch = default(Sketch);
            bool status = false;

            //Open new part document
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SolidWorks 2015\\templates\\Part.prtdot", 0, 0, 0);

            //Insert 3D sketch of two lines
            swSketchManager = (SketchManager)swModel.SketchManager;
            swSketchManager.Insert3DSketch(true);
            swSketchSegment = (SketchSegment)swSketchManager.CreateCenterLine(-0.082642, 0.005659, 0.0, -0.049926, 0.045073, 0.0);
            swSketch = (Sketch)swSketchManager.ActiveSketch;
            status = swSketch.SetWorkingPlaneOrientation(0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0);
            swSketchSegment = (SketchSegment)swSketchManager.CreateCenterLine(-0.049926, 0.045073, 0.0, -0.049926, -0.022634, -0.065874);
            swSketch = (Sketch)swSketchManager.ActiveSketch;
            status = swSketch.SetWorkingPlaneOrientation(0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0);
            swModel.ClearSelection2(true);
            swSketchManager.InsertSketch(true);

            //Insert 2D sketch of a circle
            swModel.ActivateSelectedFeature();
            swModel.ClearSelection2(true);
            swSketchManager.InsertSketch(true);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swModel.ClearSelection2(true);
            swSketchSegment = (SketchSegment)swSketchManager.CreateCircle(-0.056401, 0.005985, 0.0, -0.054697, -0.005141, 0.0);
            swModel.ClearSelection2(true);
            swSketchManager.InsertSketch(true);
            swModel.ClearSelection2(true);

            //Insert a 3D sketch plane
            swSketchManager.Insert3DSketch(true);
            status = swModelDocExt.SelectByID2("Line1@3DSketch1", "EXTSKETCHSEGMENT", -0.0565609614209999, 0.0370796232466087, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("Point2@Sketch1", "EXTSKETCHPOINT", -0.0564010297276809, 0.00598490302365917, 0, true, 0, null, 0);
            status = swSketchManager.CreateSketchPlane(9, 9, 0);
            status = swModelDocExt.SelectByID2("Plane1", "SKETCHSURFACES", 0, 0, 0, false, 0, null, 0);
            swModel.ActivateSelectedFeature();
            swModel.ClearSelection2(true);
            swSketchManager.InsertSketch(true);
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}
```

Create 3D Sketch Plane Example (VBA)

This example shows how to create a 3D sketch plane.

```
'-----
' Preconditions: Verify that the specified part template exists.
'
' Postconditions:
' 1. Inserts a 3D sketch of two lines.
' 2. Inserts a 2D sketch of a circle.
' 3. Selects a line in the 3D sketch and the center of the circle
'    in the 2D sketch.
' 4. Inserts a 3D sketch plane.
' 5. Examine the graphics area and the FeatureManager design tree.
'-----
Option Explicit
```

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchManager As SldWorks.SketchManager
Dim swSketchSegment As SldWorks.SketchSegment
Dim swSketch As SldWorks.Sketch
Dim status As Boolean
```

```
Sub main()
```

```
    Set swApp = Application.SldWorks
```

```
    'Open new part document
```

```
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2015\templates\Part.prtdot", 0, 0, 0)
```

```
    'Insert 3D sketch of two lines
```

```
    Set swSketchManager = swModel.SketchManager
```

```
    swSketchManager.Insert3DSketch True
```

```
    Set swSketchSegment = swSketchManager.CreateCenterLine(-0.082642, 0.005659, 0#, -0.049926, 0.045073, 0#)
```

```
    Set swSketch = swSketchManager.ActiveSketch
```

```
    status = swSketch.SetWorkingPlaneOrientation(0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0)
```

```
    Set swSketchSegment = swSketchManager.CreateCenterLine(-0.049926, 0.045073, 0#, -0.049926, -0.022634, -0.065874)
```

```
    Set swSketch = swSketchManager.ActiveSketch
```

```
    status = swSketch.SetWorkingPlaneOrientation(0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0)
```

```
    swModel.ClearSelection2 True
```

```
    swSketchManager.InsertSketch True
```

```
    'Insert 2D sketch of a circle
```

```
    swModel.ActivateSelectedFeature
```

```
    swModel.ClearSelection2 True
```

```
    swSketchManager.InsertSketch True
```

```
    Set swModelDocExt = swModel.Extension
```

```
    status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
```

```
    swModel.ClearSelection2 True
```

```
    Set swSketchSegment = swSketchManager.CreateCircle(-0.056401, 0.005985, 0#, -0.054697, -0.005141, 0#)
```

```
    swModel.ClearSelection2 True
```

```
    swSketchManager.InsertSketch True
```

```
    swModel.ClearSelection2 True
```

```
    'Insert a 3D sketch plane
```

```
    swSketchManager.Insert3DSketch True
```

```
    status = swModelDocExt.SelectByID2("Line1@3DSketch1", "EXTSKETCHSEGMENT", -5.65609614209999E-02, 3.70796232466087E-02, 0, True, 0, Nothing, 0)
```

```
    status = swModelDocExt.SelectByID2("Point2@Sketch1", "EXTSKETCHPOINT", -5.64010297276809E-02, 5.98490302365917E-03, 0, True, 0, Nothing, 0)
```

```
    status = swSketchManager.CreateSketchPlane(9, 9, 0)
```

```
    status = swModelDocExt.SelectByID2("Plane1", "SKETCHSURFACES", 0, 0, 0, False, 0, Nothing, 0)
```

```
    swModel.ActivateSelectedFeature
```

```
    swModel.ClearSelection2 True
```

```
    swSketchManager.InsertSketch True
```

```
End Sub
```

Create 3D Sketch Plane Example (VB.NET)

This example shows how to create a 3D sketch plane.

```

'-----
' Preconditions: Verify that the specified part template exists.
'
' Postconditions:
' 1. Inserts a 3D sketch of two lines.
' 2. Inserts a 2D sketch of a circle.
' 3. Selects a line in the 3D sketch and the center of the circle
'    in the 2D sketch.
' 4. Inserts a 3D sketch plane.
' 5. Examine the graphics area and the FeatureManager design tree.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

```

Partial Class SolidWorksMacro

```

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchManager As SketchManager
        Dim swSketchSegment As SketchSegment
        Dim swSketch As Sketch
        Dim status As Boolean

        'Open new part document
        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SolidWorks 2015\templates\Part.prtdot", 0, 0, 0)

        'Insert 3D sketch of two lines
        swSketchManager = swModel.SketchManager
        swSketchManager.Insert3DSketch(True)
        swSketchSegment = swSketchManager.CreateCenterLine(-0.082642, 0.005659, 0.0#, -0.049926, 0.045073, 0.0#)
        swSketch = swSketchManager.ActiveSketch
        status = swSketch.SetWorkingPlaneOrientation(0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0)
        swSketchSegment = swSketchManager.CreateCenterLine(-0.049926, 0.045073, 0.0#, -0.049926, -0.022634, -0.065874)
        swSketch = swSketchManager.ActiveSketch
        status = swSketch.SetWorkingPlaneOrientation(0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0)
        swModel.ClearSelection2(True)
        swSketchManager.InsertSketch(True)

        'Insert 2D sketch of a circle
        swModel.ActivateSelectedFeature()
        swModel.ClearSelection2(True)
        swSketchManager.InsertSketch(True)
        swModelDocExt = swModel.Extension
        status = swModelDocExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        swSketchSegment = swSketchManager.CreateCircle(-0.056401, 0.005985, 0.0#, -0.054697, -0.005141, 0.0#)
        swModel.ClearSelection2(True)
        swSketchManager.InsertSketch(True)
        swModel.ClearSelection2(True)

        'Insert a 3D sketch plane
        swSketchManager.Insert3DSketch(True)
        status = swModelDocExt.SelectByID("Line1@3DSketch1", "EXTSKETCHSEGMENT", -0.0565609614209999, 0.0370796232466087, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID("Point2@Sketch1", "EXTSKETCHPOINT", -0.0564010297276809, 0.00598490302365917, 0, True, 0, Nothing, 0)
        status = swSketchManager.CreateSketchPlane(9, 9, 0)
        status = swModelDocExt.SelectByID("Plane1", "SKETCHSURFACES", 0, 0, 0, False, 0, Nothing, 0)
        swModel.ActivateSelectedFeature()
        swModel.ClearSelection2(True)
        swSketchManager.InsertSketch(True)

    End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Create 3D Spline Example (VBA)

This example shows how to create a 3D spline.

```
'-----  
' Preconditions: Open a new part.  
'  
' Postconditions:  
' 1. Creates a 3D sketch of a spline as per the  
'    specified data points.  
' 2. Examine the graphics area and FeatureManager  
'    design tree.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp                As SldWorks.SldWorks  
    Dim swModel              As SldWorks.ModelDoc2  
    Dim nPtData(11)         As Double  
    Dim vPtData              As Variant  
    Dim swSketchSeg          As SldWorks.SketchSegment  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
  
    nPtData(0) = -0.1163867779204: nPtData(1) = -0.01327060333761: nPtData(2) = 0#  
    nPtData(3) = -0.08195494223363: nPtData(4) = 0.060973042362: nPtData(5) = 0.1  
    nPtData(6) = -0.03568716302953: nPtData(7) = 0.01004261874198: nPtData(8) = 0.2  
    nPtData(9) = 0.02779653401797: nPtData(10) = 0.04160513478819: nPtData(11) = 0.3  
    vPtData = nPtData  
    swModel.Insert3DSketch2 True  
    Set swSketchSeg = swModel.CreateSpline(vPtData)  
    Debug.Assert Not swSketchSeg Is Nothing  
    swModel.Insert3DSketch2 True  
  
End Sub
```

Create 3D Contact Feature Example (VBA)

This example shows how to create a 3D Contact feature for use in motion studies.

```

'-----
' Preconditions:
' 1. Verify that the specified model document exists.
' 2. In SOLIDWORKS Design, verify that View > MotionManager is selected.
' 3. In the IDE, click Tools > References and select
'    SOLIDWORKS version MotionStudy Type library.
' 4. Click OK.
' 5. Open an Immediate window.
'
' Postconditions:
' 1. Selects the faces of the components to study.
' 2. Sets the static contact friction option.
' 3. Adds Solid Body Contact1 to the MotionManager design tree on the
'    Motion Study 1 tab.
' 4. Inspect the Immediate window.
'
' NOTE: Because the model is used elsewhere,
' do not save changes when closing it.
'-----

```

Option Explicit

```
Sub main()
```

```

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swModelDocExt As SldWorks.ModelDocExtension
    Dim swMotionMgr As SwMotionStudy.MotionStudyManager
    Dim swMotionStudy1 As SwMotionStudy.MotionStudy
    Dim swContFeat As SldWorks.Simulation3DContactFeatureData
    Dim boolstatus As Boolean
    Dim swFeat As SldWorks.Feature
    Dim longstatus As Long, longwarnings As Long

    Set swApp = Application.SldWorks
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\api\beam_boltconnection.sldasm", 2, 0, "", longstatus, longwarnings)
    swApp.ActivateDoc2 "beam_boltconnection", False, longstatus
    Set swModel = swApp.ActiveDoc

    Set swModelDocExt = swModel.Extension
    Set swMotionMgr = swModelDocExt.GetMotionStudyManager()
    Set swMotionStudy1 = swMotionMgr.GetMotionStudy("Motion Study 1")

    If (swMotionStudy1 Is Nothing) Then
        Debug.Print "Motion Study 1 is not available"
    End If

    swMotionStudy1.StudyType = swMotionStudyType_e.swMotionStudyTypePhysicalSimulation
    ' Activate Motion Study 1
    swMotionStudy1.Activate

    ' Create the feature data object
    Set swContFeat = swMotionStudy1.CreateDefinition(swFmAEM3DContact)

    If swContFeat Is Nothing Then
        Debug.Print "Failed to create feature data object"
    End If

    Dim swSelMgr As SldWorks.SelectionMgr
    Set swSelMgr = swModel.SelectionManager

    ' Select the faces of the components to study
    boolstatus = swModelDocExt.SelectByID2("", "FACE", -0.131022323560273, 1.29484035505811E-02, 6.10848486814461E-02, False, 0,
Nothing, 0)
    boolstatus = swModelDocExt.SelectByID2("", "FACE", 0.08742646002122, 1.52756897197435E-02, 6.10848486815598E-02, True, 0,
Nothing, 0)

    ' Set the 3D Contact components
    Dim ContactObj(1) As Object
    Set ContactObj(0) = swSelMgr.GetSelectedObject6(1, -1)
    Set ContactObj(1) = swSelMgr.GetSelectedObject6(2, -1)

    Dim vContact As Variant
    vContact = ContactObj
    swContFeat.ContactComponents = vContact

    ' Set the static contact friction option
    swContFeat.FrictionOption = swMotionContactFrictionFull

    ' Create a 3D Contact feature
    Set swFeat = swMotionStudy1.CreateFeature(swContFeat)

    If swFeat Is Nothing Then
        Debug.Print "Failed to create feature"
    Else

```

```

Debug.Print swFeat.Name
If swContFeat.UseRestitutionCoefficient Then
    Debug.Print "    Use coefficient of restitution:"
    Debug.Print "        Coefficient of restitution: " & swContFeat.RestitutionCoefficient
Else
    Debug.Print "    Use impact force:"
    Debug.Print "        Exponent of exponential force: " & swContFeat.Exponent
    Debug.Print "        Maximum damping coefficient: " & swContFeat.MaxDamping
    Debug.Print "        Penetration at which maximum damping is applied: " & swContFeat.Penetration
    Debug.Print "        Stiffness of material at boundary of interaction: " & swContFeat.Stiffness
End If
If swContFeat.SpecifyMaterial Then
    Debug.Print "    Type of material as defined by swCosmosWorksMat:"
    Debug.Print "        First object: " & swContFeat.MaterialID(0)
    Debug.Print "        Second object: " & swContFeat.MaterialID(1)
End If
If swContFeat.FrictionOption = swMotionContactFrictionType_e.swMotionContactFrictionFull Then
    Debug.Print "    Static contact friction:"
    Debug.Print "        Coefficient: " & swContFeat.StaticFrictionCoefficient
    Debug.Print "        Velocity: " & swContFeat.StaticFrictionVelocity
Else
    Debug.Print "    Dynamic contact friction:"
    Debug.Print "        Coefficient: " & swContFeat.DynamicFrictionCoefficient
    Debug.Print "        Velocity: " & swContFeat.DynamicFrictionVelocity
End If

End If

End Sub

```

Create a Callout Independent of a Selection Example (C#)

This example shows how to create a callout that is independent of a selection.

```
//-----
// Preconditions:
// 1. Open a model document.
// 2. Click Project > Add Reference > SolidWorks.interop.swpublished
//    on the .NET tab.
// 3. Rename the namespace of this macro to match the name of your C# project.
//
// Postconditions:
// 1. Creates a callout with the specified properties.
// 2. Inspect the Immediate window for its text format properties.
//-----
using System;
using System.Collections.Generic;
using System.Text;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swpublished;
using System.Diagnostics;
namespace CreateCallout_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        ModelDocExtension swExt;
        SelectionMgr swSelMgr;
        MathUtility mathUtil;
        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swExt = swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            mathUtil = (MathUtility)swApp.GetMathUtility();
            calloutHandler handle = new calloutHandler();
            MathPoint mp;
            double[] vPnt = new double[3];
            vPnt[0] = 0.0;
            vPnt[1] = 0.0;
            vPnt[2] = 0.0;
            mp = (MathPoint)mathUtil.CreatePoint(vPnt);
            Callout myCallout;
            myCallout = swExt.CreateCallout(2, handle);
            myCallout.set_Value(1, "");
            myCallout.set_IgnoreValue(1, true);
            myCallout.set_Label2(1, "SldWorks API");
            myCallout.SkipColon = true;
            myCallout.SetLeader(true, true);
            myCallout.SetTargetPoint(1, 0.001, 0.001, 0);
            myCallout.SetTargetPoint(2, -0.001, 0.001, 0);
            myCallout.Position = mp;
            myCallout.set_ValueInactive(0, true);
            myCallout.TextBox = false;
            myCallout.Display(true);

            TextFormat swTextFormat = myCallout.TextFormat;
            ProcessTextFormat(swApp, swModel, swTextFormat);
        }

        public void ProcessTextFormat(SldWorks swApp, ModelDoc2 swModel, TextFormat swTextFormat)
        {
            Debug.Print("    Backwards          = " + swTextFormat.Backwards);
            Debug.Print("    Bold               = " + swTextFormat.Bold);
            Debug.Print("    CharHeight         = " + swTextFormat.CharHeight);
            Debug.Print("    CharHeightInPts    = " + swTextFormat.CharHeightInPts);
            Debug.Print("    CharSpacingFactor  = " + swTextFormat.CharSpacingFactor);
            Debug.Print("    Escapement         = " + swTextFormat.Escapement);
            Debug.Print("    IsHeightSpecifiedInPts = " + swTextFormat.IsHeightSpecifiedInPts());
            Debug.Print("    Italic             = " + swTextFormat.Italic);
            Debug.Print("    LineLength         = " + swTextFormat.LineLength);
            Debug.Print("    LineSpacing        = " + swTextFormat.LineSpacing);
            Debug.Print("    ObliqueAngle       = " + swTextFormat.ObliqueAngle);
            Debug.Print("    Strikeout          = " + swTextFormat.Strikeout);
            Debug.Print("    TypeFaceName       = " + swTextFormat.TypeFaceName);
            Debug.Print("    Underline          = " + swTextFormat.Underline);
            Debug.Print("    UpsideDown         = " + swTextFormat.UpsideDown);
            Debug.Print("    Vertical           = " + swTextFormat.Vertical);
            Debug.Print("    WidthFactor        = " + swTextFormat.WidthFactor);

            Debug.Print("");
        }
    }

    public SldWorks swApp;
}
[System.Runtime.InteropServices.ComVisible(true)]
public class calloutHandler : SwCalloutHandler
```

```
{
    #region ISwCalloutHandler Members
    bool ISwCalloutHandler.OnStringValueChanged(object pManipulator, int RowID, string Text)
    {
        Debug.Print("Text: " + Text);
        Debug.Print("Row: " + RowID.ToString());
        return true;
    }
    #endregion
}
```

Create a Callout Independent of a Selection Example (VBA)

This example shows how to create a callout that is independent of a selection.

```

'-----
' Preconditions:
' 1. Document is open in SOLIDWORKS
' 2. Select Tools > References > SOLIDWORKS <version> exposed type libraries for add-in use.
' 3. Copy Class1 to a class module named Class1.
'
' Postconditions:
' 1. A callout is created with the specified properties.
' 2. Inspect the Immediate window for its text format properties.
'-----

```

Option Explicit

```

Dim swApp As SldWorks.SldWorks
Dim swModel As ModelDoc2
Dim swExt As ModelDocExtension
Dim swSelMgr As SelectionMgr
Dim mathUtil As MathUtility
Dim swTextFormat As TextFormat
Dim handle As New Class1
Dim mp As MathPoint
Dim vPnt(2) As Double
Dim myCallout As Callout

Sub main()
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager
    Set mathUtil = swApp.GetMathUtility

    vPnt(0) = 0#
    vPnt(1) = 0#
    vPnt(2) = 0#
    Set mp = mathUtil.CreatePoint(vPnt)

    Set myCallout = swExt.CreateCallout(2, handle)
    myCallout.Value(1) = ""
    myCallout.IgnoreValue(1) = True
    myCallout.Label2(1) = "SldWorks API"
    myCallout.SkipColon = True
    myCallout.FontSize = 10
    Call myCallout.SetLeader(True, True)
    Call myCallout.SetTargetPoint(1, 0.001, 0.001, 0)
    Call myCallout.SetTargetPoint(2, -0.001, 0.001, 0)
    myCallout.Position = mp
    myCallout.ValueInactive(0) = True
    myCallout.TextBox = False
    myCallout.Display (True)

    Set swTextFormat = myCallout.TextFormat
    ProcessTextFormat swApp, swModel, swTextFormat
End Sub

Sub ProcessTextFormat(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swTextFormat As SldWorks.TextFormat)

    Debug.Print "    Backwards          = " & swTextFormat.Backwards
    Debug.Print "    Bold              = " & swTextFormat.Bold
    Debug.Print "    CharHeight        = " & swTextFormat.CharHeight
    Debug.Print "    CharHeightInPts   = " & swTextFormat.CharHeightInPts
    Debug.Print "    CharSpacingFactor = " & swTextFormat.CharSpacingFactor
    Debug.Print "    Escapement        = " & swTextFormat.Escapement
    Debug.Print "    IsHeightSpecifiedInPts = " & swTextFormat.IsHeightSpecifiedInPts
    Debug.Print "    Italic            = " & swTextFormat.Italic
    Debug.Print "    LineLength        = " & swTextFormat.LineLength
    Debug.Print "    LineSpacing       = " & swTextFormat.LineSpacing
    Debug.Print "    ObliqueAngle      = " & swTextFormat.ObliqueAngle
    Debug.Print "    Strikeout         = " & swTextFormat.Strikeout
    Debug.Print "    TypeFaceName      = " & swTextFormat.TypeFaceName
    Debug.Print "    UnderLine         = " & swTextFormat.UnderLine
    Debug.Print "    UpsideDown        = " & swTextFormat.UpsideDown
    Debug.Print "    Vertical          = " & swTextFormat.Vertical
    Debug.Print "    WidthFactor       = " & swTextFormat.WidthFactor

    Debug.Print ""
End Sub

'
' Class1 module
'
Implements SwCalloutHandler
Private Sub Class_Initialize()
End Sub
Private Function SwCalloutHandler_OnStringValueChanged(ByVal pManipulator As Object, ByVal RowID As Long, ByVal Text As String) As Boolean
    Debug.Print ("Text: " & Text)
    Debug.Print ("Row: " & RowID)
End Function

```

```
SwCalloutHandler_OnStringValueChanged = True
```

```
End Function
```

Create a Callout Independent of a Selection Example (VB.NET)

This example shows how to create a callout that is independent of a selection.

```

'-----
' Preconditions:
' 1. Open a model document in SOLIDWORKS.
' 2. Click Project > Add Reference > SolidWorks.interop.swpublished
'    on the .NET tab.
'
' Postconditions:
' 1. Creates a callout with the specified properties.
' 2. Inspect the Immediate window for its text format properties.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swpublished
Imports System
Imports System.Diagnostics
Partial Class SolidWorksMacro
    Dim swModel As ModelDoc2
    Dim swExt As ModelDocExtension
    Dim swSelMgr As SelectionMgr
    Dim mathUtil As MathUtility
    Dim swTextFormat As TextFormat
    Public Sub main()
        swModel = swApp.ActiveDoc
        swExt = swModel.Extension
        swSelMgr = swModel.SelectionManager
        mathUtil = swApp.GetMathUtility
        Dim handle As New calloutHandler
        Dim mp As MathPoint
        Dim vPnt(2) As Double
        vPnt(0) = 0.0#
        vPnt(1) = 0.0#
        vPnt(2) = 0.0#
        mp = mathUtil.CreatePoint(vPnt)
        Dim myCallout As Callout
        myCallout = swExt.CreateCallout(2, handle)
        myCallout.Value(1) = ""
        myCallout.IgnoreValue(1) = True
        myCallout.Label2(1) = "Sldworks API"
        myCallout.SkipColon = True
        Call myCallout.SetLeader(True, True)
        Call myCallout.SetTargetPoint(1, 0.001, 0.001, 0)
        Call myCallout.SetTargetPoint(2, -0.001, 0.001, 0)
        myCallout.Position = mp
        myCallout.ValueInactive(0) = True
        myCallout.TextBox = False
        myCallout.Display(True)

        swTextFormat = myCallout.TextFormat
        ProcessTextFormat swApp, swModel, swTextFormat
    End Sub

    Sub ProcessTextFormat(swApp As SldWorks, swModel As ModelDoc2, swTextFormat As TextFormat)

        Debug.Print "    BackWards          = " & swTextFormat.BackWards
        Debug.Print "    Bold              = " & swTextFormat.Bold
        Debug.Print "    CharHeight        = " & swTextFormat.CharHeight
        Debug.Print "    CharHeightInPts   = " & swTextFormat.CharHeightInPts
        Debug.Print "    CharSpacingFactor = " & swTextFormat.CharSpacingFactor
        Debug.Print "    Escapement        = " & swTextFormat.Escapement
        Debug.Print "    IsHeightSpecifiedInPts = " & swTextFormat.IsHeightSpecifiedInPts
        Debug.Print "    Italic            = " & swTextFormat.Italic
        Debug.Print "    LineLength        = " & swTextFormat.LineLength
        Debug.Print "    LineSpacing       = " & swTextFormat.LineSpacing
        Debug.Print "    ObliqueAngle      = " & swTextFormat.ObliqueAngle
        Debug.Print "    Strikeout         = " & swTextFormat.Strikeout
        Debug.Print "    TypeFaceName      = " & swTextFormat.TypeFaceName
        Debug.Print "    Underline         = " & swTextFormat.Underline
        Debug.Print "    UpsideDown        = " & swTextFormat.UpsideDown
        Debug.Print "    Vertical          = " & swTextFormat.Vertical
        Debug.Print "    WidthFactor       = " & swTextFormat.WidthFactor

        Debug.Print ""
    End Sub

    Public swApp As SldWorks
End Class
' calloutHandler.vb
'
<System.Runtime.InteropServices.ComVisible(True)> _
Public Class calloutHandler
    Implements SwCalloutHandler
    Private Function OnStringValueChanged(ByVal pManipulator As Object, ByVal RowID As Integer, ByVal Text As String) As Boolean

```

```
Implements SolidWorks.Interop.swpublished.ISwCalloutHandler.OnStringValueChanged
    Debug.Print("Text: " & Text)
    Debug.Print("Row: " & RowID)
    Return True
End Function
End Class
```

Create a Multi-thickness Shell Example (VBA)

This example shows how to create a multi-thickness shell.

```
'-----  
' Preconditions: Open public_documents\samples\tutorial\api\box.sldprt.  
'  
' Postconditions:  
' 1. Marks the faces to remove.  
' 2. Marks the face to shell.  
' 3. Sets the thickness of the face to shell.  
' 4. Inserts Shell1.  
' 5. Examine the graphics area and FeatureManager design tree.  
'  
' NOTE: Because the part is used elsewhere, do not  
' save changes.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim Part As SldWorks.ModelDoc2  
    Dim boolstatus As Boolean  
  
    Set swApp = CreateObject("SldWorks.Application")  
    Set Part = swApp.ActiveDoc  
    Part.ClearSelection2 True  
  
    'Mark the faces to remove with a mark of 1  
    boolstatus = Part.Extension.SelectByRay(-7.62012496738862E-03, 2.99999999999159E-02, 2.14688454421434E-02, -0.400036026779312, -0.515038074910024,  
-0.758094294050284, 7.62535384051002E-04, 2, False, 1, 0)  
    boolstatus = Part.Extension.SelectByRay(-1.46810566606064E-02, 2.48617446113713E-02, 4.60521566346301E-02, -0.400036026779312, -0.515038074910024,  
-0.758094294050284, 7.62535384051002E-04, 2, True, 1, 0)  
    'Mark the face to shell with a mark of 2  
    boolstatus = Part.Extension.SelectByRay(4.38932166734958E-02, 2.08576911617797E-02, 2.73030283917137E-02, -0.400036026779312, -0.515038074910024,  
-0.758094294050284, 7.62535384051002E-04, 2, False, 2, 0)  
    'Set the thickness of the face  
    Part.InsertFeatureShellAddThickness 0.05  
    'Insert multi-thickness shell feature  
    Part.InsertFeatureShell 0.01, False  
  
End Sub
```

Create a Thread Feature Example (C#)

This example shows how to create and edit a thread feature.

```
//-----
// Preconditions:
// 1. Open:
//    public_documents\samples\tutorial\api\holecube.sldprt
// 2. Open an Immediate window.
//
// Postconditions:
// 1. Creates Thread1.
// 2. Modifies the start angle and overrides the pitch of Thread1.
// 3. Inspect the Immediate window.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Diagnostics;

namespace ThreadFeature_CSharp
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            Feature swFeat = default(Feature);
            FeatureManager FeatMgr = default(FeatureManager);
            ThreadFeatureData swThreadFeatData = default(ThreadFeatureData);
            Edge pEdge = default(Edge);

            bool boolstatus = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            FeatMgr = swModel.FeatureManager;

            //Create and initialize a thread feature data object
            swThreadFeatData = (ThreadFeatureData)FeatMgr.CreateDefinition((int)swFeatureNameID_e.swFmSweepThread);
            swThreadFeatData.InitializeThreadData();

            //Specify the thread method type as extrude
            swThreadFeatData.ThreadMethod = 1;

            //Specify the up-to-selection end condition
            swThreadFeatData.EndCondition = (int)swThreadEndCondition_e.swThreadEndCondition_UpToSelection;

            //Select the thread's starting edge
            boolstatus = swModel.Extension.SelectByRay(0.011047195612548, -0.0190800402080527, -0.000365739009737354, 0.164466301431
            0);
            pEdge = (Edge)((SelectionMgr)(swModel.SelectionManager)).GetSelectedObject6(1, -1);
            swThreadFeatData.Edge = pEdge;
            swModel.ClearSelection2(true);

            //Select the thread's up-to reference
            boolstatus = swModel.Extension.SelectByRay(0.00850469161018452, -0.0212858011305457, -0.0254798703094821, 0.164466301431
            0);
            pEdge = (Edge)((SelectionMgr)(swModel.SelectionManager)).GetSelectedObject6(1, -1);
            swThreadFeatData.SetEndConditionReference(pEdge);
            swModel.ClearSelection2(true);

            //Create the thread feature
            swFeat = FeatMgr.CreateFeature(swThreadFeatData);
            Debug.Print("File = " + swModel.GetPathName());

            //Access the thread feature data and print its properties
            swThreadFeatData = (ThreadFeatureData)swFeat.GetDefinition();

            swThreadFeatData.AccessSelections((ModelDoc)swModel, null);
            Debug.Print("Offset the starting location of the helix? " + swThreadFeatData.Offset);
            Debug.Print("Reverse direction of offset? " + swThreadFeatData.ReverseOffset);
            Debug.Print("Offset distance: " + swThreadFeatData.OffsetDistance);
            Debug.Print("Thread starting angle: " + swThreadFeatData.ThreadStartAngle);
            Debug.Print("End condition as defined in swThreadEndCondition_e: " + swThreadFeatData.EndCondition);
            Debug.Print("End condition offset: " + swThreadFeatData.EndConditionOffset);
            Debug.Print("Reverse end condition offset? " + swThreadFeatData.EndConditionOffsetReverse);
            Debug.Print("End condition offset distance: " + swThreadFeatData.EndConditionOffsetDistance);
            Debug.Print("Keep thread length constant? " + swThreadFeatData.MaintainThreadLength);
            Debug.Print("Thread profile type: " + swThreadFeatData.Type);
            Debug.Print("Size: " + swThreadFeatData.Size);
            Debug.Print("Diameter overridden? " + swThreadFeatData.DiameterOverride);
            Debug.Print("Diameter: " + swThreadFeatData.Diameter);
            Debug.Print("Pitch overridden? " + swThreadFeatData.PitchOverride);
            Debug.Print("Pitch: " + swThreadFeatData.Pitch);
        }
    }
}
```

```

Debug.Print("Thread method as defined in swThreadMethod_e: " + swThreadFeatData.ThreadMethod);
Debug.Print("Flip the profile of the helix about an axis? " + swThreadFeatData.MirrorProfile);
Debug.Print("How to flip the profile of the helix as defined in swThreadMirrorType_e: " + swThreadFeatData.MirrorType);
Debug.Print("Helix rotation angle: " + swThreadFeatData.RotationAngle);
Debug.Print("Thread wind direction (true = clockwise, false = counterclockwise): " + swThreadFeatData.RightHanded);
Debug.Print("Multiple thread starts? " + swThreadFeatData.MultipleStart);
if (swThreadFeatData.MultipleStart == true)
{
    Debug.Print(" Number of starts? " + swThreadFeatData.NumberOfStarts);
}
Debug.Print("Trim with the end face? " + swThreadFeatData.TrimEndFace);
Debug.Print("Trim with the start face? " + swThreadFeatData.TrimStartFace);

//Change the thread start angle and pitch
swThreadFeatData.ThreadStartAngle = 0.04;
swThreadFeatData.PitchOverride = true;
swThreadFeatData.Pitch = 0.01

//Modify the feature definition
swFeat.ModifyDefinition(swThreadFeatData, swModel, null);
}

// The SldWorks swApp variable is pre-assigned for you.
public SldWorks swApp;

```

```

}
}

```

Create a Thread Feature Example (VBA)

This example shows how to create and edit a thread feature.

```

'-----
' Preconditions:
' 1. Open:
'    public_documents\samples\tutorial\api\holecube.sldprt
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates Thread1.
' 2. Modifies the start angle and overrides the pitch of Thread1.
' 3. Inspect the Immediate window.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----

```

Option Explicit

```
Sub main()
```

```

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swFeat As SldWorks.Feature
    Dim featMgr As SldWorks.FeatureManager
    Dim swThreadFeatData As SldWorks.ThreadFeatureData
    Dim pEdge As SldWorks.Edge
    Dim boolstatus As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    Set featMgr = swModel.FeatureManager

    'Create and initialize a thread feature data object
    Set swThreadFeatData = featMgr.CreateDefinition(swFmSweepThread)
    swThreadFeatData.InitializeThreadData

    'Specify the thread method type as extrude
    swThreadFeatData.ThreadMethod = swThreadMethod_Extrude

    'Specify the up-to-selection end condition
    swThreadFeatData.EndCondition = swThreadEndCondition_UpToSelection

    'Select the thread's starting edge
    boolstatus = swModel.Extension.SelectByRay(0.011047195612548, -1.90800402080527E-02, -3.65739009737354E-04, 0.164466301431523,
    -0.479983539625146, -0.861723063044243, 1.60036844432164E-03, 1, False, 1, 0)
    Set pEdge = swModel.SelectionManager.GetSelectedObject6(1, -1)
    swThreadFeatData.Edge = pEdge
    swModel.ClearSelection2 True

    'Select the thread's up-to reference
    boolstatus = swModel.Extension.SelectByRay(8.50469161018452E-03, -2.12858011305457E-02, -2.54798703094821E-02,
    0.164466301431523, -0.479983539625146, -0.861723063044243, 1.60036844432164E-03, 1, True, 1, 0)
    Set pEdge = swModel.SelectionManager.GetSelectedObject6(1, -1)
    swThreadFeatData.SetEndConditionReference pEdge
    swModel.ClearSelection2 True

    'Create the thread feature
    Set swFeat = featMgr.CreateFeature(swThreadFeatData)

    Debug.Print "File = " & swModel.GetPathName

    'Access the thread feature data and print its properties
    Set swThreadFeatData = swFeat.GetDefinition()

    swThreadFeatData.AccessSelections swModel, Nothing
    Debug.Print "Offset the starting location of the helix? " & swThreadFeatData.Offset
    Debug.Print "Reverse direction of offset? " & swThreadFeatData.ReverseOffset
    Debug.Print "Offset distance: " & swThreadFeatData.OffsetDistance
    Debug.Print "Thread starting angle: " & swThreadFeatData.ThreadStartAngle
    Debug.Print "End condition as defined in swThreadEndCondition_e: " & swThreadFeatData.EndCondition
    Debug.Print "End condition offset: " & swThreadFeatData.EndConditionOffset
    Debug.Print "Reverse end condition offset? " & swThreadFeatData.EndConditionOffsetReverse
    Debug.Print "End condition offset distance: " & swThreadFeatData.EndConditionOffsetDistance
    Debug.Print "Keep thread length constant? " & swThreadFeatData.MaintainThreadLength
    Debug.Print "Thread profile type: " & swThreadFeatData.Type
    Debug.Print "Size: " & swThreadFeatData.Size
    Debug.Print "Diameter overridden? " & swThreadFeatData.DiameterOverride
    Debug.Print "Diameter: " & swThreadFeatData.Diameter
    Debug.Print "Pitch overridden? " & swThreadFeatData.PitchOverride
    Debug.Print "Pitch: " & swThreadFeatData.Pitch
    Debug.Print "Thread method as defined in swThreadMethod_e: " & swThreadFeatData.ThreadMethod
    Debug.Print "Flip the profile of the helix about an axis? " & swThreadFeatData.MirrorProfile
    Debug.Print "How to flip the profile of the helix as defined in swThreadMirrorType_e: " & swThreadFeatData.MirrorType
    Debug.Print "Helix rotation angle: " & swThreadFeatData.RotationAngle
    Debug.Print "Thread wind direction (True = clockwise, False = counterclockwise): " & swThreadFeatData.RightHanded
    Debug.Print "Multiple thread starts? " & swThreadFeatData.MultipleStart
    If swThreadFeatData.MultipleStart = True Then

```

```
        Debug.Print "  Number of starts? " & swThreadFeatData.NumberOfStarts
    End If
    Debug.Print "Trim with the end face? " & swThreadFeatData.TrimEndFace
    Debug.Print "Trim with the start face? " & swThreadFeatData.TrimStartFace

    'Change the start angle and pitch of the thread
    swThreadFeatData.ThreadStartAngle = 0.04
    swThreadFeatData.PitchOverride = True
    swThreadFeatData.Pitch = 0.01

    'Modify the feature definition
    swFeat.ModifyDefinition swThreadFeatData, swModel, Nothing
End Sub
```

Create a Thread Feature Example (VB.NET)

This example shows how to create and edit a thread feature.

```
'-----
' Preconditions:
' 1. Open:
'    public_documents\samples\tutorial\api\holecube.sldprt
' 2. Open an Immediate window.
'
' Postconditions:
' 1. Creates Thread1.
' 2. Modifies the start angle and overrides the pitch of Thread1.
' 3. Inspect the Immediate window.
'
' NOTE: Because the model is used elsewhere, do not save changes.
'-----
```

```
Imports SolidWorks.Interop.sldworks
```

```
Imports SolidWorks.Interop.swconst
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()
```

```
        Dim swModel As ModelDoc2
        Dim swFeat As Feature
        Dim featMgr As FeatureManager
        Dim swThreadFeatData As ThreadFeatureData
        Dim pEdge As Edge

        Dim boolstatus As Boolean

        swModel = swApp.ActiveDoc
        featMgr = swModel.FeatureManager

        'Create and initialize a thread feature data object
        swThreadFeatData = featMgr.CreateDefinition(swFeatureNameID_e.swFmSweepThread)
        swThreadFeatData.InitializeThreadData

        'Specify the thread method type
        swThreadFeatData.ThreadMethod = 1

        'Specify the up-to-selection end condition
        swThreadFeatData.EndCondition = swThreadEndCondition_e.swThreadEndCondition_UpToSelection

        'Select the thread's starting edge
        boolstatus = swModel.Extension.SelectByRay(0.011047195612548, -0.0190800402080527, -0.000365739009737354, 0.164466301431523,
        pEdge = swModel.SelectionManager.GetSelectedObject6(1, -1)
        swThreadFeatData.Edge = pEdge
        swModel.ClearSelection2(True)

        'Select the thread's up-to reference
        boolstatus = swModel.Extension.SelectByRay(0.00850469161018452, -0.0212858011305457, -0.0254798703094821, 0.164466301431523,
        pEdge = swModel.SelectionManager.GetSelectedObject6(1, -1)
        swThreadFeatData.SetEndConditionReference(pEdge)
        swModel.ClearSelection2(True)

        'Create the thread feature
        swFeat = featMgr.CreateFeature(swThreadFeatData)
        Debug.Print("File = " & swModel.GetPathName)

        'Access the thread feature data and print its properties
        swThreadFeatData = swFeat.GetDefinition()

        swThreadFeatData.AccessSelections(swModel, Nothing)
        Debug.Print("Offset the starting location of the helix? " & swThreadFeatData.Offset)
        Debug.Print("Reverse direction of offset? " & swThreadFeatData.ReverseOffset)
        Debug.Print("Offset distance: " & swThreadFeatData.OffsetDistance)
        Debug.Print("Thread starting angle: " & swThreadFeatData.ThreadStartAngle)
        Debug.Print("End condition as defined in swThreadEndCondition_e: " & swThreadFeatData.EndCondition)
        Debug.Print("End condition offset: " & swThreadFeatData.EndConditionOffset)
        Debug.Print("Reverse end condition offset? " & swThreadFeatData.EndConditionOffsetReverse)
        Debug.Print("End condition offset distance: " & swThreadFeatData.EndConditionOffsetDistance)
        Debug.Print("Keep thread length constant? " & swThreadFeatData.MaintainThreadLength)
        Debug.Print("Thread profile type: " & swThreadFeatData.Type)
        Debug.Print("Size: " & swThreadFeatData.Size)
        Debug.Print("Diameter overridden? " & swThreadFeatData.DiameterOverride)
        Debug.Print("Diameter: " & swThreadFeatData.Diameter)
        Debug.Print("Pitch overridden? " & swThreadFeatData.PitchOverride)
        Debug.Print("Pitch: " & swThreadFeatData.Pitch)
        Debug.Print("Thread method as defined in swThreadMethod_e: " & swThreadFeatData.ThreadMethod)
        Debug.Print("Flip the profile of the helix about an axis? " & swThreadFeatData.MirrorProfile)
        Debug.Print("How to flip the profile of the helix as defined in swThreadMirrorType_e: " & swThreadFeatData.MirrorType)
        Debug.Print("Helix rotation angle: " & swThreadFeatData.RotationAngle)
        Debug.Print("Thread wind direction (True = clockwise, False = counterclockwise): " & swThreadFeatData.RightHanded)
```

```
Debug.Print("Multiple thread starts? " & swThreadFeatData.MultipleStart)
If swThreadFeatData.MultipleStart = True Then
    Debug.Print("  Number of starts? " & swThreadFeatData.NumberOfStarts)
End If
Debug.Print("Trim with the end face? " & swThreadFeatData.TrimEndFace)
Debug.Print("Trim with the start face? " & swThreadFeatData.TrimStartFace)
```

```
'Change the thread's start angle and pitch
swThreadFeatData.ThreadStartAngle = 0.04
swThreadFeatData.PitchOverride = True
swThreadFeatData.Pitch = 0.01
```

```
'Modify the feature definition
swFeat.ModifyDefinition(swThreadFeatData, swModel, Nothing)
```

```
End Sub
```

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

```
End Class
```

Create Advanced Hole Feature Example (C#)

This example shows how to create an Advanced Hole feature.

```
//-----
// Preconditions:
// 1. Verify that the specified part document exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Selects near and far side faces of the Advanced Hole.
// 2. Defines an Advanced Hole at a sketch point on the edge of the block
//    with the following:
//    * Countersink near side element
//    * Straight tap near side element
//    * Counterbore far side element
//    * Straight hole far side element
// 3. Gets some near and far side element data for the Advanced Hole.
// 4. Modifies the near side element array to contain a tapered tap element.
// 5. Deletes the Advanced Hole's defining sketch point on the edge of the block.
// 6. Adds two sketch points and creates two Advanced Holes at those locations
//    using the previously defined Advanced Hole.
// 7. Inspect the Immediate window and graphics area.
//
// NOTE: Because the model is used elsewhere, do not save changes.
//-----
```

```
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Diagnostics;
using System.Runtime.InteropServices;
```

```
namespace CreateAdvancedHole_CSharp
```

```
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
```

```
            ModelDoc2 swModel = default(ModelDoc2);
            Feature feat = default(Feature);
            bool boolstatus = false;
            AdvancedHoleElementData swAdvHole_Near_1 = default(AdvancedHoleElementData);
            AdvancedHoleElementData swAdvHole_Near_2 = default(AdvancedHoleElementData);
            AdvancedHoleElementData swAdvHole_Near_3 = default(AdvancedHoleElementData);
            AdvancedHoleElementData swAdvHole_Far_1 = default(AdvancedHoleElementData);
            AdvancedHoleElementData swAdvHole_Far_2 = default(AdvancedHoleElementData);
            CountersinkElementData swCounterSinkNear = default(CountersinkElementData);
            CounterboreElementData swCounterBoreFar = default(CounterboreElementData);
            StraightElementData swStraightHoleFar = default(StraightElementData);
            StraightTapElementData swStraightTapNear = default(StraightTapElementData);
            TaperedTapElementData swTaperedTapNear = default(TaperedTapElementData);
            FeatureManager swFeatureMgr = default(FeatureManager);
            double ConvFactorLength = 0;
            AdvancedHoleElementData[] advHoleNearArr = new AdvancedHoleElementData[2];
            AdvancedHoleElementData[] advHoleFarArr = new AdvancedHoleElementData[2];
            AdvancedHoleFeatureData featdata = default(AdvancedHoleFeatureData);
            AdvancedHoleElementData[] newNearArr = new AdvancedHoleElementData[2];
            AdvancedHoleElementData[] newFarArr = new AdvancedHoleElementData[2];
            Feature swSketchFeature = default(Feature);
            SelectionMgr swSelectionManager = default(SelectionMgr);
            Sketch swSketch = default(Sketch);
            object[] swSketchPointArray = null;
            int swMaxPointNumber = 0;
            object skPoint = null;
            object swSketchPoint = null;
            int swCurrentPointNumber = 0;
            int errors = 0;
            int warnings = 0;
            object ResultArray = null;
            string CalloutString = null;
            string StrDiam = null;
            string strDepth = null;
            object[] nearSide = null;
            object[] farSide = null;

            bool retVal = false;
```

```
            swModel = swApp.OpenDoc6("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\block20.sld
            swFeatureMgr = swModel.FeatureManager;
```

```
            //Conversion from inches to meters
            ConvFactorLength = 25.4 / 1000;
```

```

//Select two faces for the near side and far side hole elements
boolstatus = swModel.Extension.SelectByRay(-0.0589202612791269, 0.0260626824463657, 0.05600000000000969, -0.4000360267793
0);
//Near side
boolstatus = swModel.Extension.SelectByRay(-0.0110716643645077, 0.0211784489308116, -0.0639370439421896, 0.1826195396635
0);
//Far side

//Define near and far side hole elements

//Near side countersink
swAdvHole_Near_1 = (AdvancedHoleElementData)swModel.Extension.CreateAdvancedHoleElementData((int)swAdvWzdGeneralHoleType

swAdvHole_Near_1.Orientation = (int)swHoleElementOrientation_e.swHoleElementOrientation_Nearside;
swAdvHole_Near_1.Size = "#4";
swAdvHole_Near_1.Standard = 0;
swAdvHole_Near_1.FastenerType = (int)swWzdHoleStandardFastenerTypes_e.swStandardAnsiInchFlatHead100;
swAdvHole_Near_1.Diameter = ConvFactorLength * 0.225;
swAdvHole_Near_1.BlindDepth = 0.02055794 * ConvFactorLength;
swAdvHole_Near_1.EndCondition = (int)swEndConditions_e.swEndCondBlind;
swCounterSinkNear = (CountersinkElementData)swAdvHole_Near_1;
swCounterSinkNear.EndConditionOverride = true;
swCounterSinkNear.AngleOverride = false;

//Near side straight tap
swAdvHole_Near_2 = (AdvancedHoleElementData)swModel.Extension.CreateAdvancedHoleElementData((int)swAdvWzdGeneralHoleType

swAdvHole_Near_2.Size = "#4-40";
swAdvHole_Near_2.Standard = (int)swWzdHoleStandards_e.swStandardAnsiInch;
swAdvHole_Near_2.FastenerType = (int)swWzdHoleStandardFastenerTypes_e.swStandardAnsiInchBottomingTappedHole;
swAdvHole_Near_2.Diameter = ConvFactorLength * 0.089;
swAdvHole_Near_2.EndCondition = (int)swEndConditions_e.swEndCondUpToNext;
swAdvHole_Near_2.DiameterOverride = true;
swStraightTapNear = (StraightTapElementData)swAdvHole_Near_2;
swStraightTapNear.CustomSizing = (int)swStraightTapHoleCustomSizing_e.swStraightTapHoleCustomSizing_TapDrillDiameter;
swStraightTapNear.ThreadClass = (int)swStraightTapHoleThreadClass_e.swStraightTapHoleThreadClass_1B;
swStraightTapNear.ThreadClassOverride = true;

//Near side tapered tap
swAdvHole_Near_3 = (AdvancedHoleElementData)swModel.Extension.CreateAdvancedHoleElementData((int)swAdvWzdGeneralHoleType

swAdvHole_Near_3.Orientation = (int)swHoleElementOrientation_e.swHoleElementOrientation_Nearside;
swAdvHole_Near_3.Size = "1/16";
swAdvHole_Near_3.Standard = (int)swWzdHoleStandards_e.swStandardAnsiInch;
swAdvHole_Near_3.FastenerType = (int)swWzdHoleStandardFastenerTypes_e.swStandardAnsiInchTaperedPipeTap;
swAdvHole_Near_3.Diameter = ConvFactorLength * 0.266;
swAdvHole_Near_3.BlindDepth = 0.205 * ConvFactorLength;
swAdvHole_Near_3.EndCondition = (int)swEndConditions_e.swEndCondBlind;
swAdvHole_Near_3.DiameterOverride = true;
swTaperedTapNear = (TaperedTapElementData)swAdvHole_Near_3;
swTaperedTapNear.CustomSizing = (int)swTaperedTapCustomSizing_e.swTaperedTapCustomSizing_MinorDiameterWithCosmeticThreac
swTaperedTapNear.ThreadClass = (int)swTaperedTapThreadClass_e.swTaperedTapThreadClass_1;
swTaperedTapNear.ThreadClassOverride = true;
swTaperedTapNear.EndConditionOverride = true;

//Far side counterbore
swAdvHole_Far_1 = (AdvancedHoleElementData)swModel.Extension.CreateAdvancedHoleElementData((int)swAdvWzdGeneralHoleTypes

swAdvHole_Far_1.Orientation = (int)swHoleElementOrientation_e.swHoleElementOrientation_Farside;
swAdvHole_Far_1.Size = "#8";
swAdvHole_Far_1.Standard = (int)swWzdHoleStandards_e.swStandardAnsiInch;
swAdvHole_Far_1.FastenerType = (int)swWzdHoleStandardFastenerTypes_e.swStandardAnsiInchBinding;
swAdvHole_Far_1.Diameter = ConvFactorLength * 0.375;
swAdvHole_Far_1.BlindDepth = 0.105 * ConvFactorLength;
swAdvHole_Far_1.EndCondition = (int)swEndConditions_e.swEndCondBlind;
swAdvHole_Far_1.DiameterOverride = true;
swCounterBoreFar = (CounterboreElementData)swAdvHole_Far_1;
swCounterBoreFar.EndConditionOverride = true;

//Far side straight
swAdvHole_Far_2 = (AdvancedHoleElementData)swModel.Extension.CreateAdvancedHoleElementData((int)swAdvWzdGeneralHoleTypes

swAdvHole_Far_2.Size = "1/16";
swAdvHole_Far_2.Standard = 0;
swAdvHole_Far_2.FastenerType = (int)swWzdHoleStandardFastenerTypes_e.swStandardAnsiInchAllDrillSizes;
swAdvHole_Far_2.Diameter = ConvFactorLength * 0.0625;
swAdvHole_Far_2.BlindDepth = 0.2711 * ConvFactorLength;
swAdvHole_Far_2.EndCondition = (int)swEndConditions_e.swEndCondBlind;
swAdvHole_Far_2.DiameterOverride = true;

//Customize the hole callout for this straight element
StrDiam = swModel.Extension.GetCalloutVariableString((int)swCalloutVariable_e.swCalloutVariable_AH_Straight_Diameter);
strDepth = swModel.Extension.GetCalloutVariableString((int)swCalloutVariable_e.swCalloutVariable_AH_Straight_Depth);
CalloutString = "<MOD-DIAM> " + StrDiam + " " + "<HOLE-DEPTH> " + strDepth;
swAdvHole_Far_2.CalloutString = CalloutString;

swStraightHoleFar = (StraightElementData)swAdvHole_Far_2;

//Set near and far side element arrays
advHoleNearArr[0] = (AdvancedHoleElementData)swCounterSinkNear;
advHoleNearArr[1] = (AdvancedHoleElementData)swStraightTapNear;
advHoleFarArr[0] = (AdvancedHoleElementData)swCounterBoreFar;
advHoleFarArr[1] = (AdvancedHoleElementData)swStraightHoleFar;

```

```

DispatchWrapper[] dispArray = ObjectArrayToDispatchWrapperArray(new object[] { advHoleNearArr[0], advHoleNearArr[1] });
DispatchWrapper[] dispArray2 = ObjectArrayToDispatchWrapperArray(new object[] { advHoleFarArr[0], advHoleFarArr[1] });

//Create the Advanced Hole using the near and far side element arrays; specify to not use baseline dimensions; customize Hole Callouts
feat = swFeatureMgr.AdvancedHole2(dispArray, dispArray2, false, true, false, out ResultArray);

//Get some near and far side element data
featdata = (AdvancedHoleFeatureData)feat.GetDefinition();
featdata.AccessSelections(swModel, null);
Debug.Print("Number of near side hole elements: " + featdata.NearSideElementsCount);
Debug.Print("Number of far side hole elements: " + featdata.FarSideElementsCount);
nearSide = (object[])featdata.GetNearSideElements();
swCounterSinkNear = (CountersinkElementData)nearSide[0];
farSide = (object[])featdata.GetFarSideElements();
swCounterBoreFar = (CounterboreElementData)farSide[0];
swStraightHoleFar = (StraightElementData)farSide[1];
Debug.Print("Near side countersink:");
Debug.Print("  Hole element type as defined in swAdvWzdGeneralHoleTypes_e: " + ((AdvancedHoleElementData)swCounterSinkNear).ElementType);
Debug.Print("  Size as defined on the Advanced Hole PropertyManager page: " + ((AdvancedHoleElementData)swCounterSinkNear).Size);
Debug.Print("  Standard as defined in swWzdHoleStandards_e: " + ((AdvancedHoleElementData)swCounterSinkNear).Standard);
Debug.Print("  Fastener type as defined in swWzdHoleStandardFastenerTypes_e: " + ((AdvancedHoleElementData)swCounterSinkNear).FastenerType);
Debug.Print("  Diameter in m: " + ((AdvancedHoleElementData)swCounterSinkNear).Diameter);
Debug.Print("  Blind depth in m: " + ((AdvancedHoleElementData)swCounterSinkNear).BlindDepth);
Debug.Print("  Orientation as defined in swHoleElementOrientation_e: " + ((AdvancedHoleElementData)swCounterSinkNear).Orientation);
Debug.Print("  End condition as defined in swEndConditions_e: " + ((AdvancedHoleElementData)swCounterSinkNear).EndCondition);
Debug.Print("Far side straight:");
Debug.Print("  Hole element type as defined in swAdvWzdGeneralHoleTypes_e: " + ((AdvancedHoleElementData)swStraightHoleFar).ElementType);
Debug.Print("  Size as defined on the Advanced Hole PropertyManager page: " + ((AdvancedHoleElementData)swStraightHoleFar).Size);
Debug.Print("  Standard as defined in swWzdHoleStandards_e: " + ((AdvancedHoleElementData)swStraightHoleFar).Standard);
Debug.Print("  Fastener type as defined in swWzdHoleStandardFastenerTypes_e: " + ((AdvancedHoleElementData)swStraightHoleFar).FastenerType);
Debug.Print("  Diameter in m: " + ((AdvancedHoleElementData)swStraightHoleFar).Diameter);
Debug.Print("  Diameter override? " + ((AdvancedHoleElementData)swStraightHoleFar).DiameterOverride);
Debug.Print("  Blind depth in m: " + ((AdvancedHoleElementData)swStraightHoleFar).BlindDepth);
Debug.Print("  End condition as defined in swEndConditions_e: " + ((AdvancedHoleElementData)swStraightHoleFar).EndCondition);
Debug.Print("  Customized hole callout: " + ((AdvancedHoleElementData)swStraightHoleFar).CalloutString);

//Modify the near and far side element arrays
newNearArr[0] = (AdvancedHoleElementData)swTaperedTapNear;
newNearArr[1] = (AdvancedHoleElementData)swStraightTapNear;
//newFarArr[0] = (AdvancedHoleElementData)swCounterBoreFar;
//newFarArr[1] = (AdvancedHoleElementData)swStraightHoleFar;
featdata.SetNearSideElements(newNearArr);
//featdata.SetFarSideElements(newFarArr);
feat.ModifyDefinition(featdata, swModel, null);

//Delete the first point used to define the Advanced Hole
swSketchFeature = (Feature)feat.GetFirstSubFeature();
swSketchFeature.Select2(false, 0);
swModel.EditSketch();
swSelectionManager = (SelectionMgr)swModel.SelectionManager;
swSketch = (Sketch)swSketchFeature.GetSpecificFeature2();
swSketchPointArray = (object[])swSketch.GetSketchPoints2();
swMaxPointNumber = swSketchPointArray.GetUpperBound(0);
for (swCurrentPointNumber = 0; swCurrentPointNumber <= swMaxPointNumber; swCurrentPointNumber += 1)
{
    swSketchPoint = swSketchPointArray[swCurrentPointNumber];
    retVal = swSelectionManager.AddSelectionListObject(swSketchPoint, null);
    swModel.EditDelete();
}

//Create points for multiple Advanced Hole locations

```

```

        skPoint = swModel.SketchManager.CreatePoint(-0.0319158789518497, 0.0344489966898323, 0.056000000000004);
        skPoint = swModel.SketchManager.CreatePoint(-0.0494104502066557, 0.0080156770060853, 0.0560000000000969);

        swModel.SketchManager.InsertSketch(true);
    }

    public Dispatcher[] ObjectArrayToDispatcherArray(object[] Objects)
    {
        int ArraySize = 0;
        ArraySize = Objects.GetUpperBound(0);
        Dispatcher[] d = new Dispatcher[ArraySize + 1];
        int ArrayIndex = 0;
        for (ArrayIndex = 0; ArrayIndex <= ArraySize; ArrayIndex++)
        {
            d[ArrayIndex] = new Dispatcher(Objects[ArrayIndex]);
        }
        return d;
    }

    // The SldWorks swApp variable is pre-assigned for you.
    public SldWorks swApp;

}
}

```